

ZlibValidation

Generated by Doxygen 1.9.5

1 ZlibValidation	1
1.1 Description	1
1.2 Usage	1
1.3 Development Diary	1
1.3.1 2025-01-27	1
1.3.2 2025-01-28	1
1.3.3 2025-02-01	2
1.3.4 2025-02-10	2
1.3.5 2025-02-11	2
1.3.6 2025-02-14	2
1.3.7 2025-02-15	2
1.3.8 2025-02-17	2
1.3.9 2025-02-18	3
1.3.10 2025-02-19	3
1.3.11 2025-02-20	3
1.3.12 2025-02-25	3
1.3.13 2025-02-26	3
1.3.14 2025-02-27	3
1.3.15 2025-02-28	3
1.3.16 2025-03-01	4
1.3.17 2025-03-07	4
1.3.18 2025-03-10	4
1.3.19 2025-03-14	4
1.3.20 2025-03-15	4
1.3.21 2025-03-16	5
1.3.22 2025-03-17	5
1.3.23 2025-03-18	5
1.3.24 2025-03-19	6
1.3.25 2025-03-21	6
1.3.26 2025-03-24	6
1.3.27 2025-03-25	6
1.3.28 2025-03-26	6
2 Hierarchical Index	7
2.1 Class Hierarchy	7
3 Class Index	9
3.1 Class List	9
4 File Index	11
4.1 File List	11
5 Class Documentation	13
5.1 AttributesIterator Class Reference	13

5.1.1 Detailed Description	14
5.1.2 Constructor & Destructor Documentation	14
5.1.2.1 AttributesIterator()	14
5.1.2.2 ~AttributesIterator()	14
5.1.3 Member Function Documentation	14
5.1.3.1 end()	14
5.1.3.2 get()	15
5.1.3.3 next()	15
5.1.4 Member Data Documentation	16
5.1.4.1 attr_	16
5.1.4.2 attrs_	16
5.1.4.3 err_	16
5.2 CellExtractor Class Reference	16
5.2.1 Detailed Description	17
5.2.2 Constructor & Destructor Documentation	17
5.2.2.1 CellExtractor()	17
5.2.3 Member Function Documentation	17
5.2.3.1 foundTargetCell()	18
5.2.3.2 handle()	18
5.2.4 Member Data Documentation	18
5.2.4.1 foundTarget_	18
5.2.4.2 targetCell_	18
5.3 CellPrinter Class Reference	19
5.3.1 Detailed Description	19
5.3.2 Constructor & Destructor Documentation	20
5.3.2.1 CellPrinter()	20
5.3.3 Member Function Documentation	20
5.3.3.1 handle()	20
5.3.4 Member Data Documentation	20
5.3.4.1 foundTarget_	20
5.3.4.2 out_	20
5.3.4.3 targetCell_	21
5.4 GroupsIterator Class Reference	21
5.4.1 Detailed Description	21
5.4.2 Constructor & Destructor Documentation	22
5.4.2.1 GroupsIterator()	22
5.4.2.2 ~GroupsIterator()	22
5.4.3 Member Function Documentation	22
5.4.3.1 end()	22
5.4.3.2 get()	23
5.4.3.3 next()	23
5.4.4 Member Data Documentation	23

5.4.4.1 err_	23
5.4.4.2 group_	24
5.4.4.3 groups_	24
5.5 LibAttribute Class Reference	24
5.5.1 Detailed Description	25
5.5.2 Constructor & Destructor Documentation	25
5.5.2.1 LibAttribute()	25
5.5.2.2 ~LibAttribute()	26
5.5.3 Member Function Documentation	26
5.5.3.1 getBoolean()	26
5.5.3.2 getFloat()	27
5.5.3.3 getInt()	27
5.5.3.4 getName()	28
5.5.3.5 getString()	28
5.5.3.6 getValues()	29
5.5.3.7 isComplex()	29
5.5.4 Member Data Documentation	29
5.5.4.1 attr_	29
5.5.4.2 err_	30
5.6 liberty_value_data Struct Reference	30
5.6.1 Detailed Description	30
5.6.2 Member Data Documentation	30
5.6.2.1 dim_sizes	30
5.6.2.2 dimensions	30
5.6.2.3 index_info	31
5.6.2.4 values	31
5.7 LibFile Class Reference	31
5.7.1 Detailed Description	32
5.7.2 Constructor & Destructor Documentation	32
5.7.2.1 LibFile()	32
5.7.2.2 ~LibFile()	32
5.7.3 Member Function Documentation	32
5.7.3.1 checkTimingArcMonotonicity()	32
5.7.3.2 modify()	33
5.7.3.3 mono()	34
5.7.3.4 parse()	36
5.7.3.5 read()	38
5.7.3.6 supercell()	39
5.7.3.7 verilog()	41
5.7.3.8 writeJsonToFile()	43
5.7.4 Member Data Documentation	43
5.7.4.1 basename_	44

5.7.4.2 err_	44
5.7.4.3 filename_	44
5.7.4.4 filepath_	44
5.7.4.5 jsonname_	44
5.7.4.6 lib_json_	44
5.7.4.7 libname_	45
5.7.4.8 logger_	45
5.7.4.9 loggername_	45
5.7.4.10 process_	45
5.7.4.11 temperature_	45
5.7.4.12 voltage_	45
5.8 LibGroup Class Reference	46
5.8.1 Detailed Description	46
5.8.2 Constructor & Destructor Documentation	46
5.8.2.1 LibGroup()	47
5.8.2.2 ~LibGroup()	47
5.8.3 Member Function Documentation	47
5.8.3.1 getAttrs()	47
5.8.3.2 getGroups()	48
5.8.3.3 getName()	48
5.8.3.4 getType()	49
5.8.4 Member Data Documentation	49
5.8.4.1 err_	49
5.8.4.2 group_	49
5.9 LibraryComparator Class Reference	50
5.9.1 Detailed Description	50
5.9.2 Constructor & Destructor Documentation	50
5.9.2.1 LibraryComparator()	51
5.9.3 Member Function Documentation	52
5.9.3.1 compareCell()	52
5.9.3.2 compareLut()	53
5.9.3.3 comparePin()	54
5.9.3.4 compareTimingArc()	55
5.9.3.5 generateReport()	56
5.9.4 Member Data Documentation	57
5.9.4.1 abstol_	57
5.9.4.2 comp_json_	57
5.9.4.3 comp_lib_path_	58
5.9.4.4 ref_json_	58
5.9.4.5 ref_lib_path_	58
5.9.4.6 reltol_	58
5.10 si2drExprT Struct Reference	58

5.10.1 Detailed Description	59
5.10.2 Member Data Documentation	59
5.10.2.1 b	59
5.10.2.2 d	59
5.10.2.3 i	59
5.10.2.4 left	60
5.10.2.5 right	60
5.10.2.6 s	60
5.10.2.7 type	60
5.10.2.8	60
5.10.2.9 valuetype	60
5.11 si2OID Struct Reference	61
5.11.1 Detailed Description	61
5.11.2 Member Data Documentation	61
5.11.2.1 v1	61
5.11.2.2 v2	61
5.12 ValuesIterator Class Reference	61
5.12.1 Detailed Description	62
5.12.2 Constructor & Destructor Documentation	62
5.12.2.1 ValuesIterator()	62
5.12.2.2 ~ValuesIterator()	63
5.12.3 Member Function Documentation	63
5.12.3.1 end()	63
5.12.3.2 next()	63
5.12.4 Member Data Documentation	63
5.12.4.1 bool_	64
5.12.4.2 err_	64
5.12.4.3 exprp_	64
5.12.4.4 float_	64
5.12.4.5 int_	64
5.12.4.6 str_	64
5.12.4.7 values_	65
5.12.4.8 vtype_	65
5.13 VerilogVisitor Class Reference	65
5.13.1 Detailed Description	66
5.13.2 Constructor & Destructor Documentation	66
5.13.2.1 VerilogVisitor()	66
5.13.3 Member Function Documentation	66
5.13.3.1 handle() [1/6]	66
5.13.3.2 handle() [2/6]	67
5.13.3.3 handle() [3/6]	67
5.13.3.4 handle() [4/6]	67

5.13.3.5 handle() [5 / 6]	67
5.13.3.6 handle() [6 / 6]	68
5.13.4 Member Data Documentation	68
5.13.4.1 depth_	68
5.13.4.2 inTargetModule_	68
5.13.4.3 targetCell_	68
6 File Documentation	69
6.1 include/compare.hpp File Reference	69
6.1.1 Typedef Documentation	70
6.1.1.1 json	70
6.2 compare.hpp	70
6.3 include/iterators.hpp File Reference	71
6.4 iterators.hpp	71
6.5 include/json_utils.hpp File Reference	72
6.5.1 Typedef Documentation	74
6.5.1.1 json	74
6.5.2 Function Documentation	74
6.5.2.1 generateCellJson()	74
6.5.2.2 generateLutJson()	76
6.5.2.3 generatePinJson()	77
6.5.2.4 generatePowerJson()	80
6.6 json_utils.hpp	81
6.7 include/lib_attribute.hpp File Reference	82
6.8 lib_attribute.hpp	83
6.9 include/lib_file.hpp File Reference	83
6.9.1 Typedef Documentation	84
6.9.1.1 json	84
6.10 lib_file.hpp	84
6.11 include/lib_group.hpp File Reference	85
6.12 lib_group.hpp	86
6.13 include/si2dr_liberty.h File Reference	86
6.13.1 Macro Definition Documentation	90
6.13.1.1 LONG_DOUBLE	90
6.13.1.2 SI2_ARGS	90
6.13.1.3 SI2_LONG_MAX	90
6.13.1.4 SI2_LONG_MIN	90
6.13.1.5 SI2_ULONG_MAX	91
6.13.1.6 SI2DR_FALSE	91
6.13.1.7 SI2DR_MAX_FLOAT32	91
6.13.1.8 SI2DR_MAX_FLOAT64	91
6.13.1.9 SI2DR_MAX_INT32	91

6.13.1.10 SI2DR_MAX_STRING_LEN	91
6.13.1.11 SI2DR_MIN_FLOAT32	92
6.13.1.12 SI2DR_MIN_FLOAT64	92
6.13.1.13 SI2DR_MIN_INT32	92
6.13.1.14 SI2DR_TRUE	92
6.13.2 Typedef Documentation	92
6.13.2.1 si2BooleanT	92
6.13.2.2 si2DoubleT	92
6.13.2.3 si2drAttrComplexValIdT	93
6.13.2.4 si2drAttrIdT	93
6.13.2.5 si2drAttrSIdT	93
6.13.2.6 si2drAttrTypeT	93
6.13.2.7 si2drBooleanT	93
6.13.2.8 si2drDefinIdT	93
6.13.2.9 si2drDefinesIdT	94
6.13.2.10 si2drErrorT	94
6.13.2.11 si2drExprT	94
6.13.2.12 si2drExprTypeT	94
6.13.2.13 si2drFloat32T	94
6.13.2.14 si2drFloat64T	94
6.13.2.15 si2drGroupIdT	94
6.13.2.16 si2drGroupsIdT	95
6.13.2.17 si2drInt32T	95
6.13.2.18 si2drIterIdT	95
6.13.2.19 si2drMessageHandlerT	95
6.13.2.20 si2drNamesIdT	95
6.13.2.21 si2drObjectIdT	95
6.13.2.22 si2drObjectsIdT	96
6.13.2.23 si2drObjectTypeT	96
6.13.2.24 si2drSeverityT	96
6.13.2.25 si2drStringT	96
6.13.2.26 si2drValuesIdT	96
6.13.2.27 si2drValueTypeT	96
6.13.2.28 si2drVoidT	96
6.13.2.29 si2FloatT	97
6.13.2.30 si2IteratorIdT	97
6.13.2.31 si2LongT	97
6.13.2.32 si2ObjectIdT	97
6.13.2.33 si2StringT	97
6.13.2.34 si2TypeT	97
6.13.2.35 si2VoidT	97
6.13.3 Enumeration Type Documentation	97

6.13.3.1 si2BooleanEnum	97
6.13.3.2 si2drAttrTypeT	98
6.13.3.3 si2drErrorT	98
6.13.3.4 si2drExprTypeT	99
6.13.3.5 si2drObjectTypeT	99
6.13.3.6 si2drSeverityT	99
6.13.3.7 si2drValueTypeT	100
6.13.3.8 si2TypeEnum	100
6.13.4 Function Documentation	100
6.13.4.1 get_function()	100
6.13.4.2 liberty_destroy_value_data()	101
6.13.4.3 liberty_get_element()	101
6.13.4.4 liberty_get_values_data()	101
6.13.4.5 main_liberty_parser()	101
6.13.4.6 print_version()	101
6.13.4.7 SI2_ARGS() [1/41]	101
6.13.4.8 SI2_ARGS() [2/41]	101
6.13.4.9 SI2_ARGS() [3/41]	102
6.13.4.10 SI2_ARGS() [4/41]	102
6.13.4.11 SI2_ARGS() [5/41]	102
6.13.4.12 SI2_ARGS() [6/41]	102
6.13.4.13 SI2_ARGS() [7/41]	102
6.13.4.14 SI2_ARGS() [8/41]	102
6.13.4.15 SI2_ARGS() [9/41]	102
6.13.4.16 SI2_ARGS() [10/41]	103
6.13.4.17 SI2_ARGS() [11/41]	103
6.13.4.18 SI2_ARGS() [12/41]	103
6.13.4.19 SI2_ARGS() [13/41]	103
6.13.4.20 SI2_ARGS() [14/41]	103
6.13.4.21 SI2_ARGS() [15/41]	103
6.13.4.22 SI2_ARGS() [16/41]	103
6.13.4.23 SI2_ARGS() [17/41]	104
6.13.4.24 SI2_ARGS() [18/41]	104
6.13.4.25 SI2_ARGS() [19/41]	104
6.13.4.26 SI2_ARGS() [20/41]	104
6.13.4.27 SI2_ARGS() [21/41]	104
6.13.4.28 SI2_ARGS() [22/41]	104
6.13.4.29 SI2_ARGS() [23/41]	104
6.13.4.30 SI2_ARGS() [24/41]	105
6.13.4.31 SI2_ARGS() [25/41]	105
6.13.4.32 SI2_ARGS() [26/41]	105
6.13.4.33 SI2_ARGS() [27/41]	105

6.13.4.34 SI2_ARGS() [28/41]	105
6.13.4.35 SI2_ARGS() [29/41]	105
6.13.4.36 SI2_ARGS() [30/41]	105
6.13.4.37 SI2_ARGS() [31/41]	106
6.13.4.38 SI2_ARGS() [32/41]	106
6.13.4.39 SI2_ARGS() [33/41]	106
6.13.4.40 SI2_ARGS() [34/41]	106
6.13.4.41 SI2_ARGS() [35/41]	106
6.13.4.42 SI2_ARGS() [36/41]	106
6.13.4.43 SI2_ARGS() [37/41]	106
6.13.4.44 SI2_ARGS() [38/41]	107
6.13.4.45 SI2_ARGS() [39/41]	107
6.13.4.46 SI2_ARGS() [40/41]	107
6.13.4.47 SI2_ARGS() [41/41]	107
6.13.4.48 si2drDefaultMessageHandler()	107
6.13.4.49 si2drPIGetMessageHandler()	107
6.13.4.50 si2drPISetMessageHandler()	108
6.13.4.51 usage()	108
6.14 si2dr_liberty.h	108
6.15 include/verilog_utils.hpp File Reference	115
6.15.1 Function Documentation	116
6.15.1.1 getAST()	116
6.16 verilog_utils.hpp	117
6.17 include/version.h File Reference	118
6.17.1 Macro Definition Documentation	118
6.17.1.1 APP_AUTHOR	118
6.17.1.2 APP_CONTACT	119
6.17.1.3 APP_NAME	119
6.17.1.4 APP_VERSION	119
6.17.1.5 APP_VERSION_MAJOR	119
6.17.1.6 APP_VERSION_MINOR	119
6.17.1.7 APP_VERSION_PATCH	119
6.17.1.8 BUILD_TIMESTAMP	120
6.18 version.h	120
6.19 README.md File Reference	120
6.20 src/compare.cpp File Reference	120
6.21 compare.cpp	121
6.22 src/iterators.cpp File Reference	125
6.23 iterators.cpp	125
6.24 src/json_utils.cpp File Reference	126
6.24.1 Function Documentation	127
6.24.1.1 generateCellJson()	127

6.24.1.2 generateLutJson()	129
6.24.1.3 generatePinJson()	131
6.24.1.4 generatePowerJson()	133
6.24.1.5 generateTimingJson()	134
6.24.1.6 parseStringToVector()	136
6.25 json_utils.cpp	137
6.26 src/lib_attribute.cpp File Reference	140
6.27 lib_attribute.cpp	140
6.28 src/lib_file.cpp File Reference	141
6.29 lib_file.cpp	141
6.30 src/lib_group.cpp File Reference	147
6.31 lib_group.cpp	147
6.32 src/main.cpp File Reference	148
6.32.1 Function Documentation	149
6.32.1.1 compareLibFiles()	149
6.32.1.2 funcLibFile()	150
6.32.1.3 main()	151
6.32.1.4 monoCheckLibFile()	151
6.32.1.5 parseLibFile()	153
6.32.1.6 printInfo()	155
6.32.1.7 supercellLibFile()	156
6.32.1.8 verilogLibFile()	158
6.33 main.cpp	159
6.34 src/verilog_utils.cpp File Reference	164
6.34.1 Function Documentation	165
6.34.1.1 getAST()	165
6.35 verilog_utils.cpp	165

Chapter 1

ZlibValidation

1.1 Description

Command line tool to validate standard cell libraries in .lib format.

1.2 Usage

```
ZlibValidation
Usage: ./zlibvalidation [OPTIONS] [SUBCOMMAND]
Options:
  -h,--help          Print this help message and exit
  -v,--version        Display program version information and exit
Subcommands:
  parse              Parse the Liberty file and write JSON to a file
  mono               Check the monotonicity of timing arc values
  compare            Compare the comparison library against the reference library and report
                     differences
  supercell          Generate supercells for the given Liberty file
  zlibboost           ZlibBoost - Multi-threaded Library Processing Tool
  clear              Clear the log, JSON, map, markdown files in this directory
  verilog             Generate Verilog file for given Liberty file
  spice              Generate SPICE file for given Liberty file
```

1.3 Development Diary

1.3.1 2025-01-27

- C++CMake libsi2dr_liberty.a
- CLI11
- spdlog

1.3.2 2025-01-28

- nlohmann/json JSON
- [version.h](#) CMake
- --version --mode
- spdlog debug info
- [LibFile](#) PVT
 - PVT
- C++ #include

1.3.3 2025-02-01

- `OOPCLibAttribute`
 - `char *std::string`
- `AttributesIteratorRAII`

1.3.4 2025-02-10

- `GroupsIterator`
- `LibGroup`
- `srcincludeCMakeLists.txt`

1.3.5 2025-02-11

- `hppCMakeLists.txtCMake`
- `LibFilejsonLibFile::writeJsonToFilePVTcell_namecell_footprintcell_area`
- `[x] voltage`

1.3.6 2025-02-14

- `json_utils.cpp` JSON (lib) `generateCellJson, generatePinJson, generatePowerJson, generateLutJson` `hpp`
- `[x] (timing)`
- `parseStringToVector` Look Up Table (LUT)
- `LibFile` `lib_json_ (lib)` JSON
- `ValuesIterator`
- `LibAttribute::isComplex()` `bool`
- `LibGroup::getName()` `std::string`
- `si2drGroupsIdT` `sub_groups` `lib_group.getGroups()` `GroupsIterator`

1.3.7 2025-02-15

- `generateTimingJson`

1.3.8 2025-02-17

- `begin()`

1.3.9 2025-02-18

- `LibFile::mono()` values
- `setupLogger()`

1.3.10 2025-02-19

- `mode == "mono"` C++17 `std::filesystem::exists()` JSON JSON JSON
- `LibFile::mono()` `cell_risecell_fallrise_transitionfall_transitiontiming` LUT value

1.3.11 2025-02-20

- `LibFile::mono()` `passed()` `failed()` `cell` `liberate_lv`
- `sl018_ff_3.96_-40.lib` `liberate_lv` 205 out of 559 cells failed
- `when`

1.3.12 2025-02-25

- `CLI11` `parsemono`

1.3.13 2025-02-26

-
- `Logger -h-v`
- `LibFile::mono()` `input_slew value` `tcbn65lpbc.lib` `liberate_lv`

1.3.14 2025-02-27

- GDB
 - `si2dr_liberty` `Liberty`
 - `si2dr_liberty`
 - `si2dr_liberty`
 - `strcmp`

1.3.15 2025-02-28

- `LibFile` `readparse` `si2dr` `parsemono` `si2dr`
- `[x]` `si2drPIGetGroups` `bug:` `WARNING: si2drPIQuit: GetGroups called 1 more times than IterQuit`
- `CMakeLists.txt` `set(CMAKE_BUILD_TYPE Release)` `GDB`

1.3.16 2025-03-01

- `sub_groups_iterfors``si2drPIQuit`
- `group_iterparse``parse``si2drPIQuit``groups`
- `si2drPIQuit``groups``parse`
- `[]``parse``logPVT``(parse 6s mono 1s)``monolog`
- `LibFile`

1.3.17 2025-03-07

- `compare` `JSON`
- `[]`

1.3.18 2025-03-10

- `supercell` `LibFile::supercell` `supercell`
- `zlibboost` `zlibboost`
-
- `clear` `JSON`

1.3.19 2025-03-14

- `tabulate/table` `CMake` `FetchContent`
- `compare.cpp` `compare.hpp` `LibraryComparator` `JSON`

1.3.20 2025-03-15

- `std::filesystem::path` `LibFile` `LibFile` `basename_filename_libname_↔`
`jsonname_` `loggername_JSON`
- `(scope)` `LibFile::parse` `si2drPIQuit`
- `clear` `.map` `.md`
- `mono` `JSON` `JSON`
- `spdlog` `logger` `logger` `APP_NAME` `logger`
- `LibFile` `logger`
- `supercell` `mono`
- `LibraryComparator::generateReport()` `tabulate` `markdown`

1.3.21 2025-03-16

- `compare .md .txt .md`
- `LibraryComparator::generateReport()` `cell cell LibraryComparator::compareCell()`
`cell`
- `LibraryComparator::compareCell()` `cell cell cell comparePin cell`
- `LibraryComparator::comparePin()` `cell pin timing arc pin timing arc compareTimingArc`
`pin timing arc`
- `LibraryComparator::compareTimingArc()` `timing arc JSON timing arc "related_pin" timing`
`arc "cell_rise", "cell_fall", "rise_transition", "fall_transition" timing arc timing arc compareLut timing arc`
- `LibraryComparator::compareLut()` `compareValue JSON value JSON "index_1" "index_2"`
`LUT value (reltol) (abstol)`
- `cell_name pin_name timing_type related_pin arc_name`
- `LibraryComparator::compareValue()` `LibraryComparator::compareLut()`
-
- `abstol 0.002ns`

1.3.22 2025-03-17

- `verilog spice Verilog SPICE`
- `LibraryComparator`
 - `cell`
 - `cell Timing Delay`

1.3.23 2025-03-18

- | Cell Name | Data Type | Failed Count | Avg Diff | Avg Diff% | Max Diff | Max Diff% | Outliers |
- | Pin Name | Reference | Comparison | Diff | Diff % | Type | Arc Name | Row # | Index_1 | Column # | Index_2 | Note |
- `stdout`
- `si2drSimpleAttrGetBooleanValue`
- `LibFile::mono()` `min_pulse_width`
 - `"input_pins" cell`
 - `"timing_arcs"`
 - `"timing_type" "min_pulse_width" timing arc "rise_constraint" "fall_constraint" checkTimingArc↵`
`Monotonicity`
- `checkTimingArcMonotonicity when`
- `checkTimingArcMonotonicity`
- `checkTimingArcMonotonicity related_pin pin`

1.3.24 2025-03-19

- supercell1supercell
- voltage std::round(voltage_ * 100) / 100.0

1.3.25 2025-03-21

- func Verilog

1.3.26 2025-03-24

- C++ Verilog Verilog slang (AST) CMakeLists slang API

1.3.27 2025-03-25

- verilog_utils.cpp verilog_utils.hpp VerilogVisitor Verilog AST
- slang
 - AND2D0 CMPE42D1 DFQD1
 - (UDP) tsmc_dff "Invalid instance declaration"
 - inputoutput
 -
 -

1.3.28 2025-03-26

- UDP tsmc_dff Entry
- slang::syntax::SyntaxRewriter CellExtractor slang::syntax::SyntaxPrinter::printFile() Verilog
- CellPrinter slang::syntax::SyntaxVisitor module.toString() Verilog
- LibFile::supercell() cell_names supercell
- supercell input_pins
- Doxygen Doxygen HTML LaTeX
- [] LaTeX

Chapter 2

Hierarchical Index

2.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

AttributesIterator	13
GroupsIterator	21
LibAttribute	24
liberty_value_data	30
LibFile	31
LibGroup	46
LibraryComparator	50
si2drExprT	58
si2OID	61
slang::syntax::SyntaxRewriter	
CellExtractor	16
slang::syntax::SyntaxVisitor	
CellPrinter	19
VerilogVisitor	65
ValuesIterator	61

Chapter 3

Class Index

3.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

AttributesIterator	13
CellExtractor	16
CellPrinter	19
GroupsIterator	21
LibAttribute	24
liberty_value_data	30
LibFile	31
LibGroup	46
LibraryComparator	50
si2drExprT	58
si2OID	61
ValuesIterator	61
VerilogVisitor	65

Chapter 4

File Index

4.1 File List

Here is a list of all files with brief descriptions:

include/compare.hpp	69
include/iterators.hpp	71
include/json_utils.hpp	72
include/lib_attribute.hpp	82
include/lib_file.hpp	83
include/lib_group.hpp	85
include/si2dr_liberty.h	86
include/verilog_utils.hpp	115
include/version.h	118
src/compare.cpp	120
src/iterators.cpp	125
src/json_utils.cpp	126
src/lib_attribute.cpp	140
src/lib_file.cpp	141
src/lib_group.cpp	147
src/main.cpp	148
src/verilog_utils.cpp	164

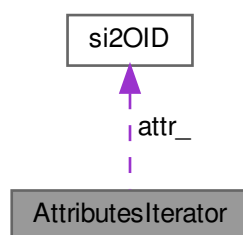
Chapter 5

Class Documentation

5.1 AttributesIterator Class Reference

```
#include <iterators.hpp>
```

Collaboration diagram for AttributesIterator:



Public Member Functions

- [AttributesIterator](#) ([si2drAttrIdT](#) attrs, [si2drErrorT](#) &err)
Constructs an [AttributesIterator](#) object.
- [~AttributesIterator](#) ()
- void [next](#) ()
- bool [end](#) ()
- [LibAttribute](#) [get](#) ()

Private Attributes

- [si2drAttrIdT](#) [attrs_](#)
- [si2drAttrIdT](#) [attr_](#)
- [si2drErrorT](#) & [err_](#)

5.1.1 Detailed Description

Definition at line 23 of file [iterators.hpp](#).

5.1.2 Constructor & Destructor Documentation

5.1.2.1 AttributesIterator()

```
AttributesIterator::AttributesIterator (
    si2drAttrsIdT attrs,
    si2drErrorT & err )
```

Constructs an [AttributesIterator](#) object.

This constructor initializes an [AttributesIterator](#) with the provided attributes identifier and error reference. It also fetches the first attribute in the collection using `si2drIterNextAttr`.

Parameters

<i>attrs</i>	The <code>si2drAttrsIdT</code> identifier for the attributes collection to iterate over
<i>err</i>	Reference to an error object for capturing any errors during operation

Definition at line 32 of file [iterators.cpp](#).

5.1.2.2 ~AttributesIterator()

```
AttributesIterator::~~AttributesIterator ( )
```

Definition at line 35 of file [iterators.cpp](#).

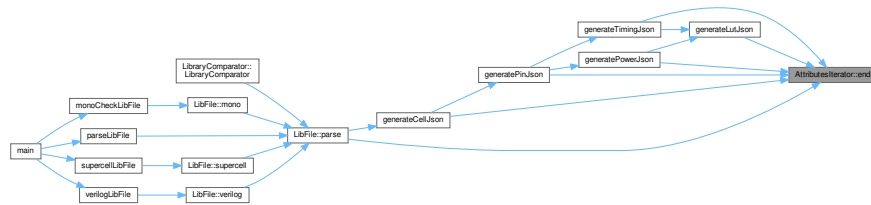
5.1.3 Member Function Documentation

5.1.3.1 end()

```
bool AttributesIterator::end ( )
```

Definition at line 38 of file [iterators.cpp](#).

Here is the caller graph for this function:

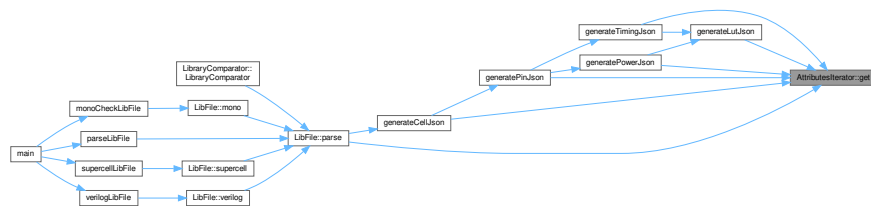


5.1.3.2 get()

`LibAttribute AttributesIterator::get ( )`

Definition at line 40 of file [iterators.cpp](#).

Here is the caller graph for this function:

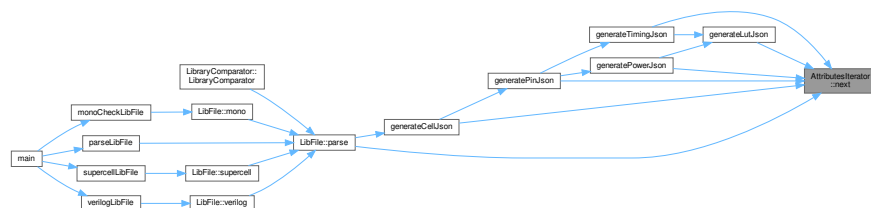


5.1.3.3 next()

`void AttributesIterator::next ( )`

Definition at line 37 of file [iterators.cpp](#).

Here is the caller graph for this function:



5.1.4 Member Data Documentation

5.1.4.1 attr_

`si2drAttrIdT` `AttributesIterator::attr_` [private]

Definition at line 33 of file [iterators.hpp](#).

5.1.4.2 attrs_

`si2drAttrsIdT` `AttributesIterator::attrs_` [private]

Definition at line 32 of file [iterators.hpp](#).

5.1.4.3 err_

`si2drErrorT&` `AttributesIterator::err_` [private]

Definition at line 34 of file [iterators.hpp](#).

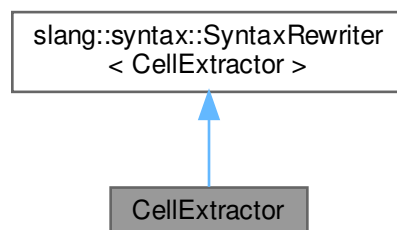
The documentation for this class was generated from the following files:

- [include/iterators.hpp](#)
- [src/iterators.cpp](#)

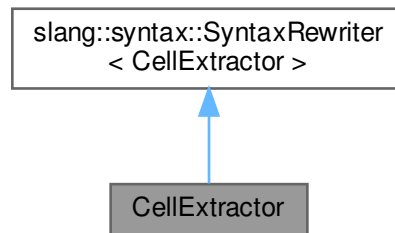
5.2 CellExtractor Class Reference

```
#include <verilog_utils.hpp>
```

Inheritance diagram for CellExtractor:



Collaboration diagram for CellExtractor:



Public Member Functions

- [CellExtractor](#) (const std::string &targetCell)
- void [handle](#) (const slang::syntax::ModuleDeclarationSyntax &module)
- bool [foundTargetCell](#) () const

Private Attributes

- std::string [targetCell_](#)
- bool [foundTarget_](#)

5.2.1 Detailed Description

Definition at line 30 of file [verilog_utils.hpp](#).

5.2.2 Constructor & Destructor Documentation

5.2.2.1 CellExtractor()

```
CellExtractor::CellExtractor (  
    const std::string & targetCell ) [explicit]
```

Definition at line 352 of file [verilog_utils.cpp](#).

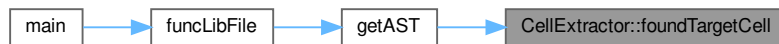
5.2.3 Member Function Documentation

5.2.3.1 foundTargetCell()

```
bool CellExtractor::foundTargetCell ( ) const
```

Definition at line 371 of file [verilog_utils.cpp](#).

Here is the caller graph for this function:



5.2.3.2 handle()

```
void CellExtractor::handle (
    const slang::syntax::ModuleDeclarationSyntax & module )
```

Definition at line 355 of file [verilog_utils.cpp](#).

5.2.4 Member Data Documentation

5.2.4.1 foundTarget_

```
bool CellExtractor::foundTarget_ [private]
```

Definition at line 38 of file [verilog_utils.hpp](#).

5.2.4.2 targetCell_

```
std::string CellExtractor::targetCell_ [private]
```

Definition at line 37 of file [verilog_utils.hpp](#).

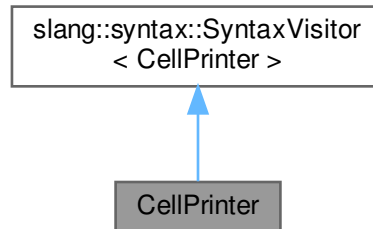
The documentation for this class was generated from the following files:

- [include/verilog_utils.hpp](#)
- [src/verilog_utils.cpp](#)

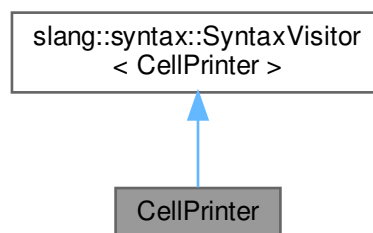
5.3 CellPrinter Class Reference

```
#include <verilog_utils.hpp>
```

Inheritance diagram for CellPrinter:



Collaboration diagram for CellPrinter:



Public Member Functions

- [CellPrinter](#) (const std::string &targetCell, std::ostream &out)
- void [handle](#) (const slang::syntax::ModuleDeclarationSyntax &module)

Private Attributes

- std::string [targetCell_](#)
- std::ostream & [out_](#)
- bool [foundTarget_](#)

5.3.1 Detailed Description

Definition at line 42 of file [verilog_utils.hpp](#).

5.3.2 Constructor & Destructor Documentation

5.3.2.1 CellPrinter()

```
CellPrinter::CellPrinter (
    const std::string & targetCell,
    std::ostream & out ) [explicit]
```

Definition at line [373](#) of file [verilog_utils.cpp](#).

5.3.3 Member Function Documentation

5.3.3.1 handle()

```
void CellPrinter::handle (
    const slang::syntax::ModuleDeclarationSyntax & module )
```

Definition at line [377](#) of file [verilog_utils.cpp](#).

5.3.4 Member Data Documentation

5.3.4.1 foundTarget_

```
bool CellPrinter::foundTarget_ [private]
```

Definition at line [50](#) of file [verilog_utils.hpp](#).

5.3.4.2 out_

```
std::ostream& CellPrinter::out_ [private]
```

Definition at line [49](#) of file [verilog_utils.hpp](#).

5.3.4.3 targetCell_

```
std::string CellPrinter::targetCell_ [private]
```

Definition at line 48 of file [verilog_utils.hpp](#).

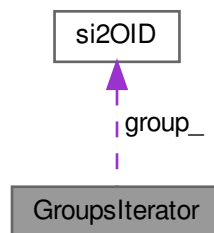
The documentation for this class was generated from the following files:

- [include/verilog_utils.hpp](#)
- [src/verilog_utils.cpp](#)

5.4 GroupsIterator Class Reference

```
#include <iterators.hpp>
```

Collaboration diagram for GroupsIterator:



Public Member Functions

- [GroupsIterator](#) ([si2drGroupsIdT](#) groups, [si2drErrorT](#) &err)
Constructs a [GroupsIterator](#) with the given groups and error reference.
- [~GroupsIterator](#) ()
- void [next](#) ()
- bool [end](#) ()
- [LibGroup](#) [get](#) ()

Private Attributes

- [si2drGroupsIdT](#) groups_
- [si2drGroupIdT](#) group_
- [si2drErrorT](#) & err_

5.4.1 Detailed Description

Definition at line 9 of file [iterators.hpp](#).

5.4.2 Constructor & Destructor Documentation

5.4.2.1 GroupsIterator()

```
GroupsIterator::GroupsIterator (
    si2drGroupsIdT groups,
    si2drErrorT & err )
```

Constructs a [GroupsIterator](#) with the given groups and error reference.

Initializes the iterator with the provided groups collection and advances to the first group in the collection.

Parameters

<i>groups</i>	The si2drGroupsIdT collection to iterate over
<i>err</i>	Reference to an si2drErrorT object for error handling

Definition at line 12 of file [iterators.cpp](#).

5.4.2.2 ~GroupsIterator()

```
GroupsIterator::~~GroupsIterator ( )
```

Definition at line 15 of file [iterators.cpp](#).

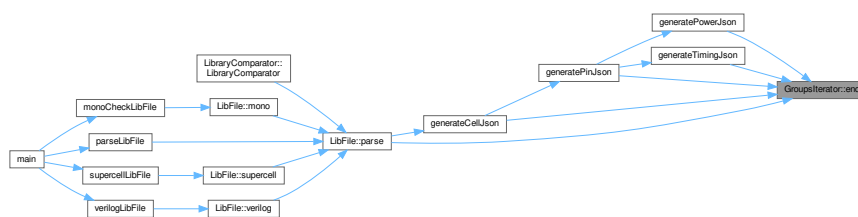
5.4.3 Member Function Documentation

5.4.3.1 end()

```
bool GroupsIterator::end ( )
```

Definition at line 18 of file [iterators.cpp](#).

Here is the caller graph for this function:

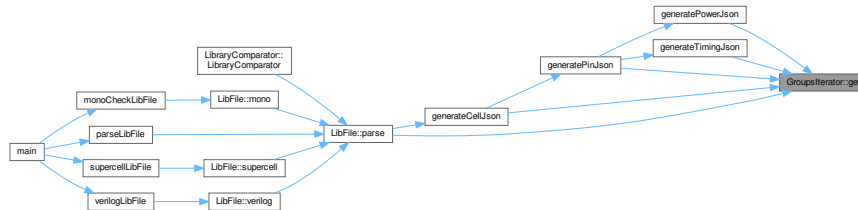


5.4.3.2 get()

`LibGroup GroupsIterator::get ( )`

Definition at line 20 of file [iterators.cpp](#).

Here is the caller graph for this function:

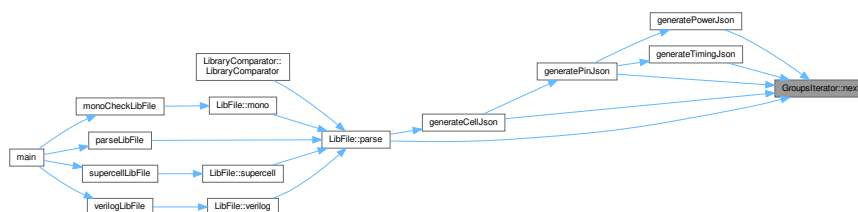


5.4.3.3 next()

`void GroupsIterator::next ( )`

Definition at line 17 of file [iterators.cpp](#).

Here is the caller graph for this function:



5.4.4 Member Data Documentation

5.4.4.1 err_

`si2drErrorT& GroupsIterator::err_ [private]`

Definition at line 20 of file [iterators.hpp](#).

5.4.4.2 group_

`si2drGroupIdT GroupsIterator::group_ [private]`

Definition at line 19 of file [iterators.hpp](#).

5.4.4.3 groups_

`si2drGroupsIdT GroupsIterator::groups_ [private]`

Definition at line 18 of file [iterators.hpp](#).

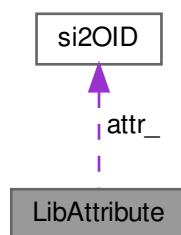
The documentation for this class was generated from the following files:

- [include/iterators.hpp](#)
- [src/iterators.cpp](#)

5.5 LibAttribute Class Reference

```
#include <lib_attribute.hpp>
```

Collaboration diagram for LibAttribute:



Public Member Functions

- [LibAttribute](#) ([si2drAttrIdT](#) attr, [si2drErrorT](#) &err)
Constructs a [LibAttribute](#) object from a [si2drAttrIdT](#) attribute.
- [~LibAttribute](#) ()
- [std::string](#) [getName](#) ()
Gets the name of the attribute.
- [bool](#) [isComplex](#) ()
Checks if the attribute is a complex attribute.
- [si2drValuesIdT](#) [getValues](#) ()
Retrieves the values of this complex attribute.
- [long int](#) [getInt](#) ()
Retrieves the integer value of the attribute.
- [double](#) [getFloat](#) ()
Retrieves the float value from the attribute.
- [std::string](#) [getString](#) ()
Retrieves the string value of the attribute.
- [bool](#) [getBoolean](#) ()
Retrieves the boolean value from the attribute.

Private Attributes

- [si2drAttrIdT](#) attr_
- [si2drErrorT](#) & err_

5.5.1 Detailed Description

Definition at line 8 of file [lib_attribute.hpp](#).

5.5.2 Constructor & Destructor Documentation

5.5.2.1 LibAttribute()

```
LibAttribute::LibAttribute (
    si2drAttrIdT attr,
    si2drErrorT & err )
```

Constructs a [LibAttribute](#) object from a [si2drAttrIdT](#) attribute.

Parameters

<i>attr</i>	The si2drAttrIdT attribute to wrap
<i>err</i>	Reference to an error object for reporting any errors during construction

Definition at line 9 of file [lib_attribute.cpp](#).

5.5.2.2 ~LibAttribute()

```
LibAttribute::~~LibAttribute ( )
```

Definition at line 11 of file [lib_attribute.cpp](#).

5.5.3 Member Function Documentation

5.5.3.1 getBoolean()

```
bool LibAttribute::getBoolean ( )
```

Retrieves the boolean value from the attribute.

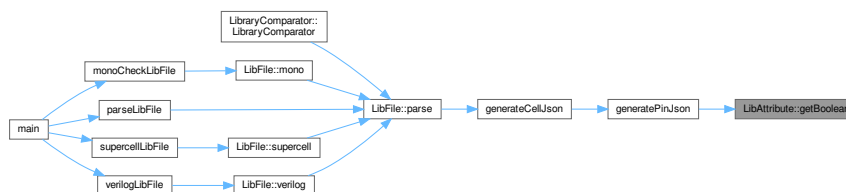
This method gets the boolean value stored in the attribute using the `si2drSimpleAttrGetBooleanValue` function. The function uses the internal attribute handler (`attr_`) and error handler (`err_`).

Returns

true if the attribute value is true, false otherwise

Definition at line 89 of file [lib_attribute.cpp](#).

Here is the caller graph for this function:



5.5.3.2 getFloat()

```
double LibAttribute::getFloat ( )
```

Retrieves the float value from the attribute.

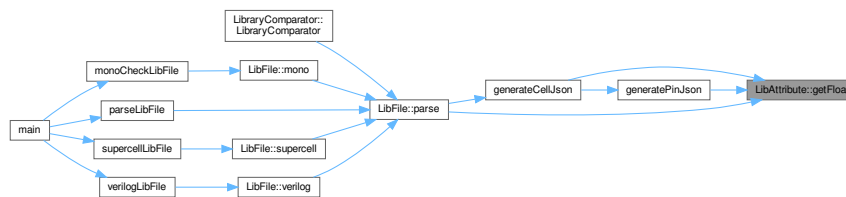
This method extracts the 64-bit floating point value from the attribute using the SI2DR API.

Returns

The float value stored in the attribute.

Definition at line 63 of file [lib_attribute.cpp](#).

Here is the caller graph for this function:



5.5.3.3 getInt()

```
long int LibAttribute::getInt ( )
```

Retrieves the integer value of the attribute.

Gets the 32-bit integer value from the si2dr attribute.

Returns

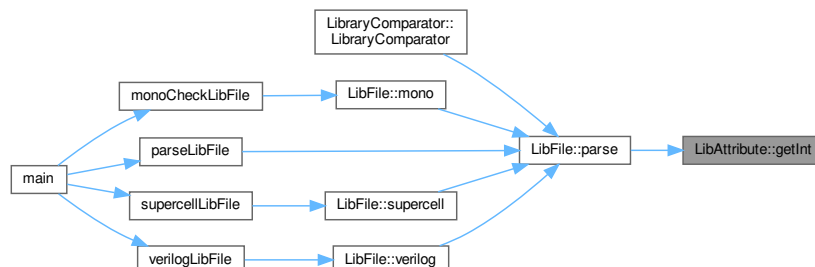
The integer value stored in the attribute.

Note

This method accesses the internal attribute handler and error handler.

Definition at line 53 of file [lib_attribute.cpp](#).

Here is the caller graph for this function:



5.5.3.4 getName()

```
std::string LibAttribute::getName ( )
```

Gets the name of the attribute.

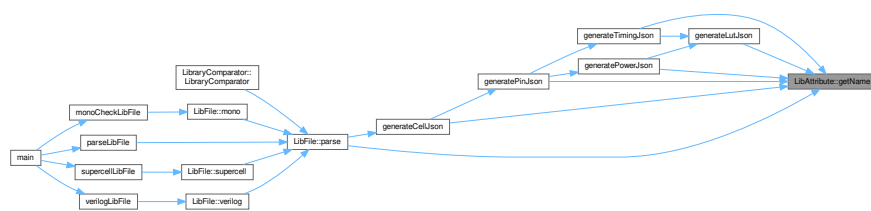
Retrieves the name of the attribute using the si2drAttrGetName function from the underlying SI2DR API.

Returns

A std::string containing the attribute name. Returns an empty string if the name cannot be retrieved.

Definition at line 22 of file [lib_attribute.cpp](#).

Here is the caller graph for this function:



5.5.3.5 getString()

```
std::string LibAttribute::getString ( )
```

Retrieves the string value of the attribute.

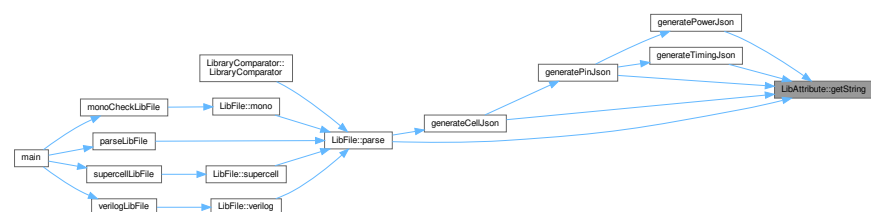
This method converts the internal attribute value to a standard C++ string. It uses the si2drSimpleAttrGetStringValue function to get the string value from the underlying attribute representation.

Returns

std::string The string value of the attribute, or an empty string if the attribute does not contain a string value or is invalid.

Definition at line 75 of file [lib_attribute.cpp](#).

Here is the caller graph for this function:



5.5.3.6 getValues()

```
si2drValuesIdT LibAttribute::getValues ( )
```

Retrieves the values of this complex attribute.

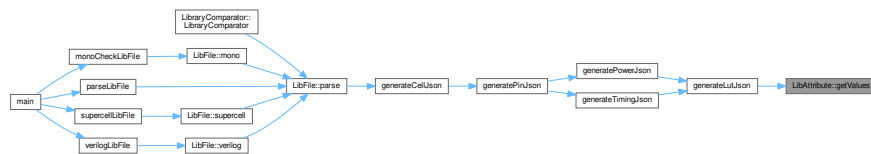
This method returns the values associated with a complex attribute using the SI2DR (Silicon Integration Initiative Design Representation) API.

Returns

si2drValuesIdT A handle to the values of the complex attribute.

Definition at line 42 of file [lib_attribute.cpp](#).

Here is the caller graph for this function:



5.5.3.7 isComplex()

```
bool LibAttribute::isComplex ( )
```

Checks if the attribute is a complex attribute.

Returns

true if the attribute is complex, false otherwise.

Definition at line 32 of file [lib_attribute.cpp](#).

5.5.4 Member Data Documentation

5.5.4.1 attr_

```
si2drAttrIdT LibAttribute::attr_ [private]
```

Definition at line 21 of file [lib_attribute.hpp](#).

5.5.4.2 err_

```
si2drErrorT& LibAttribute::err_ [private]
```

Definition at line 22 of file [lib_attribute.hpp](#).

The documentation for this class was generated from the following files:

- [include/lib_attribute.hpp](#)
- [src/lib_attribute.cpp](#)

5.6 liberty_value_data Struct Reference

```
#include <si2dr_liberty.h>
```

Public Attributes

- int [dimensions](#)
- int * [dim_sizes](#)
- LONG_DOUBLE ** [index_info](#)
- LONG_DOUBLE * [values](#)

5.6.1 Detailed Description

Definition at line 612 of file [si2dr_liberty.h](#).

5.6.2 Member Data Documentation

5.6.2.1 dim_sizes

```
int* liberty_value_data::dim_sizes
```

Definition at line 615 of file [si2dr_liberty.h](#).

5.6.2.2 dimensions

```
int liberty_value_data::dimensions
```

Definition at line 614 of file [si2dr_liberty.h](#).

5.6.2.3 index_info

`LONG_DOUBLE** liberty_value_data::index_info`

Definition at line 617 of file [si2dr_liberty.h](#).

5.6.2.4 values

`LONG_DOUBLE* liberty_value_data::values`

Definition at line 621 of file [si2dr_liberty.h](#).

The documentation for this struct was generated from the following file:

- [include/si2dr_liberty.h](#)

5.7 LibFile Class Reference

```
#include <lib_file.hpp>
```

Public Member Functions

- [LibFile](#) (const std::string &filepath, const std::string &loggername)
- [~LibFile](#) ()
- void [writeJsonToFile](#) ()
Writes the JSON data stored in the lib_json_ member to a file.
- void [parse](#) ()
Parses the library file and extracts relevant information.
- void [modify](#) ()
- void [mono](#) (const bool is_slew)
Checks monotonicity of timing values in the current liberty file.
- void [supercell](#) (const int chain_length, const std::vector< std::string > &cell_names)
Creates supercell mappings and writes them to a .map file.
- void [verilog](#) (const int chain_length, const std::vector< std::string > &cell_names)

Public Attributes

- std::shared_ptr< spdlog::logger > [logger_](#)
- std::filesystem::path [filepath_](#)
- std::string [basename_](#)
- std::string [filename_](#)
- std::string [libname_](#) = ""
- std::string [jsonname_](#) = ""
- std::string [loggername_](#) = ""
- [json lib_json_](#) = json::object()

Private Member Functions

- void `read` ()
Reads the Liberty file specified by the `filename_` member variable.
- bool `checkTimingArcMonotonicity` (const `json` &cell, const `json` &pin, const `json` &arc, const std::string &type, bool is_slew)
Checks the monotonicity of timing arc values in a given cell.

Private Attributes

- `si2drErrorT` `err_`
- int `process_` = 0
- float `voltage_` = 0.0
- int `temperature_` = 0

5.7.1 Detailed Description

Definition at line 13 of file `lib_file.hpp`.

5.7.2 Constructor & Destructor Documentation

5.7.2.1 LibFile()

```
LibFile::LibFile (
    const std::string & filepath,
    const std::string & loggername )
```

Definition at line 15 of file `lib_file.cpp`.

5.7.2.2 ~LibFile()

```
LibFile::~LibFile ( )
```

Definition at line 35 of file `lib_file.cpp`.

5.7.3 Member Function Documentation

5.7.3.1 checkTimingArcMonotonicity()

```
bool LibFile::checkTimingArcMonotonicity (
    const json & cell,
    const json & pin,
    const json & arc,
    const std::string & timing_arc_name,
    bool is_slew ) [private]
```

Checks the monotonicity of timing arc values in a given cell.

This function verifies if the timing arc values in the provided JSON objects are monotonic. It logs detailed information about the cell, pin, and related pin being checked, and reports any non-monotonic values found.

Parameters

<i>cell</i>	The JSON object representing the cell.
<i>pin</i>	The JSON object representing the pin.
<i>arc</i>	The JSON object representing the timing arc.
<i>timing_arc_name</i>	The name of the timing arc to check.
<i>is_slew</i>	Flag indicating whether to check for input slew monotonicity.

Returns

true if the timing arc values are monotonic, false otherwise.

The function performs the following steps:

1. Logs the cell, pin, related pin, and timing arc name being checked.
2. Assumes the values are monotonic initially.
3. Checks if the timing arc contains the specified name and if it has a "values" array.
4. Generates a matrix of values from the JSON array.
5. Validates the format of the "values" array and logs errors for invalid formats.
6. Checks for non-numeric values in the "values" array and logs warnings for such values.
7. Checks the monotonic incrementality of rows in the matrix.
8. If *is_slew* is true, checks the monotonic incrementality of columns in the matrix.
9. Logs warnings for non-monotonic values, including the "when" key if present.
10. Returns the monotonicity status.

Definition at line 208 of file [lib_file.cpp](#).

Here is the caller graph for this function:



5.7.3.2 modify()

```
void LibFile::modify ( )
```

Definition at line 180 of file [lib_file.cpp](#).

5.7.3.3 mono()

```
void LibFile::mono (
    const bool is_slew )
```

Checks monotonicity of timing values in the current liberty file.

This function validates that various timing tables in the library are monotonically increasing with respect to output load. It checks the following data:

- cell_rise, retaining_rise
- cell_fall, retaining_fall
- rise_transition, retain_rise_slew
- fall_transition, retain_fall_slew
- mpw (minimum pulse width)

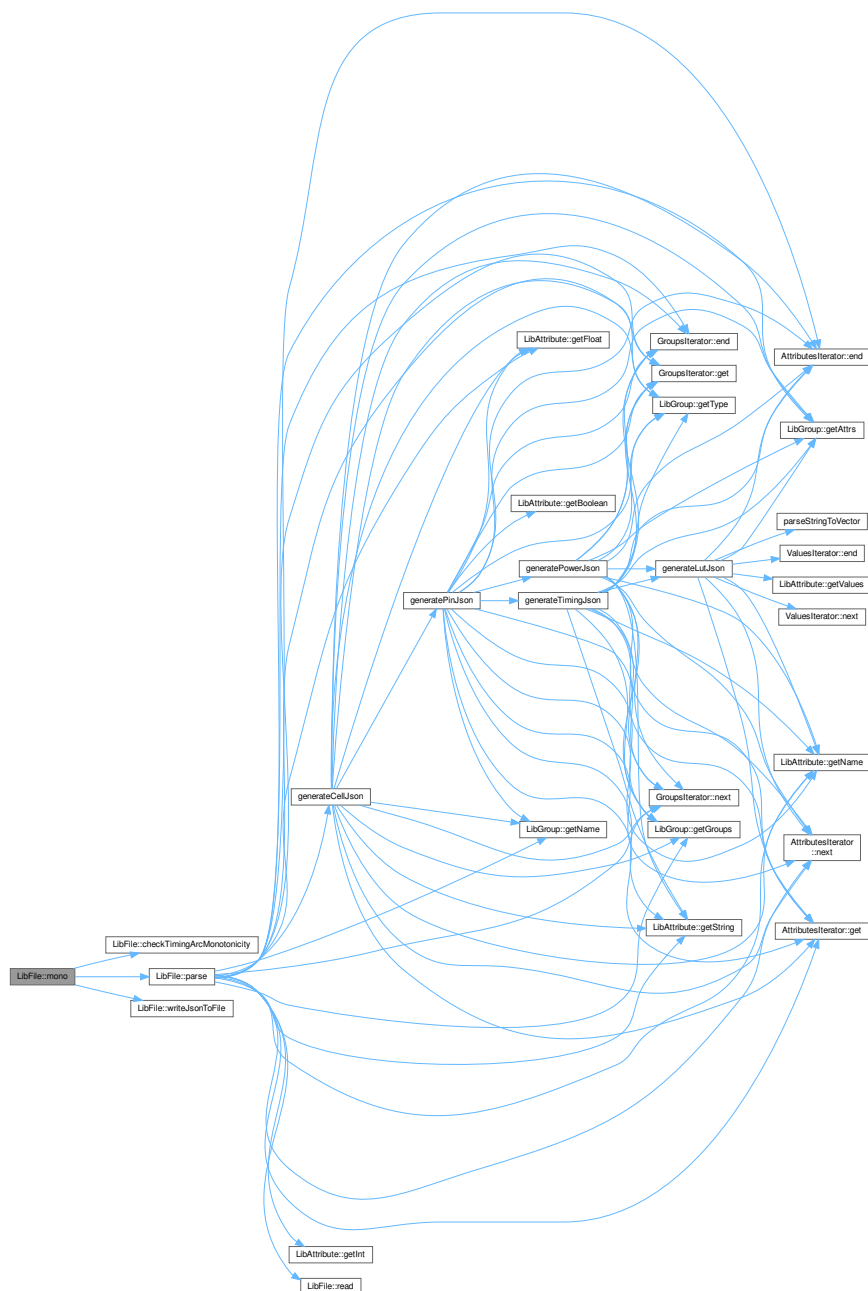
The function first checks if a parsed JSON representation of the Liberty file exists. If not, it parses the Liberty file and creates the JSON. It then examines each cell and its timing arcs to verify monotonicity. Results are logged, including statistics on passing/failing cells and a list of all cells that failed the monotonicity check.

Parameters

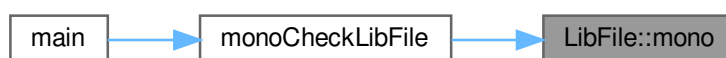
<i>is_slew</i>	Boolean flag that determines if slew-based checks should be performed
----------------	---

Definition at line 328 of file [lib_file.cpp](#).

Here is the call graph for this function:



Here is the caller graph for this function:



5.7.3.4 parse()

```
void LibFile::parse ( )
```

Parses the library file and extracts relevant information.

This function performs the following tasks:

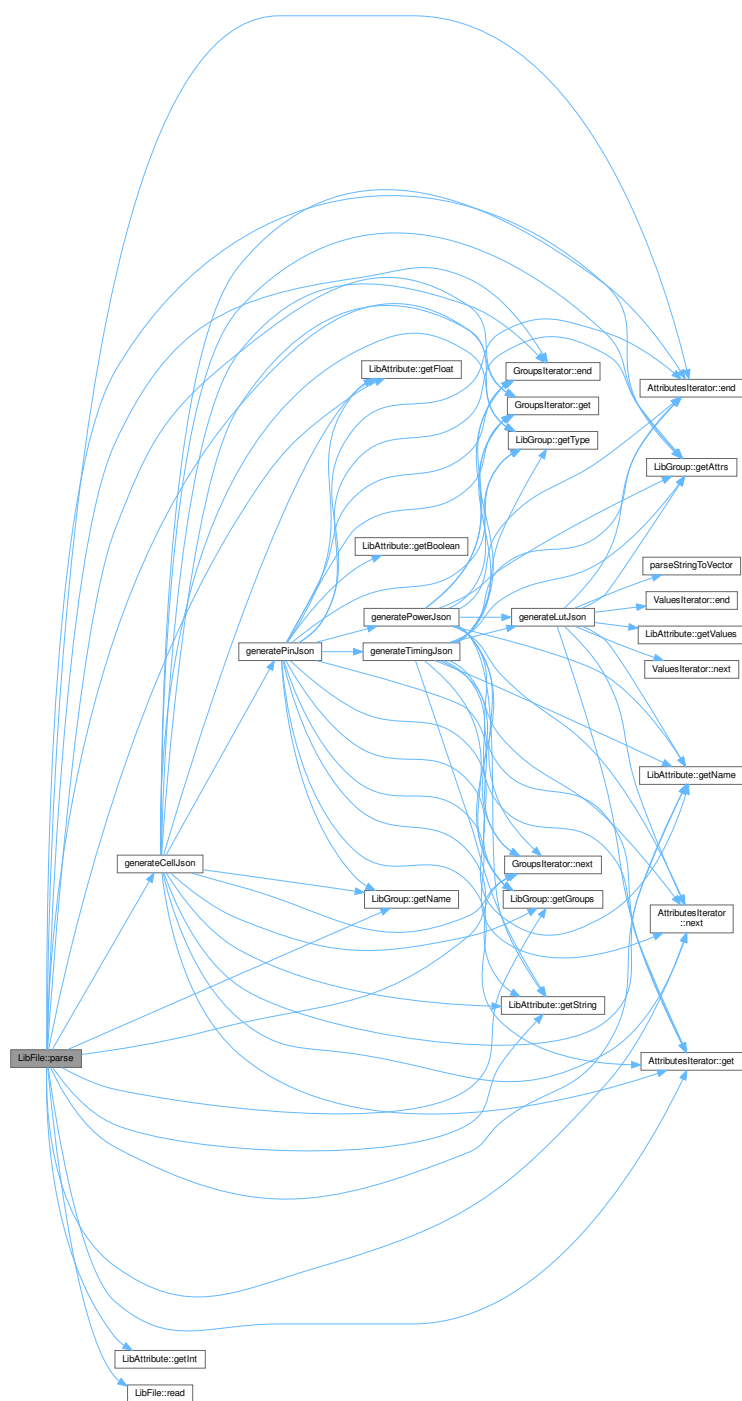
- Logs the start of the parsing process.
- Iterates through the top-level groups in the library file.
- Extracts and logs the library name.
- Iterates through the second-level groups within each top-level group.
- Handles cells by generating JSON representations and storing them.
- Handles operating conditions by extracting PVT (Process, Voltage, Temperature) attributes and storing them.

The extracted information is stored in the `lib_json_` member variable.

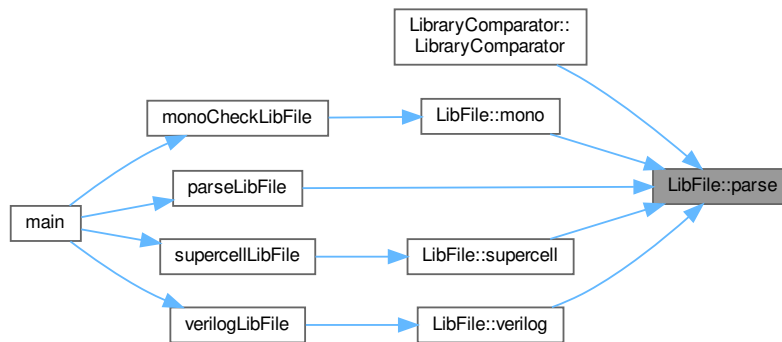
Logging is performed at various levels (info, warn, debug) to provide detailed information about the parsing process.

Definition at line 114 of file [lib_file.cpp](#).

Here is the call graph for this function:



Here is the caller graph for this function:



5.7.3.5 read()

```
void LibFile::read ( ) [private]
```

Reads the Liberty file specified by the `filename_` member variable.

This function attempts to read a Liberty file and logs the process. It measures the time taken to read the file and logs any errors encountered during the read operation. If an error occurs, the function logs the error and terminates the program.

- Logs the start of the read operation.
- Measures the duration of the read operation.
- Checks for specific errors (invalid name or syntax error) and logs them.
- Terminates the program with specific exit codes if errors are detected.
- Logs the completion of the read operation and the time taken.

Note

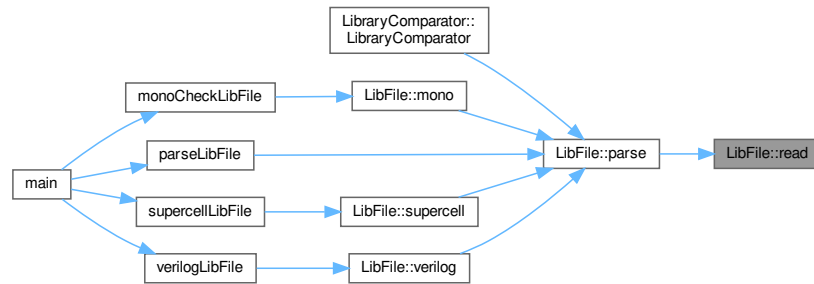
This function uses the `spdlog` library for logging and the `si2drReadLibertyFile` function for reading the Liberty file.

Exceptions

<i>This</i>	function will terminate the program with exit codes 301 or 401 if specific errors are encountered.
-------------	--

Definition at line 78 of file [lib_file.cpp](#).

Here is the caller graph for this function:



5.7.3.6 supercell()

```
void LibFile::supercell (
    const int chain_length,
    const std::vector< std::string > & cell_names )
```

Creates supercell mappings and writes them to a .map file.

Supercells are named according to the pattern: <cellname>**X**<chain_length><input_pin>__<output_pin>

This function:

1. Checks for an existing JSON file with cell data
2. If not found, parses the Liberty file and generates the JSON
3. For each cell in the library, identifies input and output pins
4. Creates supercell mappings for all input/output pin combinations
5. Writes these mappings to a .map file (<basename>.map)
6. Handles sequential cells differently, using a chain length of 1

Parameters

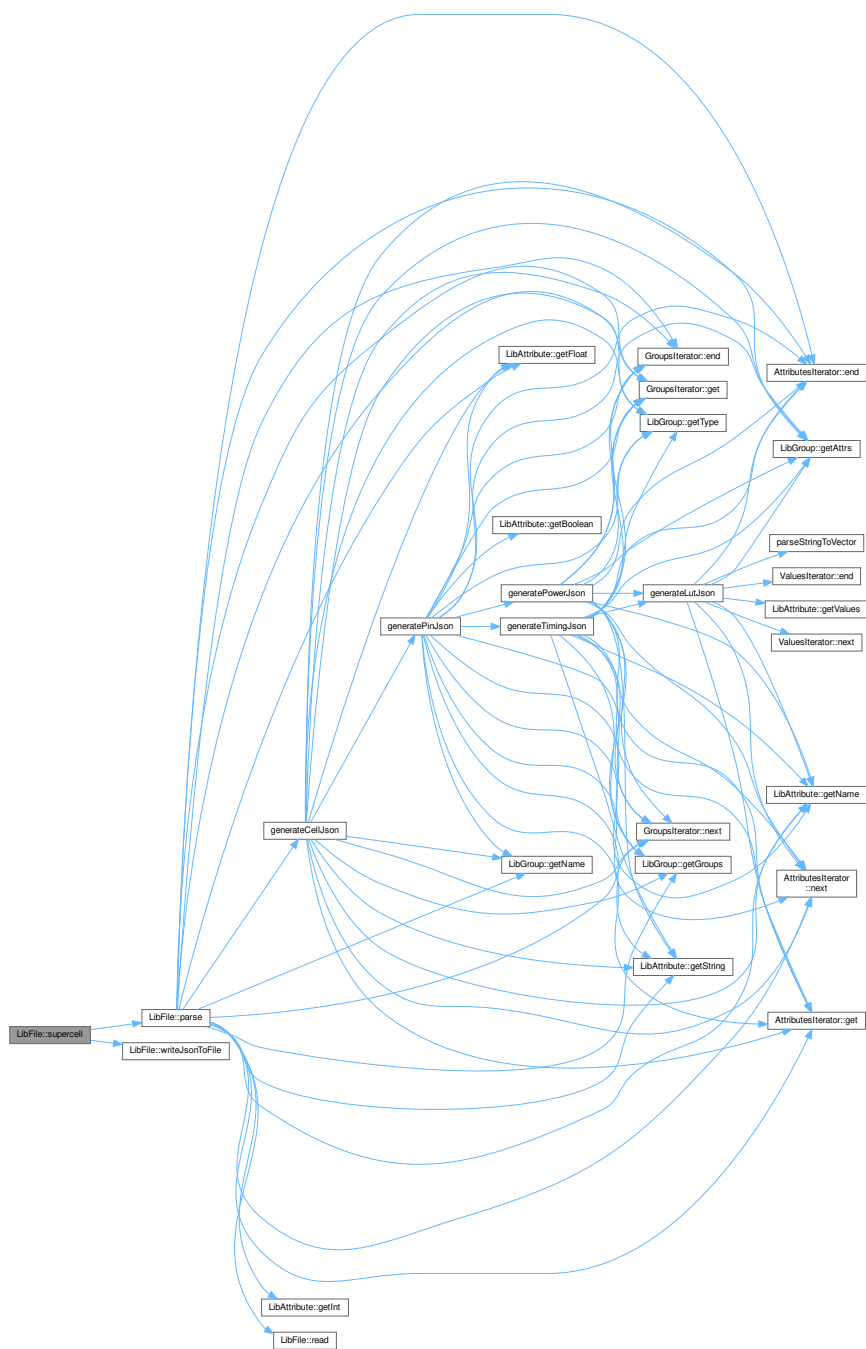
<i>chain_length</i>	The chain length to use for combinational cells
---------------------	---

Note

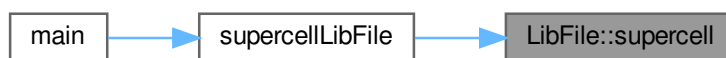
Sequential cells (those with clock pins) will always use chain length 1

Definition at line 455 of file [lib_file.cpp](#).

Here is the call graph for this function:



Here is the caller graph for this function:

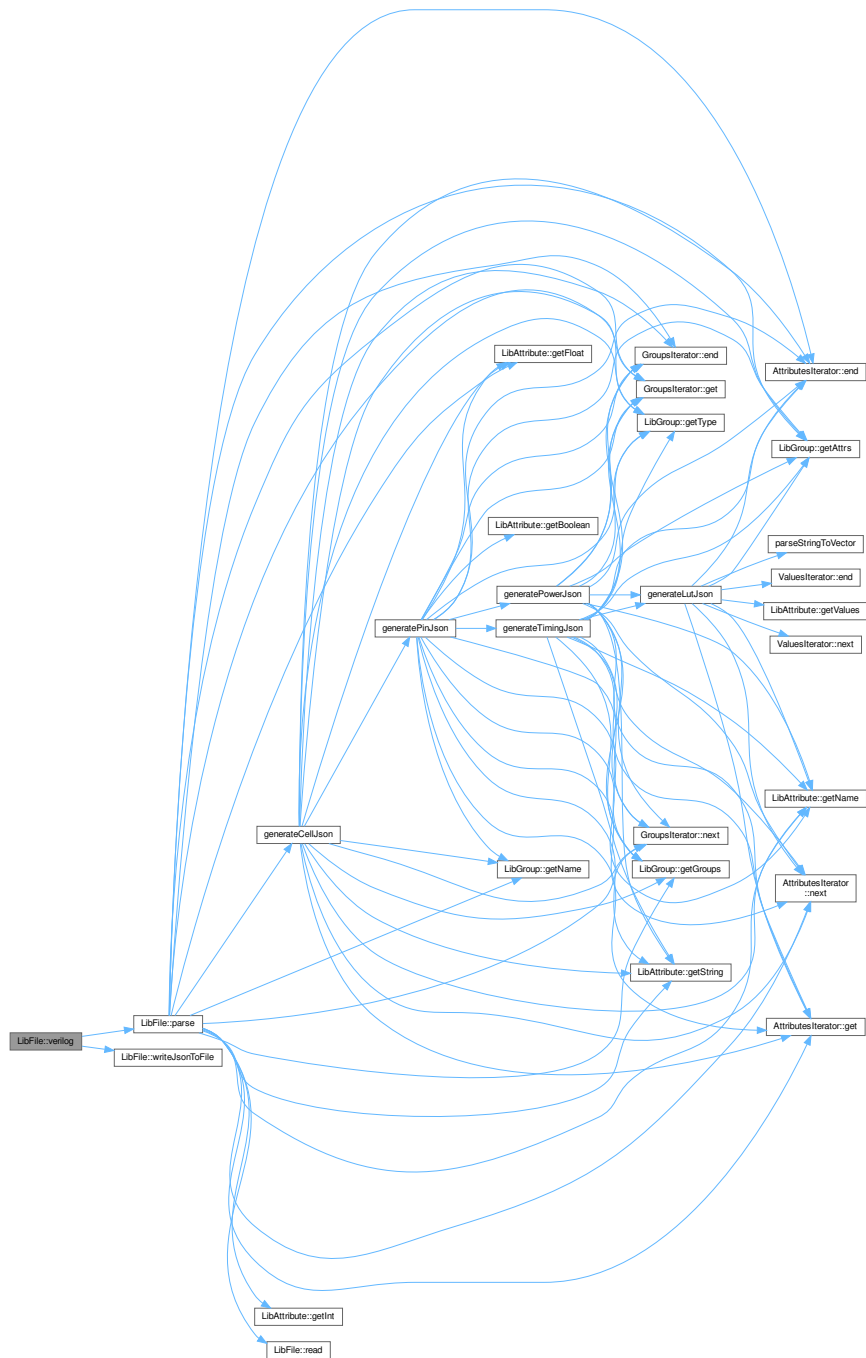


5.7.3.7 verilog()

```
void LibFile::verilog (  
    const int chain_length,  
    const std::vector< std::string > & cell_names )
```

Definition at line 584 of file [lib_file.cpp](#).

Here is the call graph for this function:



Here is the caller graph for this function:



5.7.3.8 writeJsonToFile()

```
void LibFile::writeJsonToFile ( )
```

Writes the JSON data stored in the `lib_json_` member to a file.

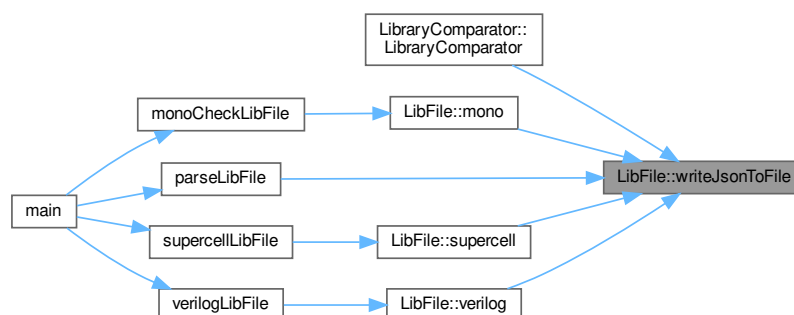
This function attempts to open the specified file for writing. If the file cannot be opened, an error message is logged. If the file is successfully opened, the JSON data is written to the file with an indentation of 2 spaces. After writing, the file is closed and a success message is logged.

Parameters

<i>jsonname</i> ↔	The name of the file to which the JSON data will be written.
—	

Definition at line 47 of file [lib_file.cpp](#).

Here is the caller graph for this function:



5.7.4 Member Data Documentation

5.7.4.1 basename_

```
std::string LibFile::basename_
```

Definition at line 19 of file [lib_file.hpp](#).

5.7.4.2 err_

```
si2drErrorT LibFile::err_ [private]
```

Definition at line 33 of file [lib_file.hpp](#).

5.7.4.3 filename_

```
std::string LibFile::filename_
```

Definition at line 20 of file [lib_file.hpp](#).

5.7.4.4 filepath_

```
std::filesystem::path LibFile::filepath_
```

Definition at line 18 of file [lib_file.hpp](#).

5.7.4.5 jsonname_

```
std::string LibFile::jsonname_ = ""
```

Definition at line 22 of file [lib_file.hpp](#).

5.7.4.6 lib_json_

```
json LibFile::lib_json_ = json::object()
```

Definition at line 24 of file [lib_file.hpp](#).

5.7.4.7 libname_

```
std::string LibFile::libname_ = ""
```

Definition at line 21 of file [lib_file.hpp](#).

5.7.4.8 logger_

```
std::shared_ptr<spdlog::logger> LibFile::logger_
```

Definition at line 17 of file [lib_file.hpp](#).

5.7.4.9 loggername_

```
std::string LibFile::loggername_ = ""
```

Definition at line 23 of file [lib_file.hpp](#).

5.7.4.10 process_

```
int LibFile::process_ = 0 [private]
```

Definition at line 34 of file [lib_file.hpp](#).

5.7.4.11 temperature_

```
int LibFile::temperature_ = 0 [private]
```

Definition at line 36 of file [lib_file.hpp](#).

5.7.4.12 voltage_

```
float LibFile::voltage_ = 0.0 [private]
```

Definition at line 35 of file [lib_file.hpp](#).

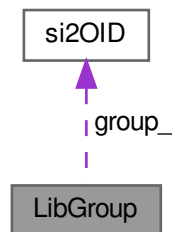
The documentation for this class was generated from the following files:

- [include/lib_file.hpp](#)
- [src/lib_file.cpp](#)

5.8 LibGroup Class Reference

```
#include <lib_group.hpp>
```

Collaboration diagram for LibGroup:



Public Member Functions

- `LibGroup (si2drGroupIdT group, si2drErrorT &err)`
Constructor for the [LibGroup](#) class.
- `~LibGroup ()`
- `std::string getName ()`
Get the name of the [LibGroup](#).
- `std::string getType ()`
Retrieves the type of the current group.
- `si2drAttrIdT getAttrs ()`
Get attributes of the library group.
- `si2drGroupsIdT getGroups ()`
Retrieves all groups contained within this group.

Private Attributes

- `si2drGroupIdT group_`
- `si2drErrorT & err_`

5.8.1 Detailed Description

Definition at line 8 of file [lib_group.hpp](#).

5.8.2 Constructor & Destructor Documentation

5.8.2.1 LibGroup()

```
LibGroup::LibGroup (
    si2drGroupIdT group,
    si2drErrorT & err )
```

Constructor for the [LibGroup](#) class.

Initializes a [LibGroup](#) instance with the provided si2drGroupIdT and error reference.

Parameters

<i>group</i>	The si2drGroupIdT group identifier.
<i>err</i>	Reference to an si2drErrorT object to store error information.

Definition at line 11 of file [lib_group.cpp](#).

5.8.2.2 ~LibGroup()

```
LibGroup::~LibGroup ( )
```

Definition at line 13 of file [lib_group.cpp](#).

5.8.3 Member Function Documentation

5.8.3.1 getAttrs()

```
si2drAttrsIdT LibGroup::getAttrs ( )
```

Get attributes of the library group.

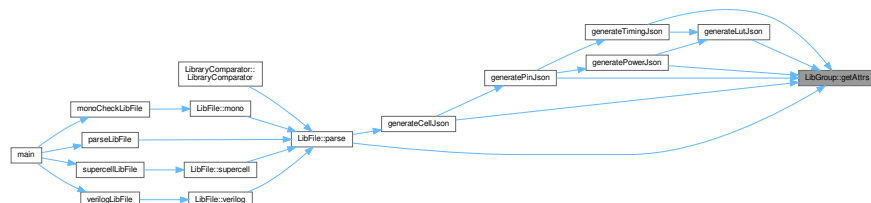
Retrieves the attributes associated with this library group by calling the si2drGroupGetAttrs function on the internal group representation.

Returns

si2drAttrsIdT The identifier for the attributes of this library group.

Definition at line 51 of file [lib_group.cpp](#).

Here is the caller graph for this function:



5.8.3.2 getGroups()

```
si2drGroupsIdT LibGroup::getGroups ( )
```

Retrieves all groups contained within this group.

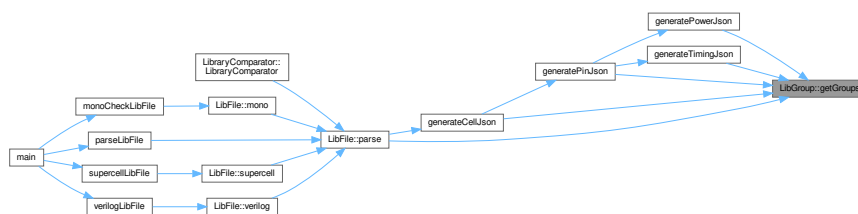
This method calls the si2drGroupGetGroups function to fetch all sub-groups of the current group.

Returns

si2drGroupsIdT A handle to the collection of groups.

Definition at line 61 of file [lib_group.cpp](#).

Here is the caller graph for this function:



5.8.3.3 getName()

```
std::string LibGroup::getName ( )
```

Get the name of the [LibGroup](#).

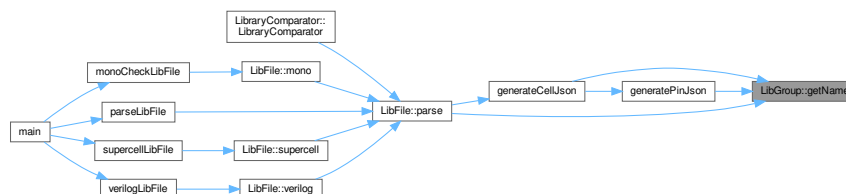
Retrieves the first name associated with the group using the si2dr API. The function properly manages the iterator resources by quitting it after use.

Returns

std::string The name of the group if available, otherwise an empty string.

Definition at line 23 of file [lib_group.cpp](#).

Here is the caller graph for this function:



5.8.3.4 getType()

```
std::string LibGroup::getType ( )
```

Retrieves the type of the current group.

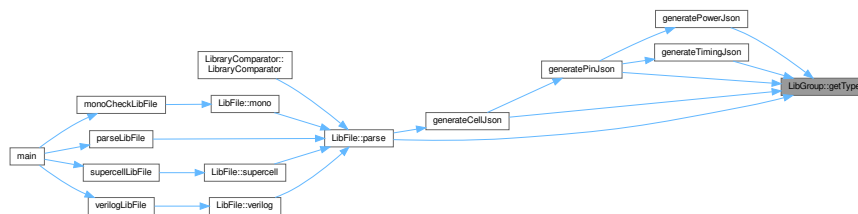
This method queries the group type from the underlying si2dr library.

Returns

A string containing the group type if available, or an empty string if the type cannot be determined or if an error occurs.

Definition at line 38 of file [lib_group.cpp](#).

Here is the caller graph for this function:



5.8.4 Member Data Documentation

5.8.4.1 err_

```
si2drErrorT& LibGroup::err_ [private]
```

Definition at line 19 of file [lib_group.hpp](#).

5.8.4.2 group_

```
si2drGroupIdT LibGroup::group_ [private]
```

Definition at line 18 of file [lib_group.hpp](#).

The documentation for this class was generated from the following files:

- [include/lib_group.hpp](#)
- [src/lib_group.cpp](#)

5.9 LibraryComparator Class Reference

```
#include <compare.hpp>
```

Public Member Functions

- [LibraryComparator](#) ([LibFile](#) &ref_libfile, [LibFile](#) &comp_libfile, double reltol, double abstol)
Constructs a [LibraryComparator](#) object.
- void [generateReport](#) (const std::string &output_file)
Generates a comparison report between the reference and comparison libraries.

Public Attributes

- std::filesystem::path [ref_lib_path_](#)
- std::filesystem::path [comp_lib_path_](#)

Private Member Functions

- void [compareCell](#) (const std::string &cell_name, const [json](#) &ref_cell, const [json](#) &comp_cell, Table &table)
Compares the output pins of a cell between a reference JSON and a comparison JSON.
- void [comparePin](#) (const std::string &cell_name, const std::string &pin_name, const [json](#) &ref_pin, const [json](#) &comp_pin, Table &table)
Compares the timing arcs of a given pin between a reference JSON and a comparison JSON.
- void [compareTimingArc](#) (const std::string &cell_name, const std::string &pin_name, const std::string &timing_type, const [json](#) &ref_timing_arc, const [json](#) &comp_timing_arc, Table &table)
Compares a timing arc between two JSON objects.
- void [compareLut](#) (const std::string &cell_name, const std::string &pin_name, const std::string &timing_type, const std::string &related_pin, const std::string &arc_name, const [json](#) &ref_lut, const [json](#) &comp_lut, Table &table)
Compares lookup tables (LUTs) between reference and comparison libraries for a specific timing arc.

Private Attributes

- [json](#) [ref_json_](#)
- [json](#) [comp_json_](#)
- double [reltol_](#)
- double [abstol_](#)

5.9.1 Detailed Description

Definition at line 12 of file [compare.hpp](#).

5.9.2 Constructor & Destructor Documentation

5.9.2.1 LibraryComparator()

```
LibraryComparator::LibraryComparator (
    LibFile & ref_libfile,
    LibFile & comp_libfile,
    double reltol,
    double abstol )
```

Constructs a [LibraryComparator](#) object.

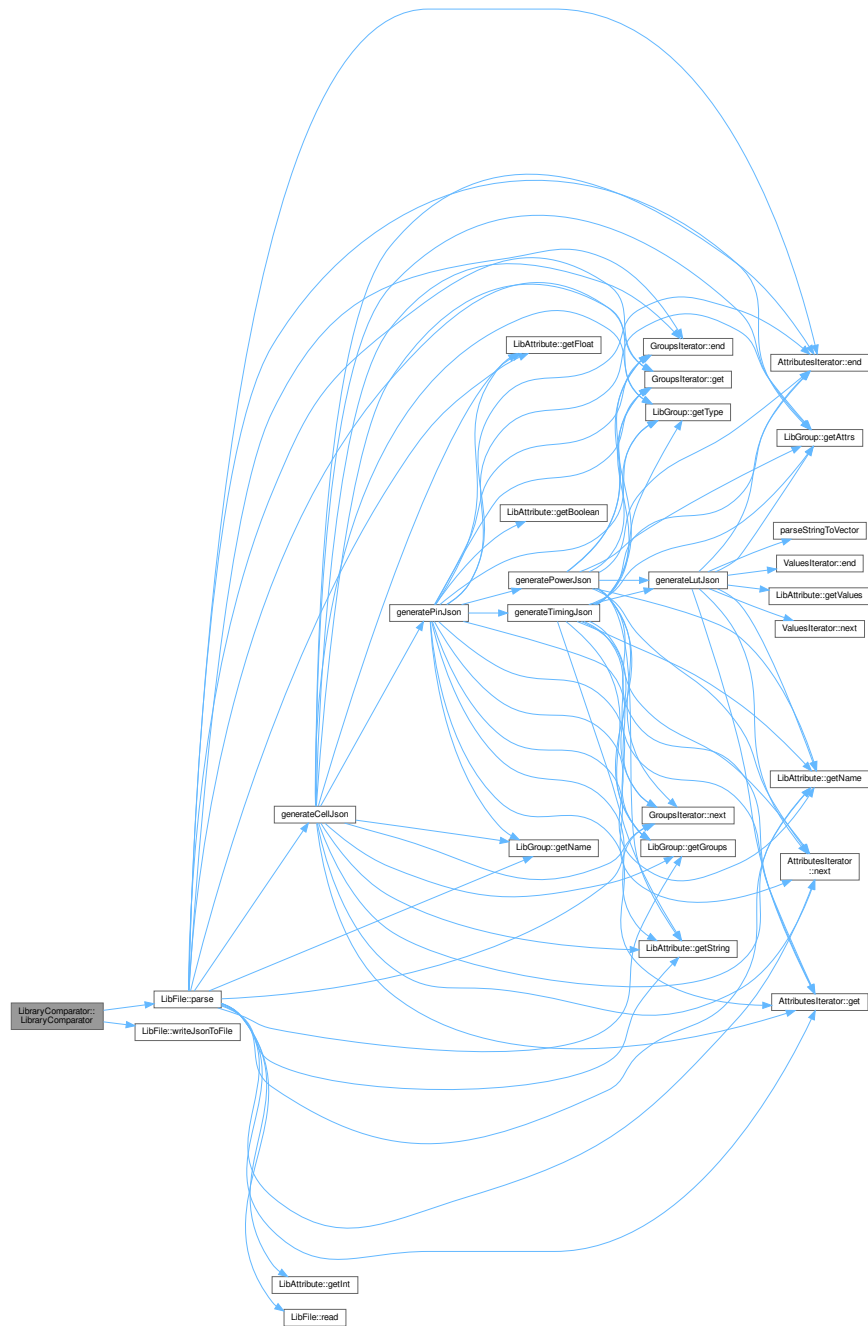
This constructor initializes a [LibraryComparator](#) object by loading and parsing JSON files associated with the provided reference and comparison library files. If the JSON files do not exist, they are generated by parsing the library files.

Parameters

<i>ref_libfile</i>	Reference to the reference library file object.
<i>comp_libfile</i>	Reference to the comparison library file object.
<i>reltol</i>	Relative tolerance value for comparison.

Definition at line 23 of file [compare.cpp](#).

Here is the call graph for this function:



5.9.3 Member Function Documentation

5.9.3.1 compareCell()

```
void LibraryComparator::compareCell (
    const std::string & cell_name,
```

```
const json & ref_cell,
const json & comp_cell,
Table & table ) [private]
```

Compares the output pins of a cell between a reference JSON and a comparison JSON.

This function iterates through the output pins of the comparison cell and checks if each pin exists in the reference cell. If a pin is found in both cells, it calls the comparePin function to compare the details of the pin. If a pin is not found in the reference cell, a warning is logged. If the comparison cell does not contain any output pins, an informational message is logged.

Parameters

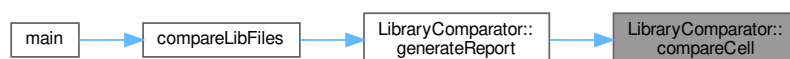
<i>cell_name</i>	The name of the cell being compared.
<i>ref_cell</i>	The reference JSON object containing the cell data.
<i>comp_cell</i>	The comparison JSON object containing the cell data.
<i>table</i>	The table object where the comparison results are stored.

Definition at line 298 of file [compare.cpp](#).

Here is the call graph for this function:



Here is the caller graph for this function:



5.9.3.2 compareLut()

```
void LibraryComparator::compareLut (
    const std::string & cell_name,
    const std::string & pin_name,
    const std::string & timing_type,
    const std::string & related_pin,
    const std::string & arc_name,
    const json & ref_lut,
    const json & comp_lut,
    Table & table ) [private]
```

Compares lookup tables (LUTs) between reference and comparison libraries for a specific timing arc.

This function compares the lookup table values between reference and comparison libraries for a given timing arc, checking the consistency of indices and values. It verifies that the indices match between libraries and then compares each value in the LUT against relative and absolute tolerance thresholds. Any discrepancies that exceed the tolerance limits are recorded in the provided table.

Parameters

<i>cell_name</i>	The name of the cell containing the LUT.
<i>pin_name</i>	The name of the pin associated with the LUT.
<i>timing_type</i>	The timing type of the arc (e.g., "rise_transition", "fall_transition").
<i>related_pin</i>	The related pin for the timing arc.
<i>arc_name</i>	The name of the specific timing arc being compared.
<i>ref_lut</i>	The lookup table from the reference library (in JSON format).
<i>comp_lut</i>	The lookup table from the comparison library (in JSON format).
<i>table</i>	Table object to store comparison results for any mismatches found.

Note

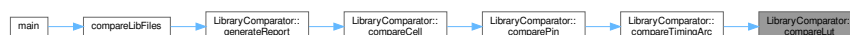
The function logs various information/warning/error messages:

- Logs index mismatches as warnings
- Logs value mismatches as debug messages
- Logs format errors in LUT structure as errors
- Records values exceeding tolerance in the provided table

The comparison uses both relative tolerance (*reltol_*) and absolute tolerance (*abstol_*) to determine if values differ significantly.

Definition at line 98 of file [compare.cpp](#).

Here is the caller graph for this function:



5.9.3.3 comparePin()

```

void LibraryComparator::comparePin (
    const std::string & cell_name,
    const std::string & pin_name,
    const json & ref_pin,
    const json & comp_pin,
    Table & table ) [private]
  
```

Compares the timing arcs of a given pin between a reference JSON and a comparison JSON.

This function iterates through the timing arcs of the comparison pin and looks for matching timing types in the reference pin. If a matching timing type is found, it calls the `compareTimingArc` function to compare the timing arcs. If a timing type is not found in the reference JSON, a warning is logged.

Parameters

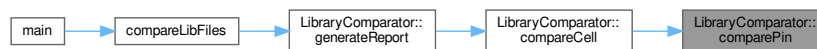
<i>cell_name</i>	The name of the cell containing the pin.
<i>pin_name</i>	The name of the pin to compare.
<i>ref_pin</i>	The reference JSON object containing the pin's data.
<i>comp_pin</i>	The comparison JSON object containing the pin's data.
<i>table</i>	A reference to a Table object where comparison results are stored.

Definition at line 259 of file [compare.cpp](#).

Here is the call graph for this function:



Here is the caller graph for this function:



5.9.3.4 compareTimingArc()

```

void LibraryComparator::compareTimingArc (
    const std::string & cell_name,
    const std::string & pin_name,
    const std::string & timing_type,
    const json & ref_timing_arc,
    const json & comp_timing_arc,
    Table & table ) [private]
  
```

Compares a timing arc between two JSON objects.

This function compares a specific timing arc (e.g., `cell_rise`, `cell_fall`, `rise_transition`, `fall_transition`) between a reference JSON object and a comparison JSON object. It extracts the related pin from the reference timing arc and iterates through a predefined list of arc names. If an arc name is found in the comparison timing arc, it checks if the same arc name exists in the reference timing arc. If both exist, it calls the `compareLut` function to compare the LUT data associated with the arc. If the arc name is found in the comparison JSON but not in the reference JSON, a warning message is logged.

Parameters

<i>cell_name</i>	The name of the cell.
<i>pin_name</i>	The name of the pin.
<i>timing_type</i>	The type of timing (e.g., "combinational", "sequential").
<i>ref_timing_arc</i>	The reference JSON object containing the timing arc information.
<i>comp_timing_arc</i>	The comparison JSON object containing the timing arc information.
<i>table</i>	The table to store comparison results.

Definition at line 219 of file [compare.cpp](#).

Here is the call graph for this function:



Here is the caller graph for this function:



5.9.3.5 generateReport()

```
void LibraryComparator::generateReport (
    const std::string & output_file )
```

Generates a comparison report between the reference and comparison libraries.

This function generates a detailed comparison report between the reference library and the comparison library. The report includes metadata such as the reference and comparison library paths, absolute and relative tolerances, and the application details. It also includes a legend explaining various symbols used in the report.

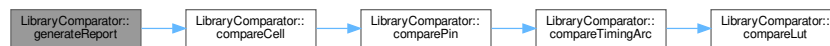
The function iterates through each cell in the comparison library, compares it with the corresponding cell in the reference library, and generates a table of differences. If there are any differences, it calculates summary statistics such as the average and maximum differences, and the number of outliers. The results are written to the specified output file in either Markdown or plain text format.

Parameters

<code>output_file</code>	The path to the output file where the report will be written.
--------------------------	---

Definition at line 340 of file [compare.cpp](#).

Here is the call graph for this function:



Here is the caller graph for this function:



5.9.4 Member Data Documentation

5.9.4.1 abstol_

```
double LibraryComparator::abstol_ [private]
```

Definition at line 23 of file [compare.hpp](#).

5.9.4.2 comp_json_

```
json LibraryComparator::comp_json_ [private]
```

Definition at line 21 of file [compare.hpp](#).

5.9.4.3 comp_lib_path_

```
std::filesystem::path LibraryComparator::comp_lib_path_
```

Definition at line 16 of file [compare.hpp](#).

5.9.4.4 ref_json_

```
json LibraryComparator::ref_json_ [private]
```

Definition at line 20 of file [compare.hpp](#).

5.9.4.5 ref_lib_path_

```
std::filesystem::path LibraryComparator::ref_lib_path_
```

Definition at line 15 of file [compare.hpp](#).

5.9.4.6 reltol_

```
double LibraryComparator::reltol_ [private]
```

Definition at line 22 of file [compare.hpp](#).

The documentation for this class was generated from the following files:

- [include/compare.hpp](#)
- [src/compare.cpp](#)

5.10 si2drExprT Struct Reference

```
#include <si2dr_liberty.h>
```

Collaboration diagram for si2drExprT:



Public Attributes

- [si2drExprTypeT](#) type
- union {
 - [si2drInt32T](#) i
 - [si2drFloat64T](#) d
 - [si2drStringT](#) s
 - [si2drBooleanT](#) b
- [si2drValueTypeT](#) valuetype
- struct [si2drExprT](#) * left
- struct [si2drExprT](#) * right

5.10.1 Detailed Description

Definition at line 172 of file [si2dr_liberty.h](#).

5.10.2 Member Data Documentation

5.10.2.1 b

```
si2drBooleanT si2drExprT::b
```

Definition at line 180 of file [si2dr_liberty.h](#).

5.10.2.2 d

```
si2drFloat64T si2drExprT::d
```

Definition at line 178 of file [si2dr_liberty.h](#).

5.10.2.3 i

```
si2drInt32T si2drExprT::i
```

Definition at line 177 of file [si2dr_liberty.h](#).

5.10.2.4 left

```
struct si2drExprT* si2drExprT::left
```

Definition at line [185](#) of file [si2dr_liberty.h](#).

5.10.2.5 right

```
struct si2drExprT* si2drExprT::right
```

Definition at line [186](#) of file [si2dr_liberty.h](#).

5.10.2.6 s

```
si2drStringT si2drExprT::s
```

Definition at line [179](#) of file [si2dr_liberty.h](#).

5.10.2.7 type

```
si2drExprTypeT si2drExprT::type
```

Definition at line [174](#) of file [si2dr_liberty.h](#).

5.10.2.8

```
union { ... } si2drExprT::u
```

5.10.2.9 valuetype

```
si2drValueTypeT si2drExprT::valuetype
```

Definition at line [183](#) of file [si2dr_liberty.h](#).

The documentation for this struct was generated from the following file:

- [include/si2dr_liberty.h](#)

5.11 si2OID Struct Reference

```
#include <si2dr_liberty.h>
```

Public Attributes

- void * [v1](#)
- void * [v2](#)

5.11.1 Detailed Description

Definition at line 79 of file [si2dr_liberty.h](#).

5.11.2 Member Data Documentation

5.11.2.1 v1

```
void* si2OID::v1
```

Definition at line 79 of file [si2dr_liberty.h](#).

5.11.2.2 v2

```
void * si2OID::v2
```

Definition at line 79 of file [si2dr_liberty.h](#).

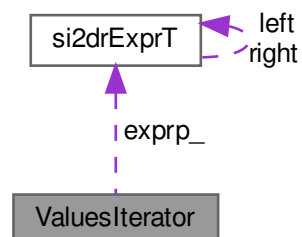
The documentation for this struct was generated from the following file:

- include/[si2dr_liberty.h](#)

5.12 ValuesIterator Class Reference

```
#include <iterators.hpp>
```

Collaboration diagram for ValuesIterator:



Public Member Functions

- [ValuesIterator](#) ([si2drValuesIdT](#) values, [si2drErrorT](#) &err)
Constructs a [ValuesIterator](#).
- [~ValuesIterator](#) ()
- void [next](#) ()
- bool [end](#) ()

Public Attributes

- [si2drValuesIdT](#) values_
- [si2drValueTypeT](#) vtype_
- [si2drInt32T](#) int_
- [si2drFloat64T](#) float_
- [si2drStringT](#) str_
- [si2drBooleanT](#) bool_
- [si2drExprT](#) * exprp_
- [si2drErrorT](#) & err_

5.12.1 Detailed Description

Definition at line 37 of file [iterators.hpp](#).

5.12.2 Constructor & Destructor Documentation

5.12.2.1 ValuesIterator()

```
ValuesIterator::ValuesIterator (
    si2drValuesIdT values,
    si2drErrorT & err )
```

Constructs a [ValuesIterator](#).

This constructor initializes a [ValuesIterator](#) with the specified values and error reference. It also advances the iterator to the first complex value.

Parameters

<i>values</i>	The values identifier to iterate over.
<i>err</i>	Reference to an error object that will be updated with any errors that occur.

Definition at line 51 of file [iterators.cpp](#).

5.12.2.2 ~ValuesIterator()

ValuesIterator::~~ValuesIterator ()

Definition at line 54 of file [iterators.cpp](#).

5.12.3 Member Function Documentation

5.12.3.1 end()

bool ValuesIterator::end ()

Definition at line 59 of file [iterators.cpp](#).

Here is the caller graph for this function:

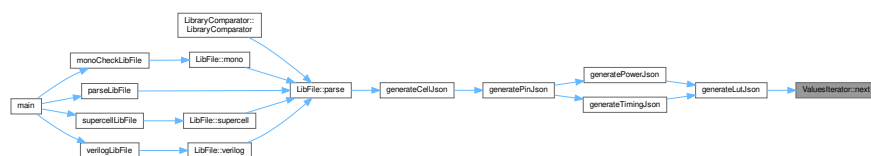


5.12.3.2 next()

void ValuesIterator::next ()

Definition at line 56 of file [iterators.cpp](#).

Here is the caller graph for this function:



5.12.4 Member Data Documentation

5.12.4.1 bool_

`si2drBooleanT` ValuesIterator::bool_

Definition at line 49 of file [iterators.hpp](#).

5.12.4.2 err_

`si2drErrorT&` ValuesIterator::err_

Definition at line 51 of file [iterators.hpp](#).

5.12.4.3 exprp_

`si2drExprT*` ValuesIterator::exprp_

Definition at line 50 of file [iterators.hpp](#).

5.12.4.4 float_

`si2drFloat64T` ValuesIterator::float_

Definition at line 47 of file [iterators.hpp](#).

5.12.4.5 int_

`si2drInt32T` ValuesIterator::int_

Definition at line 46 of file [iterators.hpp](#).

5.12.4.6 str_

`si2drStringT` ValuesIterator::str_

Definition at line 48 of file [iterators.hpp](#).

5.12.4.7 values_

`si2drValuesIdT` `ValuesIterator::values_`

Definition at line 44 of file [iterators.hpp](#).

5.12.4.8 vtype_

`si2drValueTypeT` `ValuesIterator::vtype_`

Definition at line 45 of file [iterators.hpp](#).

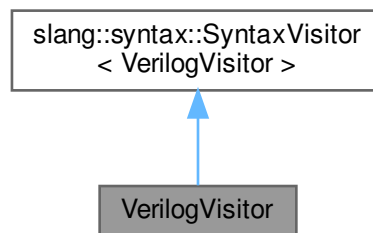
The documentation for this class was generated from the following files:

- [include/iterators.hpp](#)
- [src/iterators.cpp](#)

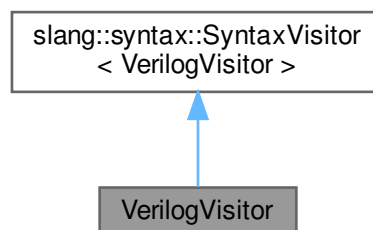
5.13 VerilogVisitor Class Reference

```
#include <verilog_utils.hpp>
```

Inheritance diagram for VerilogVisitor:



Collaboration diagram for VerilogVisitor:



Public Member Functions

- [VerilogVisitor](#) (const std::string &targetCell)
- void [handle](#) (const slang::syntax::SyntaxNode &node)
- void [handle](#) (const slang::syntax::ModuleDeclarationSyntax &module)
- void [handle](#) (const slang::syntax::PortDeclarationSyntax &portDecl)
- void [handle](#) (const slang::syntax::HierarchyInstantiationSyntax &hierarchyInst)
- void [handle](#) (const slang::syntax::PrimitiveInstantiationSyntax &primitiveInst)
- void [handle](#) (const slang::syntax::SpecifyBlockSyntax &specifyBlock)

Private Attributes

- std::string [targetCell_](#)
- int [depth_](#)
- bool [inTargetModule_](#)

5.13.1 Detailed Description

Definition at line 13 of file [verilog_utils.hpp](#).

5.13.2 Constructor & Destructor Documentation

5.13.2.1 VerilogVisitor()

```
VerilogVisitor::VerilogVisitor (  
    const std::string & targetCell ) [explicit]
```

Definition at line 8 of file [verilog_utils.cpp](#).

5.13.3 Member Function Documentation

5.13.3.1 handle() [1/6]

```
void VerilogVisitor::handle (  
    const slang::syntax::HierarchyInstantiationSyntax & hierarchyInst )
```

Definition at line 83 of file [verilog_utils.cpp](#).

5.13.3.2 handle() [2/6]

```
void VerilogVisitor::handle (
    const slang::syntax::ModuleDeclarationSyntax & module )
```

Definition at line 22 of file [verilog_utils.cpp](#).

5.13.3.3 handle() [3/6]

```
void VerilogVisitor::handle (
    const slang::syntax::PortDeclarationSyntax & portDecl )
```

Definition at line 51 of file [verilog_utils.cpp](#).

5.13.3.4 handle() [4/6]

```
void VerilogVisitor::handle (
    const slang::syntax::PrimitiveInstantiationSyntax & primitiveInst )
```

Definition at line 196 of file [verilog_utils.cpp](#).

5.13.3.5 handle() [5/6]

```
void VerilogVisitor::handle (
    const slang::syntax::SpecifyBlockSyntax & specifyBlock )
```

Definition at line 263 of file [verilog_utils.cpp](#).

Here is the call graph for this function:



5.13.3.6 `handle()` [6/6]

```
void VerilogVisitor::handle (
    const slang::syntax::SyntaxNode & node )
```

Definition at line 11 of file [verilog_utils.cpp](#).

Here is the caller graph for this function:



5.13.4 Member Data Documentation

5.13.4.1 `depth_`

```
int VerilogVisitor::depth_ [private]
```

Definition at line 25 of file [verilog_utils.hpp](#).

5.13.4.2 `inTargetModule_`

```
bool VerilogVisitor::inTargetModule_ [private]
```

Definition at line 26 of file [verilog_utils.hpp](#).

5.13.4.3 `targetCell_`

```
std::string VerilogVisitor::targetCell_ [private]
```

Definition at line 24 of file [verilog_utils.hpp](#).

The documentation for this class was generated from the following files:

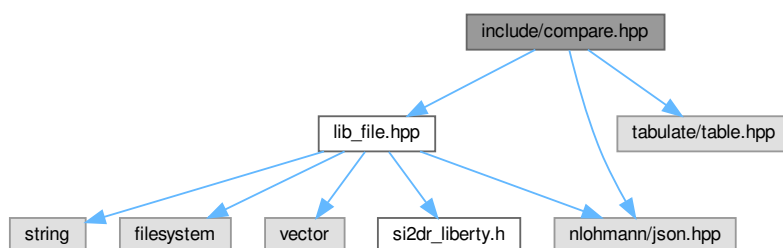
- [include/verilog_utils.hpp](#)
- [src/verilog_utils.cpp](#)

Chapter 6

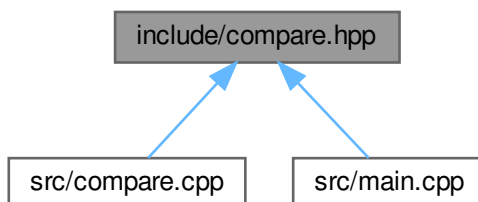
File Documentation

6.1 include/compare.hpp File Reference

```
#include "lib_file.hpp"
#include "nlohmann/json.hpp"
#include "tabulate/table.hpp"
Include dependency graph for compare.hpp:
```



This graph shows which files directly or indirectly include this file:



Classes

- class [LibraryComparator](#)

Typedefs

- using [json](#) = [nlohmann::json](#)

6.1.1 Typedef Documentation

6.1.1.1 json

using [json](#) = [nlohmann::json](#)

Definition at line 9 of file [compare.hpp](#).

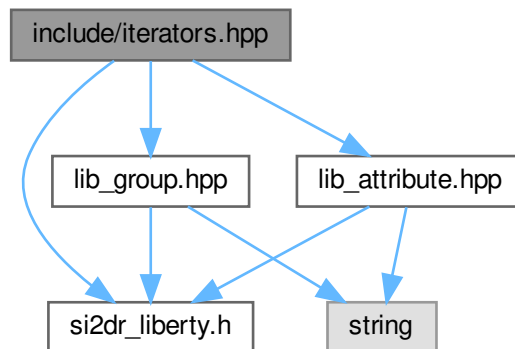
6.2 compare.hpp

[Go to the documentation of this file.](#)

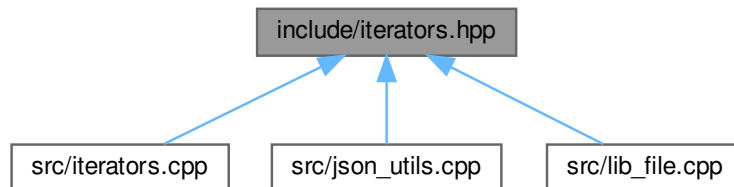
```
00001 #ifndef COMPARE_HPP
00002 #define COMPARE_HPP
00003
00004 #include "lib_file.hpp"
00005
00006 #include "nlohmann/json.hpp"
00007 #include "tabulate/table.hpp"
00008
00009 using json = nlohmann::json;
00010 using namespace tabulate;
00011
00012 class LibraryComparator {
00013 public:
00014     LibraryComparator(LibFile &ref_libfile, LibFile &comp_libfile, double reltol, double abstol);
00015     std::filesystem::path ref_lib_path_;
00016     std::filesystem::path comp_lib_path_;
00017     void generateReport(const std::string &output_file);
00018
00019 private:
00020     json ref_json_;
00021     json comp_json_;
00022     double reltol_;
00023     double abstol_;
00024
00025     void compareCell(const std::string &cell_name, const json &ref_cell, const json &comp_cell,
00026                     Table &table);
00027     void comparePin(const std::string &cell_name, const std::string &pin_name, const json &ref_pin,
00028                    const json &comp_pin, Table &table);
00029     void compareTimingArc(const std::string &cell_name, const std::string &pin_name,
00030                           const std::string &timing_type, const json &ref_timing_arc,
00031                           const json &comp_timing_arc, Table &table);
00032     void compareLut(const std::string &cell_name, const std::string &pin_name,
00033                     const std::string &timing_type, const std::string &related_pin,
00034                     const std::string &arc_name, const json &ref_lut,
00035                     const json &comp_lut, Table &table);
00036     // void configureTableFormat(Table &table);
00037 };
00038
00039 #endif // COMPARE_HPP
```

6.3 include/iterators.hpp File Reference

```
#include "si2dr_liberty.h"  
#include "lib_attribute.hpp"  
#include "lib_group.hpp"  
Include dependency graph for iterators.hpp:
```



This graph shows which files directly or indirectly include this file:



Classes

- class [GroupsIterator](#)
- class [AttributesIterator](#)
- class [ValuesIterator](#)

6.4 iterators.hpp

[Go to the documentation of this file.](#)

```
00001 #ifndef ITERATORS_H  
00002 #define ITERATORS_H
```

```

00003
00004 #include "si2dr_liberty.h"
00005
00006 #include "lib_attribute.hpp"
00007 #include "lib_group.hpp"
00008
00009 class GroupsIterator {
00010 public:
00011     GroupsIterator(si2drGroupsIdT groups, si2drErrorT &err);
00012     ~GroupsIterator();
00013     void next();
00014     bool end();
00015     LibGroup get();
00016
00017 private:
00018     si2drGroupsIdT groups_;
00019     si2drGroupIdT group_;
00020     si2drErrorT &err_;
00021 };
00022
00023 class AttributesIterator {
00024 public:
00025     AttributesIterator(si2drAttrsIdT attrs, si2drErrorT &err);
00026     ~AttributesIterator();
00027     void next();
00028     bool end();
00029     LibAttribute get();
00030
00031 private:
00032     si2drAttrsIdT attrs_;
00033     si2drAttrIdT attr_;
00034     si2drErrorT &err_;
00035 };
00036
00037 class ValuesIterator {
00038 public:
00039     ValuesIterator(si2drValuesIdT values, si2drErrorT &err);
00040     ~ValuesIterator();
00041     void next();
00042     bool end();
00043
00044     si2drValuesIdT values_;
00045     si2drValueTypeT vtype_;
00046     si2drInt32T int_;
00047     si2drFloat64T float_;
00048     si2drStringT str_;
00049     si2drBooleanT bool_;
00050     si2drExprT *exprp_;
00051     si2drErrorT &err_;
00052 };
00053
00054 #endif // ITERATORS_H

```

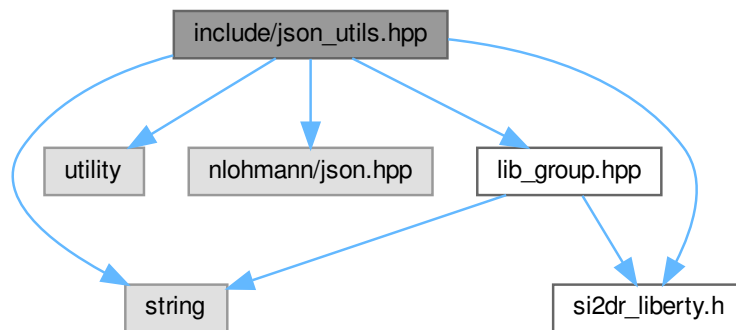
6.5 include/json_utils.hpp File Reference

```

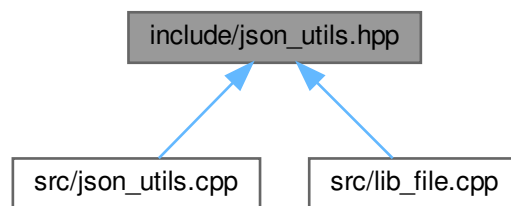
#include <string>
#include <utility>
#include "nlohmann/json.hpp"
#include "si2dr_liberty.h"
#include "lib_group.hpp"

```

Include dependency graph for json_utils.hpp:



This graph shows which files directly or indirectly include this file:



Typedefs

- using `json` = `nlohmann::json`

Functions

- `json generateLutJson (LibGroup &lib_lut_group, si2drErrorT &err)`
Generates a JSON object representing a lookup table (LUT) from a given library group.
- `json generatePowerJson (LibGroup &lib_power_group, si2drErrorT &err)`
Generates a JSON object representation of a power group from Liberty format.
- `std::pair< std::string, json > generatePinJson (LibGroup &lib_pin_group, si2drErrorT &err)`
Generates a JSON representation of a library pin group.
- `json generateCellJson (LibGroup &lib_cell_group, si2drErrorT &err)`
Generates a JSON representation of a library cell group.

6.5.1 Typedef Documentation

6.5.1.1 json

```
using json = nlohmann::json
```

Definition at line 12 of file [json_utils.hpp](#).

6.5.2 Function Documentation

6.5.2.1 generateCellJson()

```
json generateCellJson (
    LibGroup & lib_cell_group,
    si2drErrorT & err )
```

Generates a JSON representation of a library cell group.

This function builds a JSON object containing cell properties by extracting data from the provided [LibGroup](#) object. It processes:

- Cell name
- Basic attributes (area, cell_leakage_power, cell_footprint)
- Pin information (categorized by direction: input, output, internal, inout)

The function iterates through both attributes and sub-groups of the cell group, converting relevant information to a structured JSON format.

Parameters

<i>lib_cell_group</i>	The LibGroup object representing the cell
<i>err</i>	Reference to an error tracking object used during iteration

Returns

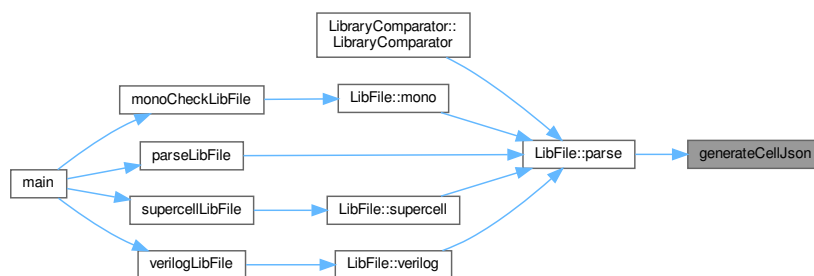
json A JSON object containing the cell's structured data

Definition at line 280 of file [json_utils.cpp](#).

Here is the call graph for this function:



Here is the caller graph for this function:



6.5.2.2 generateLutJson()

```
json generateLutJson (
    LibGroup & lib_lut_group,
    si2drErrorT & err )
```

Generates a JSON object representing a lookup table (LUT) from a given library group.

This function iterates over the attributes of the provided library group and processes them to generate a JSON object. It specifically handles attributes named "index_1", "index_2", and "values", converting their string representations into vectors and adding them to the JSON object. If an attribute with an unknown name is encountered, a warning is logged.

Parameters

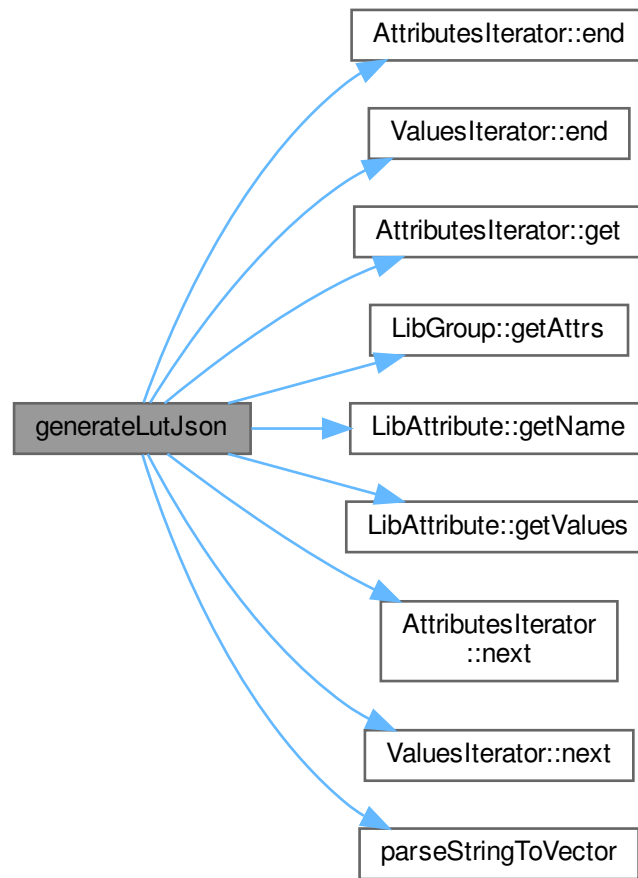
<i>lib_lut_group</i>	The library group containing the LUT attributes.
<i>err</i>	Reference to an error variable to capture any errors during attribute iteration.

Returns

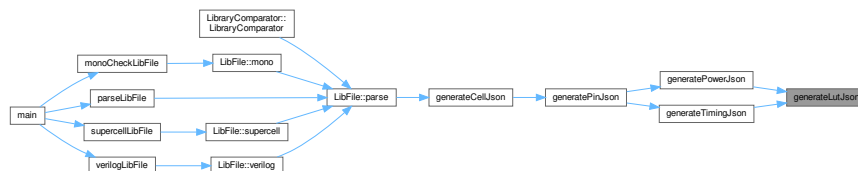
A JSON object representing the LUT.

Definition at line 57 of file [json_utils.cpp](#).

Here is the call graph for this function:



Here is the caller graph for this function:



6.5.2.3 generatePinJson()

```

std::pair< std::string, json > generatePinJson (
    LibGroup & lib_pin_group,
    si2drErrorT & err )
  
```

Generates a JSON representation of a library pin group.

This function processes a Liberty format pin group and converts it into a JSON object. It extracts various pin attributes such as direction, capacitance values, and functional attributes. It also recursively processes sub-groups including timing and power arcs.

Parameters

<i>lib_pin_group</i>	The Liberty pin group to be processed
<i>err</i>	Reference to a <code>si2drErrorT</code> object to track any errors that occur during processing

Returns

A pair containing:

- first: The pin direction as a string ("input", "output", "inout", etc.)
- second: A JSON object containing all the extracted pin information including:
 - pin_name: Name of the pin
 - direction: Pin direction (stored in the returned pair's first element)
 - Various attributes like max_transition, capacitance values, function, etc.
 - timing_arcs: Array of timing arcs if present
 - power_arcs: Array of power arcs if present

Note

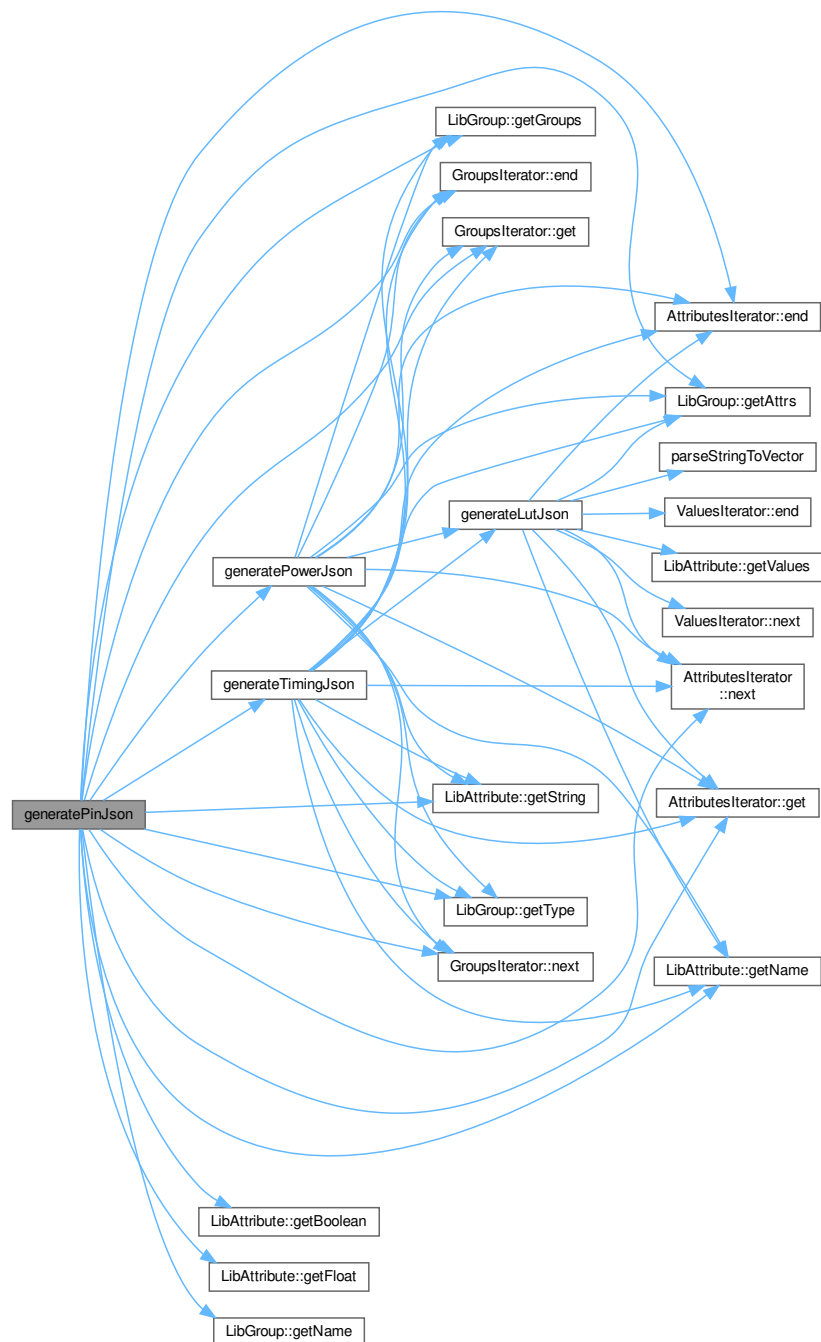
Handles the following pin attributes:

- direction
- max_transition, capacitance, rise_capacitance, fall_capacitance, max_capacitance
- function, power_down_function, related_ground_pin, related_power_pin, three_state
- clock

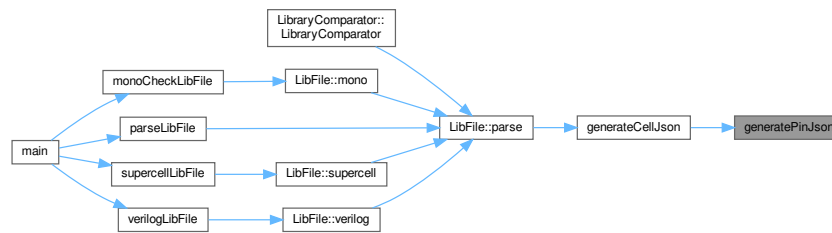
Processes "internal_power" and "timing" sub-groups recursively

Definition at line 218 of file [json_utils.cpp](#).

Here is the call graph for this function:



Here is the caller graph for this function:



6.5.2.4 generatePowerJson()

```

json generatePowerJson (
    LibGroup & lib_power_group,
    si2drErrorT & err )

```

Generates a JSON object representation of a power group from Liberty format.

This function processes a [LibGroup](#) representing a power group from a Liberty file and converts it to a JSON object. It extracts relevant attributes such as "when", "related_pin", and "related_pg_pin" and processes subgroups like "rise_power" and "fall_power", typically containing lookup tables (LUTs).

Parameters

<i>lib_power_group</i>	The Liberty power group to convert to JSON
<i>err</i>	Reference to a <code>si2drErrorT</code> object for error handling

Returns

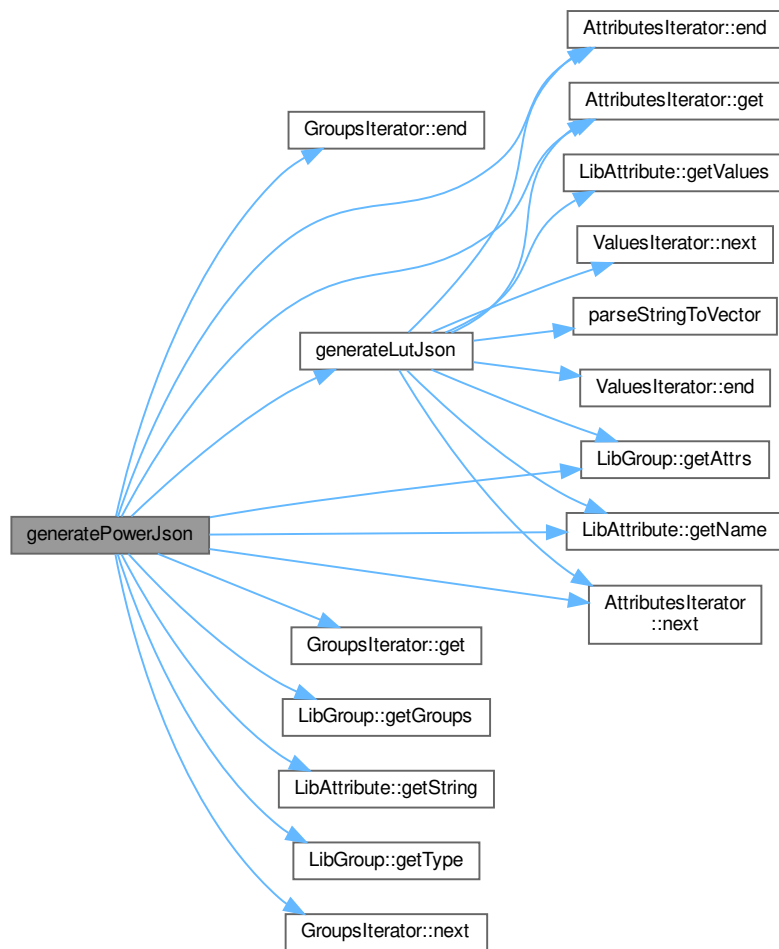
json A JSON object containing the power group information

Note

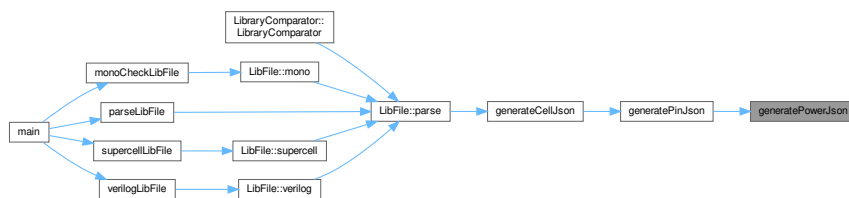
The function currently handles the attributes "when", "related_pin", "related_pg_pin" and the subgroup types "rise_power" and "fall_power". Other subgroup types will generate a warning.

Definition at line 160 of file [json_utils.cpp](#).

Here is the call graph for this function:



Here is the caller graph for this function:



6.6 json_utils.hpp

[Go to the documentation of this file.](#)

```
00001 #ifndef JSON_UTILS_HPP
```

```

00002 #define JSON_UTILS_HPP
00003
00004 #include <string>
00005 #include <utility>
00006
00007 #include "nlohmann/json.hpp"
00008 #include "si2dr_liberty.h"
00009
00010 #include "lib_group.hpp"
00011
00012 using json = nlohmann::json;
00013
00014 json generateLutJson(LibGroup &lib_lut_group, si2drErrorT &err);
00015 json generatePowerJson(LibGroup &lib_power_group, si2drErrorT &err);
00016 std::pair<std::string, json> generatePinJson(LibGroup &lib_pin_group, si2drErrorT &err);
00017 json generateCellJson(LibGroup &lib_cell_group, si2drErrorT &err);
00018
00019 #endif // JSON_UTILS_HPP

```

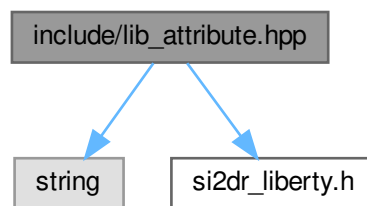
6.7 include/lib_attribute.hpp File Reference

```

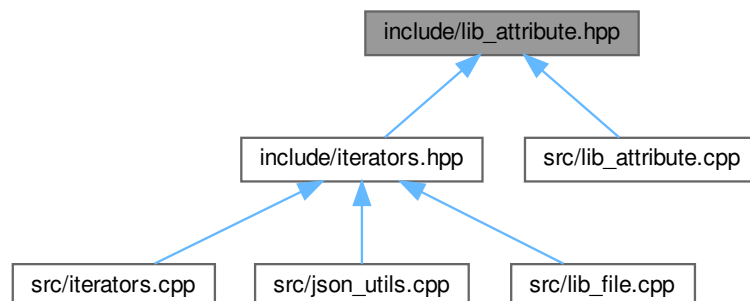
#include <string>
#include "si2dr_liberty.h"

```

Include dependency graph for lib_attribute.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class [LibAttribute](#)

6.8 lib_attribute.hpp

[Go to the documentation of this file.](#)

```

00001 #ifndef LIB_ATTRIBUTE_H
00002 #define LIB_ATTRIBUTE_H
00003
00004 #include <string>
00005
00006 #include "si2dr_liberty.h"
00007
00008 class LibAttribute {
00009 public:
00010     LibAttribute(si2drAttrIdT attr, si2drErrorT &err);
00011     ~LibAttribute();
00012     std::string getName();
00013     bool isComplex();
00014     si2drValuesIdT getValues();
00015     long int getInt();
00016     double getFloat();
00017     std::string getString();
00018     bool getBoolean();
00019
00020 private:
00021     si2drAttrIdT attr_;
00022     si2drErrorT &err_;
00023 };
00024
00025 #endif // LIB_ATTRIBUTE_H

```

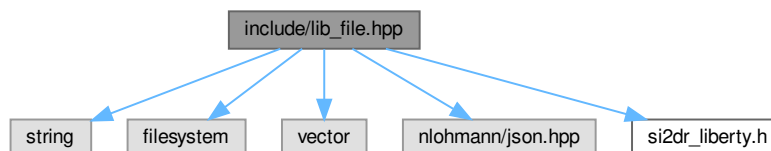
6.9 include/lib_file.hpp File Reference

```

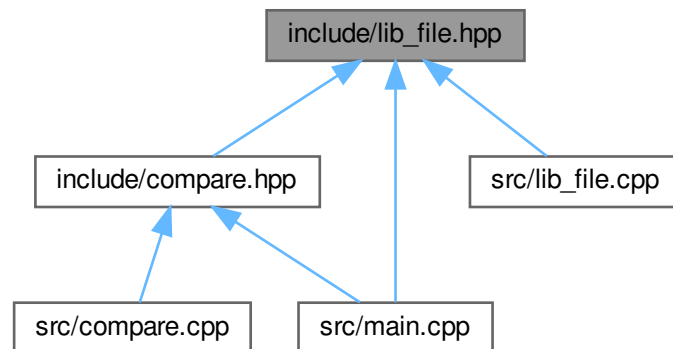
#include <string>
#include <filesystem>
#include <vector>
#include "nlohmann/json.hpp"
#include "si2dr_liberty.h"

```

Include dependency graph for lib_file.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class [LibFile](#)

Typedefs

- using [json](#) = `nlohmann::json`

6.9.1 Typedef Documentation

6.9.1.1 json

```
using json = nlohmann::json
```

Definition at line 11 of file [lib_file.hpp](#).

6.10 lib_file.hpp

[Go to the documentation of this file.](#)

```

00001 #ifndef LIB_FILE_H
00002 #define LIB_FILE_H
00003
00004 #include <string>
00005 #include <filesystem>
00006 #include <vector>
00007
00008 #include "nlohmann/json.hpp"
00009 #include "si2dr_liberty.h"
00010

```

```

00011 using json = nlohmann::json;
00012
00013 class LibFile {
00014 public:
00015     LibFile(const std::string &filepath, const std::string &loggername);
00016     ~LibFile();
00017     std::shared_ptr<spdlog::logger> logger_;
00018     std::filesystem::path filepath_; // full path to the file
00019     std::string basename_; // file name without extension
00020     std::string filename_; // full file name with extension
00021     std::string libname_ = ""; // library name obtained through parsing
00022     std::string jsonname_ = ""; // json file name to store parsed data
00023     std::string loggername_ = ""; // log file name
00024     json lib_json_ = json::object();
00025     void writeJsonToFile();
00026     void parse();
00027     void modify();
00028     void mono(const bool is_slew);
00029     void supercell(const int chain_length, const std::vector<std::string> &cell_names);
00030     void verilog(const int chain_length, const std::vector<std::string> &cell_names);
00031
00032 private:
00033     si2drErrorT err_;
00034     int process_ = 0;
00035     float voltage_ = 0.0;
00036     int temperature_ = 0;
00037     void read();
00038     bool checkTimingArcMonotonicity(const json &cell, const json &pin, const json &arc, const
std::string &type, bool is_slew);
00039 };
00040
00041 #endif // LIB_FILE_H

```

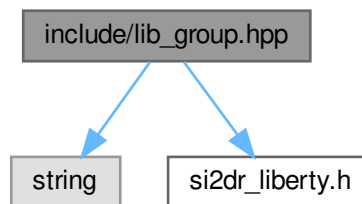
6.11 include/lib_group.hpp File Reference

```

#include <string>
#include "si2dr_liberty.h"

```

Include dependency graph for lib_group.hpp:



```

graph BT
    src_iterators[src/iterators.cpp] --> include_iterators[include/iterators.hpp]
    src_lib_file[src/lib_file.cpp] --> include_iterators
    src_lib_file --> include_json_utils[include/json_utils.hpp]
    src_json_utils[src/json_utils.cpp] --> include_json_utils
    include_iterators --> include_lib_group[include/lib_group.hpp]
    include_json_utils --> include_lib_group
    src_lib_group[src/lib_group.cpp] --> include_lib_group
    style include_lib_group fill:#ccc
  
```

- class LibGroup

[Go to the documentation of this file.](#)

6.13 include/si2dr_liberty.h File Reference

```

graph TD
    si2dr_liberty["include/si2dr_liberty.h"]
    lib_attrbute_hpp["include/lib_attrbute.hpp"]
    lib_group_hpp["include/lib_group.hpp"]
    lib_file_hpp["include/lib_file.hpp"]
    iterators_hpp["include/iterators.hpp"]
    json_utils_hpp["include/json_utils.hpp"]
    compare_hpp["include/compare.hpp"]
    lib_attrbute_cpp["src/lib_attrbute.cpp"]
    lib_group_cpp["src/lib_group.cpp"]
    json_utils_cpp["src/json_utils.cpp"]
    lib_file_cpp["src/lib_file.cpp"]
    compare_cpp["src/compare.cpp"]
    main_cpp["src/main.cpp"]

    lib_attrbute_hpp --> si2dr_liberty
    lib_group_hpp --> si2dr_liberty
    lib_file_hpp --> si2dr_liberty
    lib_attrbute_cpp --> lib_attrbute_hpp
    iterators_hpp --> lib_attrbute_hpp
    lib_group_hpp --> iterators_hpp
    json_utils_hpp --> lib_file_hpp
    compare_hpp --> lib_file_hpp
    main_cpp --> lib_file_hpp
    iterators_hpp --> iterators_hpp
    json_utils_hpp --> iterators_hpp
    compare_hpp --> iterators_hpp
    json_utils_hpp --> json_utils_hpp
    lib_file_cpp --> json_utils_hpp
    compare_hpp --> compare_hpp
    main_cpp --> compare_hpp
  
```

Classes

- struct [si2OID](#)
- struct [si2drExprT](#)
- struct [liberty_value_data](#)

Macros

- #define [SI2_ARGS](#)(args) ()
- #define [SI2_LONG_MAX](#) 2147483647
- #define [SI2_LONG_MIN](#) -2147483647
- #define [SI2_ULONG_MAX](#) 4294967295
- #define [SI2DR_MIN_FLOAT32](#) -1E+37
- #define [SI2DR_MAX_FLOAT32](#) 1E+37
- #define [SI2DR_MIN_FLOAT64](#) -1.797693e+308
- #define [SI2DR_MAX_FLOAT64](#) 1.797693e+308
- #define [SI2DR_MIN_INT32](#) -2147483647
- #define [SI2DR_MAX_INT32](#) 2147483647
- #define [SI2DR_MAX_STRING_LEN](#) 1048576 /*1024*1024*/
- #define [SI2DR_TRUE](#) SI2_TRUE
- #define [SI2DR_FALSE](#) SI2_FALSE
- #define [LONG_DOUBLE](#) long double

Typedefs

- typedef enum [si2BooleanEnum](#) [si2BooleanT](#)
- typedef long int [si2LongT](#)
- typedef float [si2FloatT](#)
- typedef double [si2DoubleT](#)
- typedef char * [si2StringT](#)
- typedef void [si2VoidT](#)
- typedef enum [si2TypeEnum](#) [si2TypeT](#)
- typedef struct [si2OID](#) [si2ObjectIdT](#)
- typedef void * [si2IteratorIdT](#)
- typedef [si2BooleanT](#) [si2drBooleanT](#)
- typedef [si2LongT](#) [si2drInt32T](#)
- typedef [si2FloatT](#) [si2drFloat32T](#)
- typedef [si2DoubleT](#) [si2drFloat64T](#)
- typedef [si2StringT](#) [si2drStringT](#)
- typedef [si2VoidT](#) [si2drVoidT](#)
- typedef [si2IteratorIdT](#) [si2drIterIdT](#)
- typedef [si2ObjectIdT](#) [si2drObjectIdT](#)
- typedef [si2drObjectIdT](#) [si2drGroupIdT](#)
- typedef [si2drObjectIdT](#) [si2drAttrIdT](#)
- typedef [si2drObjectIdT](#) [si2drAttrComplexValIdT](#)
- typedef [si2drObjectIdT](#) [si2drDefineIdT](#)
- typedef [si2drIterIdT](#) [si2drObjectsIdT](#)
- typedef [si2drIterIdT](#) [si2drGroupsIdT](#)
- typedef [si2drIterIdT](#) [si2drAttrIdT](#)
- typedef [si2drIterIdT](#) [si2drDefinesIdT](#)
- typedef [si2drIterIdT](#) [si2drNamesIdT](#)
- typedef [si2drIterIdT](#) [si2drValuesIdT](#)
- typedef enum [si2drSeverityT](#) [si2drSeverityT](#)

- typedef enum [si2drObjectTypeT](#) [si2drObjectTypeT](#)
- typedef enum [si2drAttrTypeT](#) [si2drAttrTypeT](#)
- typedef enum [si2drValueTypeT](#) [si2drValueTypeT](#)
- typedef enum [si2drExprTypeT](#) [si2drExprTypeT](#)
- typedef struct [si2drExprT](#) [si2drExprT](#)
- typedef enum [si2drErrorT](#) [si2drErrorT](#)
- typedef [si2drVoidT](#)(* [si2drMessageHandlerT](#)) ([si2drSeverityT](#) sev, [si2drErrorT](#) errToPrint, [si2drStringT](#) auxText, [si2drErrorT](#) *err)

Enumerations

- enum [si2BooleanEnum](#) { [SI2_FALSE](#) = 0 , [SI2_TRUE](#) = 1 }
- enum [si2TypeEnum](#) {
[SI2_UNDEFINED_VALUETYPE](#) = 0 , [SI2_BOOLEAN](#) = 1 , [SI2_LONG](#) = 2 , [SI2_FLOAT](#) = 3 ,
[SI2_DOUBLE](#) = 4 , [SI2_STRING](#) = 5 , [SI2_VOID](#) = 6 , [SI2_OID](#) = 7 ,
[SI2_ITER](#) = 8 , [SI2_EXPR](#) = 9 , [SI2_MAX_VALUETYPE](#) = 10 }
- enum [si2drSeverityT](#) { [SI2DR_SEVERITY_NOTE](#) = 0 , [SI2DR_SEVERITY_WARN](#) = 1 , [SI2DR_SEVERITY_ERR](#) = 2 }
- enum [si2drObjectTypeT](#) {
[SI2DR_UNDEFINED_OBJECTTYPE](#) = 0 , [SI2DR_GROUP](#) = 1 , [SI2DR_ATTR](#) = 2 , [SI2DR_DEFINE](#) = 3 ,
[SI2DR_COMPLEXVAL](#) = 4 , [SI2DR_MAX_OBJECTTYPE](#) = 5 }
- enum [si2drAttrTypeT](#) { [SI2DR_SIMPLE](#) , [SI2DR_COMPLEX](#) }
- enum [si2drValueTypeT](#) {
[SI2DR_UNDEFINED_VALUETYPE](#) = [SI2_UNDEFINED_VALUETYPE](#) , [SI2DR_INT32](#) = [SI2_LONG](#) ,
[SI2DR_STRING](#) = [SI2_STRING](#) , [SI2DR_FLOAT64](#) = [SI2_DOUBLE](#) ,
[SI2DR_BOOLEAN](#) = [SI2_BOOLEAN](#) , [SI2DR_EXPR](#) = [SI2_EXPR](#) , [SI2DR_MAX_VALUETYPE](#) = [SI2_MAX_VALUETYPE](#) }
- enum [si2drExprTypeT](#) {
[SI2DR_EXPR_VAL](#) = 0 , [SI2DR_EXPR_OP_ADD](#) = 1 , [SI2DR_EXPR_OP_SUB](#) = 2 , [SI2DR_EXPR_OP_MUL](#) = 3 ,
[SI2DR_EXPR_OP_DIV](#) = 4 , [SI2DR_EXPR_OP_PAREN](#) = 5 , [SI2DR_EXPR_OP_LOG2](#) = 6 ,
[SI2DR_EXPR_OP_LOG10](#) = 7 ,
[SI2DR_EXPR_OP_EXP](#) = 8 }
- enum [si2drErrorT](#) {
[SI2DR_NO_ERROR](#) = 0 , [SI2DR_INTERNAL_SYSTEM_ERROR](#) = 1 , [SI2DR_INVALID_VALUE](#) = 4 ,
[SI2DR_INVALID_NAME](#) = 5 ,
[SI2DR_INVALID_OBJECTTYPE](#) = 6 , [SI2DR_INVALID_ATTRTYPE](#) = 10 , [SI2DR_UNUSABLE_OID](#) = 7 ,
[SI2DR_OBJECT_ALREADY_EXISTS](#) = 8 ,
[SI2DR_OBJECT_NOT_FOUND](#) = 9 , [SI2DR_SYNTAX_ERROR](#) = 2 , [SI2DR_TRACE_FILES_CANNOT_BE_OPENED](#) = 3 ,
[SI2DR_PIINIT_NOT_CALLED](#) = 11 ,
[SI2DR_SEMANTIC_ERROR](#) = 12 , [SI2DR_REFERENCE_ERROR](#) = 13 , [SI2DR_MAX_ERROR](#) = 14 }

Functions

- [si2drVoidT](#) [si2drDefaultMessageHandler](#) ([si2drSeverityT](#) sev, [si2drErrorT](#) errToPrint, [si2drStringT](#) auxText, [si2drErrorT](#) *err)
- [si2drMessageHandlerT](#) [si2drPIGetMessageHandler](#) ([si2drErrorT](#) *err)
- [si2drVoidT](#) [si2drPISetMessageHandler](#) ([si2drMessageHandlerT](#) MsgHandFuncPtr, [si2drErrorT](#) *err)
- [si2drGroupIdT](#) [si2drPICreateGroup](#) [SI2_ARGS](#) (([si2drStringT](#) name, [si2drStringT](#) group_type, [si2drErrorT](#) *err))
- [si2drAttrIdT](#) [si2drGroupCreateAttr](#) [SI2_ARGS](#) (([si2drGroupIdT](#) group, [si2drStringT](#) name, [si2drAttrTypeT](#) type, [si2drErrorT](#) *err))
- [si2drAttrTypeT](#) [si2drAttrGetType](#) [SI2_ARGS](#) (([si2drAttrIdT](#) attr, [si2drErrorT](#) *err))
- [si2drVoidT](#) [si2drComplexAttrAddInt32Value](#) [SI2_ARGS](#) (([si2drAttrIdT](#) attr, [si2drInt32T](#) intgr, [si2drErrorT](#) *err))

- `si2drVoidT` `si2drComplexAttrAddStringValue` `SI2_ARGS` ((`si2drAttrIdT` attr, `si2drStringT` string, `si2drErrorT` *err))
- `si2drVoidT` `si2drComplexAttrAddBooleanValue` `SI2_ARGS` ((`si2drAttrIdT` attr, `si2drBooleanT` boolval, `si2drErrorT` *err))
- `si2drVoidT` `si2drComplexAttrAddFloat64Value` `SI2_ARGS` ((`si2drAttrIdT` attr, `si2drFloat64T` float64, `si2drErrorT` *err))
- `si2drVoidT` `si2drComplexAttrAddExprValue` `SI2_ARGS` ((`si2drAttrIdT` attr, `si2drExprT` *expr, `si2drErrorT` *err))
- `si2drVoidT` `si2drIterNextComplexValue` `SI2_ARGS` ((`si2drValuesIdT` iter, `si2drValueTypeT` *type, `si2drInt32T` *intgr, `si2drFloat64T` *float64, `si2drStringT` *string, `si2drBooleanT` *boolval, `si2drExprT` **expr, `si2drErrorT` *err))
- `si2drAttrComplexValIdT` `si2drIterNextComplex` `SI2_ARGS` ((`si2drValuesIdT` iter, `si2drErrorT` *err))
- `si2drVoidT` `si2drSimpleAttrSetBooleanValue` `SI2_ARGS` ((`si2drAttrIdT` attr, `si2drBooleanT` intgr, `si2drErrorT` *err))
- `si2drVoidT` `si2drExprDestroy` `SI2_ARGS` ((`si2drExprT` *expr, `si2drErrorT` *err))
- `si2drExprT` `*si2drCreateExpr` `SI2_ARGS` ((`si2drExprTypeT` type, `si2drErrorT` *err))
- `si2drExprT` `*si2drCreateBooleanValExpr` `SI2_ARGS` ((`si2drBooleanT` b, `si2drErrorT` *err))
- `si2drExprT` `*si2drCreateDoubleValExpr` `SI2_ARGS` ((`si2drFloat64T` d, `si2drErrorT` *err))
- `si2drExprT` `*si2drCreateStringValExpr` `SI2_ARGS` ((`si2drStringT` s, `si2drErrorT` *err))
- `si2drExprT` `*si2drCreateIntValExpr` `SI2_ARGS` ((`si2drInt32T` i, `si2drErrorT` *err))
- `si2drExprT` `*si2drCreateBinaryOpExpr` `SI2_ARGS` ((`si2drExprT` *left, `si2drExprTypeT` optype, `si2drExprT` *right, `si2drErrorT` *err))
- `si2drExprT` `*si2drCreateUnaryOpExpr` `SI2_ARGS` ((`si2drExprTypeT` optype, `si2drExprT` *expr, `si2drErrorT` *err))
- `si2drDefinelDt` `si2drGroupCreateDefine` `SI2_ARGS` ((`si2drGroupIdT` group, `si2drStringT` name, `si2drStringT` allowed_group_name, `si2drValueTypeT` valtype, `si2drErrorT` *err))
- `si2drVoidT` `si2drDefineGetInfo` `SI2_ARGS` ((`si2drDefinelDt` def, `si2drStringT` *name, `si2drStringT` *allowed_group_name, `si2drValueTypeT` *valtype, `si2drErrorT` *err))
- `si2drStringT` `si2drDefineGetName` `SI2_ARGS` ((`si2drDefinelDt` def, `si2drErrorT` *err))
- `si2drGroupIdT` `si2drGroupCreateGroup` `SI2_ARGS` ((`si2drGroupIdT` group, `si2drStringT` name, `si2drStringT` group_type, `si2drErrorT` *err))
- `si2drStringT` `si2drGroupGetGroupType` `SI2_ARGS` ((`si2drGroupIdT` group, `si2drErrorT` *err))
- `si2drVoidT` `si2drGroupSetComment` `SI2_ARGS` ((`si2drGroupIdT` group, `si2drStringT` comment, `si2drErrorT` *err))
- `si2drVoidT` `si2drAttrSetComment` `SI2_ARGS` ((`si2drAttrIdT` attr, `si2drStringT` comment, `si2drErrorT` *err))
- `si2drVoidT` `si2drDefineSetComment` `SI2_ARGS` ((`si2drDefinelDt` def, `si2drStringT` comment, `si2drErrorT` *err))
- `si2drVoidT` `si2drGroupAddName` `SI2_ARGS` ((`si2drGroupIdT` group, `si2drStringT` name, `si2drErrorT` *err))
- `si2drGroupIdT` `si2drPIFindGroupByName` `SI2_ARGS` ((`si2drStringT` name, `si2drStringT` type, `si2drErrorT` *err))
- `si2drGroupIdT` `si2drGroupFindGroupByName` `SI2_ARGS` ((`si2drGroupIdT` group, `si2drStringT` name, `si2drStringT` type, `si2drErrorT` *err))
- `si2drDefinelDt` `si2drPIFindDefineByName` `SI2_ARGS` ((`si2drStringT` name, `si2drErrorT` *err))
- `si2drGroupsIdT` `si2drPIGetGroups` `SI2_ARGS` ((`si2drErrorT` *err))
- `si2drVoidT` `si2drObjectDelete` `SI2_ARGS` ((`si2drObjectIdT` object, `si2drErrorT` *err))
- `si2drStringT` `si2drPIGetErrorText` `SI2_ARGS` ((`si2drErrorT` errorCode, `si2drErrorT` *err))
- `si2drBooleanT` `si2drObjectIsSame` `SI2_ARGS` ((`si2drObjectIdT` object1, `si2drObjectIdT` object2, `si2drErrorT` *err))
- `si2drVoidT` `si2drObjectSetFileName` `SI2_ARGS` ((`si2drObjectIdT` object, `si2drStringT` filename, `si2drErrorT` *err))
- `si2drVoidT` `si2drObjectSetLineNo` `SI2_ARGS` ((`si2drObjectIdT` object, `si2drInt32T` lineno, `si2drErrorT` *err))
- `si2drVoidT` `si2drReadLibertyFile` `SI2_ARGS` ((char *filename, `si2drErrorT` *err))
- `si2drVoidT` `si2drWriteLibertyFile` `SI2_ARGS` ((char *filename, `si2drGroupIdT` group, char *cellname, `si2drErrorT` *err))
- `si2drVoidT` `si2drPISetTraceMode` `SI2_ARGS` ((`si2drStringT` fname, `si2drErrorT` *err))
- `si2drVoidT` `si2drGroupMoveAfter` `SI2_ARGS` ((`si2drGroupIdT` groupToMove, `si2drGroupIdT` targetGroup, `si2drErrorT` *err))

- [LONG_DOUBLE](#) [liberty_get_element](#) (struct [liberty_value_data](#) *vd,...)
- void [liberty_destroy_value_data](#) (struct [liberty_value_data](#) *vd)
- struct [liberty_value_data](#) * [liberty_get_values_data](#) (si2drGroupIdT table_group)
- void [print_version](#) ()
- void [usage](#) ()
- int [main_liberty_parser](#) (int argc, char *argv[])
- void [get_function](#) (char *filename)

6.13.1 Macro Definition Documentation

6.13.1.1 LONG_DOUBLE

```
#define LONG_DOUBLE long double
```

Definition at line 609 of file [si2dr_liberty.h](#).

6.13.1.2 SI2_ARGS

```
#define SI2_ARGS(  
    args ) ()
```

Definition at line 34 of file [si2dr_liberty.h](#).

6.13.1.3 SI2_LONG_MAX

```
#define SI2_LONG_MAX 2147483647
```

Definition at line 74 of file [si2dr_liberty.h](#).

6.13.1.4 SI2_LONG_MIN

```
#define SI2_LONG_MIN -2147483647
```

Definition at line 75 of file [si2dr_liberty.h](#).

6.13.1.5 SI2_ULONG_MAX

```
#define SI2_ULONG_MAX 4294967295
```

Definition at line 76 of file [si2dr_liberty.h](#).

6.13.1.6 SI2DR_FALSE

```
#define SI2DR_FALSE SI2_FALSE
```

Definition at line 96 of file [si2dr_liberty.h](#).

6.13.1.7 SI2DR_MAX_FLOAT32

```
#define SI2DR_MAX_FLOAT32 1E+37
```

Definition at line 87 of file [si2dr_liberty.h](#).

6.13.1.8 SI2DR_MAX_FLOAT64

```
#define SI2DR_MAX_FLOAT64 1.797693e+308
```

Definition at line 89 of file [si2dr_liberty.h](#).

6.13.1.9 SI2DR_MAX_INT32

```
#define SI2DR_MAX_INT32 2147483647
```

Definition at line 91 of file [si2dr_liberty.h](#).

6.13.1.10 SI2DR_MAX_STRING_LEN

```
#define SI2DR_MAX_STRING_LEN 1048576 /*1024*1024*/
```

Definition at line 93 of file [si2dr_liberty.h](#).

6.13.1.11 SI2DR_MIN_FLOAT32

```
#define SI2DR_MIN_FLOAT32 -1E+37
```

Definition at line 86 of file [si2dr_liberty.h](#).

6.13.1.12 SI2DR_MIN_FLOAT64

```
#define SI2DR_MIN_FLOAT64 -1.797693e+308
```

Definition at line 88 of file [si2dr_liberty.h](#).

6.13.1.13 SI2DR_MIN_INT32

```
#define SI2DR_MIN_INT32 -2147483647
```

Definition at line 90 of file [si2dr_liberty.h](#).

6.13.1.14 SI2DR_TRUE

```
#define SI2DR_TRUE SI2_TRUE
```

Definition at line 95 of file [si2dr_liberty.h](#).

6.13.2 Typedef Documentation

6.13.2.1 si2BooleanT

```
typedef enum si2BooleanEnum si2BooleanT
```

6.13.2.2 si2DoubleT

```
typedef double si2DoubleT
```

Definition at line 52 of file [si2dr_liberty.h](#).

6.13.2.3 si2drAttrComplexValIdT

```
typedef si2drObjectIdT si2drAttrComplexValIdT
```

Definition at line 111 of file [si2dr_liberty.h](#).

6.13.2.4 si2drAttrIdT

```
typedef si2drObjectIdT si2drAttrIdT
```

Definition at line 110 of file [si2dr_liberty.h](#).

6.13.2.5 si2drAttrIdT

```
typedef si2drIterIdT si2drAttrIdT
```

Definition at line 116 of file [si2dr_liberty.h](#).

6.13.2.6 si2drAttrTypeT

```
typedef enum si2drAttrTypeT si2drAttrTypeT
```

6.13.2.7 si2drBooleanT

```
typedef si2BooleanT si2drBooleanT
```

Definition at line 98 of file [si2dr_liberty.h](#).

6.13.2.8 si2drDefineIdT

```
typedef si2drObjectIdT si2drDefineIdT
```

Definition at line 112 of file [si2dr_liberty.h](#).

6.13.2.9 si2drDefinesIdT

```
typedef si2drIterIdT si2drDefinesIdT
```

Definition at line 117 of file [si2dr_liberty.h](#).

6.13.2.10 si2drErrorT

```
typedef enum si2drErrorT si2drErrorT
```

6.13.2.11 si2drExprT

```
typedef struct si2drExprT si2drExprT
```

6.13.2.12 si2drExprTypeT

```
typedef enum si2drExprTypeT si2drExprTypeT
```

6.13.2.13 si2drFloat32T

```
typedef si2FloatT si2drFloat32T
```

Definition at line 101 of file [si2dr_liberty.h](#).

6.13.2.14 si2drFloat64T

```
typedef si2DoubleT si2drFloat64T
```

Definition at line 102 of file [si2dr_liberty.h](#).

6.13.2.15 si2drGroupIdT

```
typedef si2drObjectIdT si2drGroupIdT
```

Definition at line 109 of file [si2dr_liberty.h](#).

6.13.2.16 si2drGroupsIdT

```
typedef si2drIterIdT si2drGroupsIdT
```

Definition at line 115 of file [si2dr_liberty.h](#).

6.13.2.17 si2drInt32T

```
typedef si2LongT si2drInt32T
```

Definition at line 100 of file [si2dr_liberty.h](#).

6.13.2.18 si2drIterIdT

```
typedef si2IteratorIdT si2drIterIdT
```

Definition at line 106 of file [si2dr_liberty.h](#).

6.13.2.19 si2drMessageHandlerT

```
typedef si2drVoidT(* si2drMessageHandlerT) (si2drSeverityT sev, si2drErrorT errToPrint, si2drStringT  
auxText, si2drErrorT *err)
```

Definition at line 215 of file [si2dr_liberty.h](#).

6.13.2.20 si2drNamesIdT

```
typedef si2drIterIdT si2drNamesIdT
```

Definition at line 118 of file [si2dr_liberty.h](#).

6.13.2.21 si2drObjectIdT

```
typedef si2ObjectIdT si2drObjectIdT
```

Definition at line 107 of file [si2dr_liberty.h](#).

6.13.2.22 si2drObjectsIdT

```
typedef si2drIterIdT si2drObjectsIdT
```

Definition at line 114 of file [si2dr_liberty.h](#).

6.13.2.23 si2drObjectTypeT

```
typedef enum si2drObjectTypeT si2drObjectTypeT
```

6.13.2.24 si2drSeverityT

```
typedef enum si2drSeverityT si2drSeverityT
```

6.13.2.25 si2drStringT

```
typedef si2StringT si2drStringT
```

Definition at line 103 of file [si2dr_liberty.h](#).

6.13.2.26 si2drValuesIdT

```
typedef si2drIterIdT si2drValuesIdT
```

Definition at line 119 of file [si2dr_liberty.h](#).

6.13.2.27 si2drValueTypeT

```
typedef enum si2drValueTypeT si2drValueTypeT
```

6.13.2.28 si2drVoidT

```
typedef si2VoidT si2drVoidT
```

Definition at line 104 of file [si2dr_liberty.h](#).

6.13.2.29 si2FloatT

```
typedef float si2FloatT
```

Definition at line 51 of file [si2dr_liberty.h](#).

6.13.2.30 si2IteratorIdT

```
typedef void* si2IteratorIdT
```

Definition at line 83 of file [si2dr_liberty.h](#).

6.13.2.31 si2LongT

```
typedef long int si2LongT
```

Definition at line 50 of file [si2dr_liberty.h](#).

6.13.2.32 si2ObjectIdT

```
typedef struct si2OID si2ObjectIdT
```

6.13.2.33 si2StringT

```
typedef char* si2StringT
```

Definition at line 53 of file [si2dr_liberty.h](#).

6.13.2.34 si2TypeT

```
typedef enum si2TypeEnum si2TypeT
```

6.13.2.35 si2VoidT

```
typedef void si2VoidT
```

Definition at line 54 of file [si2dr_liberty.h](#).

6.13.3 Enumeration Type Documentation

6.13.3.1 si2BooleanEnum

```
enum si2BooleanEnum
```

Enumerator

SI2_FALSE	
SI2_TRUE	

Definition at line 49 of file [si2dr_liberty.h](#).

6.13.3.2 si2drAttrTypeT

enum [si2drAttrTypeT](#)

Enumerator

SI2DR_SIMPLE	
SI2DR_COMPLEX	

Definition at line 139 of file [si2dr_liberty.h](#).

6.13.3.3 si2drErrorT

enum [si2drErrorT](#)

Enumerator

SI2DR_NO_ERROR	
SI2DR_INTERNAL_SYSTEM_ERROR	
SI2DR_INVALID_VALUE	
SI2DR_INVALID_NAME	
SI2DR_INVALID_OBJECTTYPE	
SI2DR_INVALID_ATTRTYPE	
SI2DR_UNUSABLE_OID	
SI2DR_OBJECT_ALREADY_EXISTS	
SI2DR_OBJECT_NOT_FOUND	
SI2DR_SYNTAX_ERROR	
SI2DR_TRACE_FILES_CANNOT_BE_OPENED	
SI2DR_PIINIT_NOT_CALLED	
SI2DR_SEMANTIC_ERROR	
SI2DR_REFERENCE_ERROR	
SI2DR_MAX_ERROR	

Definition at line 190 of file [si2dr_liberty.h](#).

6.13.3.4 si2drExprTypeT

enum [si2drExprTypeT](#)

Enumerator

SI2DR_EXPR_VAL	
SI2DR_EXPR_OP_ADD	
SI2DR_EXPR_OP_SUB	
SI2DR_EXPR_OP_MUL	
SI2DR_EXPR_OP_DIV	
SI2DR_EXPR_OP_PAREN	
SI2DR_EXPR_OP_LOG2	
SI2DR_EXPR_OP_LOG10	
SI2DR_EXPR_OP_EXP	

Definition at line [157](#) of file [si2dr_liberty.h](#).

6.13.3.5 si2drObjectTypeT

enum [si2drObjectTypeT](#)

Enumerator

SI2DR_UNDEFINED_OBJECTTYPE	
SI2DR_GROUP	
SI2DR_ATTR	
SI2DR_DEFINE	
SI2DR_COMPLEXVAL	
SI2DR_MAX_OBJECTTYPE	

Definition at line [129](#) of file [si2dr_liberty.h](#).

6.13.3.6 si2drSeverityT

enum [si2drSeverityT](#)

Enumerator

SI2DR_SEVERITY_NOTE	
SI2DR_SEVERITY_WARN	
SI2DR_SEVERITY_ERR	

Definition at line [121](#) of file [si2dr_liberty.h](#).

6.13.3.7 si2drValueTypeT

enum [si2drValueTypeT](#)

Enumerator

SI2DR_UNDEFINED_VALUETYPE	
SI2DR_INT32	
SI2DR_STRING	
SI2DR_FLOAT64	
SI2DR_BOOLEAN	
SI2DR_EXPR	
SI2DR_MAX_VALUETYPE	

Definition at line [146](#) of file [si2dr_liberty.h](#).

6.13.3.8 si2TypeEnum

enum [si2TypeEnum](#)

Enumerator

SI2_UNDEFINED_VALUETYPE	
SI2_BOOLEAN	
SI2_LONG	
SI2_FLOAT	
SI2_DOUBLE	
SI2_STRING	
SI2_VOID	
SI2_OID	
SI2_ITER	
SI2_EXPR	
SI2_MAX_VALUETYPE	

Definition at line [56](#) of file [si2dr_liberty.h](#).

6.13.4 Function Documentation

6.13.4.1 get_function()

```
void get_function (
    char * filename )
```

6.13.4.2 liberty_destroy_value_data()

```
void liberty_destroy_value_data (
    struct liberty_value_data * vd )
```

6.13.4.3 liberty_get_element()

```
LONG_DOUBLE liberty_get_element (
    struct liberty_value_data * vd,
    ... )
```

6.13.4.4 liberty_get_values_data()

```
struct liberty_value_data * liberty_get_values_data (
    si2drGroupIdT table_group )
```

6.13.4.5 main_liberty_parser()

```
int main_liberty_parser (
    int argc,
    char * argv[] )
```

6.13.4.6 print_version()

```
void print_version ( )
```

6.13.4.7 SI2_ARGS() [1/41]

```
si2drVoidT si2drReadLibertyFile SI2_ARGS (
    (char *filename, si2drErrorT *err) )
```

6.13.4.8 SI2_ARGS() [2/41]

```
si2drVoidT si2drWriteLibertyFile SI2_ARGS (
    (char *filename, si2drGroupIdT group, char *cellname, si2drErrorT *err) )
```

6.13.4.9 SI2_ARGS() [3/41]

```
si2drVoidT si2drComplexAttrAddBooleanValue SI2_ARGS (  
    (si2drAttrIdT attr, si2drBooleanT boolval, si2drErrorT *err) )
```

6.13.4.10 SI2_ARGS() [4/41]

```
si2drVoidT si2drSimpleAttrSetBooleanValue SI2_ARGS (  
    (si2drAttrIdT attr, si2drBooleanT intgr, si2drErrorT *err) )
```

6.13.4.11 SI2_ARGS() [5/41]

```
si2drStringT si2drAttrGetComment SI2_ARGS (  
    (si2drAttrIdT attr, si2drErrorT *err) )
```

6.13.4.12 SI2_ARGS() [6/41]

```
si2drVoidT si2drSimpleAttrSetExprValue SI2_ARGS (  
    (si2drAttrIdT attr, si2drExprT *expr, si2drErrorT *err) )
```

6.13.4.13 SI2_ARGS() [7/41]

```
si2drVoidT si2drSimpleAttrSetFloat64Value SI2_ARGS (  
    (si2drAttrIdT attr, si2drFloat64T float64, si2drErrorT *err) )
```

6.13.4.14 SI2_ARGS() [8/41]

```
si2drVoidT si2drSimpleAttrSetInt32Value SI2_ARGS (  
    (si2drAttrIdT attr, si2drInt32T intgr, si2drErrorT *err) )
```

6.13.4.15 SI2_ARGS() [9/41]

```
si2drVoidT si2drAttrSetComment SI2_ARGS (  
    (si2drAttrIdT attr, si2drStringT comment, si2drErrorT *err) )
```

6.13.4.16 SI2_ARGS() [10/41]

```
si2drVoidT si2drSimpleAttrSetStringValue SI2_ARGS (  
    (si2drAttrIdT attr, si2drStringT string, si2drErrorT *err) )
```

6.13.4.17 SI2_ARGS() [11/41]

```
si2drExprT *si2drCreateBooleanValExpr SI2_ARGS (  
    (si2drBooleanT b, si2drErrorT *err) )
```

6.13.4.18 SI2_ARGS() [12/41]

```
si2drStringT si2drDefineGetComment SI2_ARGS (  
    (si2drDefineIdT def, si2drErrorT *err) )
```

6.13.4.19 SI2_ARGS() [13/41]

```
si2drVoidT si2drDefineGetInfo SI2_ARGS (  
    (si2drDefineIdT def, si2drStringT *name, si2drStringT *allowed_group_name, si2drValueType  
*valtype, si2drErrorT *err) )
```

6.13.4.20 SI2_ARGS() [14/41]

```
si2drVoidT si2drDefineSetComment SI2_ARGS (  
    (si2drDefineIdT def, si2drStringT comment, si2drErrorT *err) )
```

6.13.4.21 SI2_ARGS() [15/41]

```
si2drBooleanT si2drPIGetNocheckMode SI2_ARGS (  
    (si2drErrorT *err) )
```

6.13.4.22 SI2_ARGS() [16/41]

```
si2drStringT si2drPIGetErrorText SI2_ARGS (  
    (si2drErrorT errorCode, si2drErrorT *err) )
```

6.13.4.23 SI2_ARGS() [17/41]

```
si2drExprT *si2drOpExprGetRightExpr SI2_ARGS (  
    (si2drExprT *expr, si2drErrorT *err) )
```

6.13.4.24 SI2_ARGS() [18/41]

```
si2drExprT *si2drCreateBinaryOpExpr SI2_ARGS (  
    (si2drExprT *left, si2drExprTypeT optype, si2drExprT *right, si2drErrorT *err) )
```

6.13.4.25 SI2_ARGS() [19/41]

```
si2drExprT *si2drCreateUnaryOpExpr SI2_ARGS (  
    (si2drExprTypeT optype, si2drExprT *expr, si2drErrorT *err) )
```

6.13.4.26 SI2_ARGS() [20/41]

```
si2drExprT *si2drCreateExpr SI2_ARGS (  
    (si2drExprTypeT type, si2drErrorT *err) )
```

6.13.4.27 SI2_ARGS() [21/41]

```
si2drExprT *si2drCreateDoubleValExpr SI2_ARGS (  
    (si2drFloat64T d, si2drErrorT *err) )
```

6.13.4.28 SI2_ARGS() [22/41]

```
si2drVoidT si2drCheckLibertyLibrary SI2_ARGS (  
    (si2drGroupIdT group, si2drErrorT *err) )
```

6.13.4.29 SI2_ARGS() [23/41]

```
si2drVoidT si2drGroupSetComment SI2_ARGS (  
    (si2drGroupIdT group, si2drStringT comment, si2drErrorT *err) )
```

6.13.4.30 SI2_ARGS() [24/41]

```
si2drAttrIdT si2drGroupCreateAttr SI2_ARGS (  
    (si2drGroupIdT group, si2drStringT name, si2drAttrTypeT type, si2drErrorT *err) )
```

6.13.4.31 SI2_ARGS() [25/41]

```
si2drDefineIdT si2drGroupFindDefineByName SI2_ARGS (  
    (si2drGroupIdT group, si2drStringT name, si2drErrorT *err) )
```

6.13.4.32 SI2_ARGS() [26/41]

```
si2drDefineIdT si2drGroupCreateDefine SI2_ARGS (  
    (si2drGroupIdT group, si2drStringT name, si2drStringT allowed_group_name, si2drValueTypeT  
    valtype, si2drErrorT *err) )
```

6.13.4.33 SI2_ARGS() [27/41]

```
si2drGroupIdT si2drGroupCreateGroup SI2_ARGS (  
    (si2drGroupIdT group, si2drStringT name, si2drStringT group_type, si2drErrorT  
    *err) )
```

6.13.4.34 SI2_ARGS() [28/41]

```
si2drGroupIdT si2drGroupFindGroupByName SI2_ARGS (  
    (si2drGroupIdT group, si2drStringT name, si2drStringT type, si2drErrorT *err) )
```

6.13.4.35 SI2_ARGS() [29/41]

```
si2drVoidT si2drGroupMoveBefore SI2_ARGS (  
    (si2drGroupIdT groupToMove, si2drGroupIdT targetGroup, si2drErrorT *err) )
```

6.13.4.36 SI2_ARGS() [30/41]

```
si2drExprT *si2drCreateIntValExpr SI2_ARGS (  
    (si2drInt32T i, si2drErrorT *err) )
```

6.13.4.37 SI2_ARGS() [31/41]

```
si2drStringT si2drObjectGetFileName SI2_ARGS (  
    (si2drObjectIdT object, si2drErrorT *err) )
```

6.13.4.38 SI2_ARGS() [32/41]

```
si2drVoidT si2drObjectSetLineNo SI2_ARGS (  
    (si2drObjectIdT object, si2drInt32T lineno, si2drErrorT *err) )
```

6.13.4.39 SI2_ARGS() [33/41]

```
si2drVoidT si2drObjectSetFileName SI2_ARGS (  
    (si2drObjectIdT object, si2drStringT filename, si2drErrorT *err) )
```

6.13.4.40 SI2_ARGS() [34/41]

```
si2drBooleanT si2drObjectIsSame SI2_ARGS (  
    (si2drObjectIdT object1, si2drObjectIdT object2, si2drErrorT *err) )
```

6.13.4.41 SI2_ARGS() [35/41]

```
si2drVoidT si2drPISetTraceMode SI2_ARGS (  
    (si2drStringT fname, si2drErrorT *err) )
```

6.13.4.42 SI2_ARGS() [36/41]

```
si2drDefineIdT si2drPIFindDefineByName SI2_ARGS (  
    (si2drStringT name, si2drErrorT *err) )
```

6.13.4.43 SI2_ARGS() [37/41]

```
si2drGroupIdT si2drPICreateGroup SI2_ARGS (  
    (si2drStringT name, si2drStringT group_type, si2drErrorT *err) )
```

6.13.4.44 SI2_ARGS() [38/41]

```
si2drGroupIdT si2drPIFindGroupByName SI2_ARGS (  
    (si2drStringT name, si2drStringT type, si2drErrorT *err) )
```

6.13.4.45 SI2_ARGS() [39/41]

```
si2drExprT *si2drCreateStringValExpr SI2_ARGS (  
    (si2drStringT s, si2drErrorT *err) )
```

6.13.4.46 SI2_ARGS() [40/41]

```
si2drVoidT si2drIterQuit SI2_ARGS (  
    (si2drValuesIdT iter, si2drErrorT *err) )
```

6.13.4.47 SI2_ARGS() [41/41]

```
si2drVoidT si2drIterNextComplexValue SI2_ARGS (  
    (si2drValuesIdT iter, si2drValueTypeT *type, si2drInt32T *intgr, si2drFloat64T  
*float64, si2drStringT *string, si2drBooleanT *boolval, si2drExprT **expr, si2drErrorT *err) )
```

6.13.4.48 si2drDefaultMessageHandler()

```
si2drVoidT si2drDefaultMessageHandler (  
    si2drSeverityT sev,  
    si2drErrorT errToPrint,  
    si2drStringT auxText,  
    si2drErrorT * err )
```

6.13.4.49 si2drPIGetMessageHandler()

```
si2drMessageHandlerT si2drPIGetMessageHandler (  
    si2drErrorT * err )
```

6.13.4.50 si2drPISetMessageHandler()

```
si2drVoidT si2drPISetMessageHandler (
    si2drMessageHandlerT MsgHandFuncPtr,
    si2drErrorT * err )
```

6.13.4.51 usage()

```
void usage ( )
```

6.14 si2dr_liberty.h

[Go to the documentation of this file.](#)

```
00001 /*****
00002 Copyright (c) 1996-2005 Synopsys, Inc.          ALL RIGHTS RESERVED
00003
00004 The contents of this file are subject to the restrictions and limitations
00005 set forth in the SYNOPSIS Open Source License Version 1.0 (the "License");
00006 you may not use this file except in compliance with such restrictions
00007 and limitations. You may obtain instructions on how to receive a copy of
00008 the License at
00009
00010 http://www.synopsys.com/partners/tapin/tapinprogram.html.
00011
00012 Software distributed by Original Contributor under the License is
00013 distributed on an "AS IS" basis, WITHOUT WARRANTY OF ANY KIND, either
00014 expressed or implied. See the License for the specific language governing
00015 rights and limitations under the License.
00016
00017 *****/
00018
00019 /* define a PI for manipulating the liberty data */
00020
00021
00022 /* basic data types */
00023
00024
00025 #ifndef SI2DR_H
00026 #define SI2DR_H
00027
00028 #ifdef SI2_ARGS
00029 #undef SI2_ARGS
00030 #endif
00031 #if defined(__STDC__) || defined(HPPA) || defined(ibmpoweraix) || defined(_MSC_VER) ||
    defined(__cplusplus)
00032 #define SI2_ARGS(args) args
00033 #else
00034 #define SI2_ARGS(args) ()
00035 #endif
00036
00037
00038 /* *****
00039 File contents : LIBERTY-DR type definitions.
00040 ***** */
00041 #ifdef __cplusplus
00042 extern "C" {
00043 #endif
00044
00045 /*
00046 * These primitive types are compatible with the ANSI C Standard (ANSI/
00047 * X3.159-1990 Programming Language C Standard.)
00048 */
00049 typedef enum si2BooleanEnum { SI2_FALSE = 0, SI2_TRUE = 1 } si2BooleanT;
00050 typedef long int si2LongT;
00051 typedef float si2FloatT;
00052 typedef double si2DoubleT;
00053 typedef char* si2StringT;
00054 typedef void si2VoidT;
00055
00056 typedef enum si2TypeEnum {
00057     SI2_UNDEFINED_VALUETYPE = 0,
```

```

00058     SI2_BOOLEAN = 1,
00059     SI2_LONG = 2,
00060     SI2_FLOAT = 3,
00061     SI2_DOUBLE = 4,
00062     SI2_STRING = 5,
00063     SI2_VOID = 6,
00064     SI2_OID = 7,
00065     SI2_ITER = 8,
00066     SI2_EXPR = 9,
00067     SI2_MAX_VALUETYPE = 10
00068 } si2TypeT;
00069
00070 /*
00071  * Size limits
00072  */
00073
00074 #define SI2_LONG_MAX    2147483647
00075 #define SI2_LONG_MIN    -2147483647
00076 #define SI2_ULONG_MAX   4294967295
00077
00078
00079 typedef struct si2OID {void *v1, *v2;} si2ObjectIdT;
00080 /*
00081  * Iterators are blind handles the size of a single pointer.
00082  */
00083 typedef void *si2IteratorIdT;
00084
00085
00086 #define SI2DR_MIN_FLOAT32    -1E+37
00087 #define SI2DR_MAX_FLOAT32    1E+37
00088 #define SI2DR_MIN_FLOAT64    -1.797693e+308
00089 #define SI2DR_MAX_FLOAT64    1.797693e+308
00090 #define SI2DR_MIN_INT32      -2147483647
00091 #define SI2DR_MAX_INT32      2147483647
00092
00093 #define SI2DR_MAX_STRING_LEN  1048576 /*1024*1024*/
00094
00095 #define SI2DR_TRUE SI2_TRUE
00096 #define SI2DR_FALSE SI2_FALSE
00097
00098 typedef si2BooleanT si2drBooleanT;
00099
00100 typedef si2LongT    si2drInt32T;
00101 typedef si2FloatT   si2drFloat32T;
00102 typedef si2DoubleT   si2drFloat64T;
00103 typedef si2StringT   si2drStringT;
00104 typedef si2VoidT     si2drVoidT;
00105
00106 typedef si2IteratorIdT    si2drIterIdT;
00107 typedef si2ObjectIdT      si2drObjectIdT;
00108
00109 typedef si2drObjectIdT si2drGroupIdT;
00110 typedef si2drObjectIdT si2drAttrIdT;
00111 typedef si2drObjectIdT si2drAttrComplexValIdT;
00112 typedef si2drObjectIdT si2drDefineIdT;
00113
00114 typedef si2drIterIdT si2drObjectsIdT;
00115 typedef si2drIterIdT si2drGroupsIdT;
00116 typedef si2drIterIdT si2drAttrsIdT;
00117 typedef si2drIterIdT si2drDefinesIdT;
00118 typedef si2drIterIdT si2drNamesIdT;
00119 typedef si2drIterIdT si2drValuesIdT;
00120
00121 typedef enum si2drSeverityT
00122 {
00123     SI2DR_SEVERITY_NOTE = 0,
00124     SI2DR_SEVERITY_WARN = 1,
00125     SI2DR_SEVERITY_ERR  = 2
00126 } si2drSeverityT;
00127
00128
00129 typedef enum si2drObjectTypeT
00130 {
00131     SI2DR_UNDEFINED_OBJECTTYPE = 0,
00132     SI2DR_GROUP                 = 1,
00133     SI2DR_ATTR                  = 2,
00134     SI2DR_DEFINE                = 3,
00135     SI2DR_COMPLEXVAL            = 4,
00136     SI2DR_MAX_OBJECTTYPE       = 5
00137 } si2drObjectTypeT;
00138
00139 typedef enum si2drAttrTypeT
00140 {
00141     SI2DR_SIMPLE,
00142     SI2DR_COMPLEX
00143 } si2drAttrTypeT;
00144

```

```

00145
00146 typedef enum si2drValueType
00147 {
00148     SI2DR_UNDEFINED_VALUETYPE = SI2_UNDEFINED_VALUETYPE,
00149     SI2DR_INT32                = SI2_LONG,
00150     SI2DR_STRING               = SI2_STRING,
00151     SI2DR_FLOAT64              = SI2_DOUBLE,
00152     SI2DR_BOOLEAN              = SI2_BOOLEAN,
00153     SI2DR_EXPR                 = SI2_EXPR,
00154     SI2DR_MAX_VALUETYPE       = SI2_MAX_VALUETYPE
00155 } si2drValueType;
00156
00157 typedef enum si2drExprTypeT
00158 {
00159     SI2DR_EXPR_VAL = 0, /* unary ops just have left arg set: add, sub, logs, paren */
00160     SI2DR_EXPR_OP_ADD = 1,
00161     SI2DR_EXPR_OP_SUB = 2,
00162     SI2DR_EXPR_OP_MUL = 3,
00163     SI2DR_EXPR_OP_DIV = 4,
00164     SI2DR_EXPR_OP_PAREN = 5,
00165     SI2DR_EXPR_OP_LOG2 = 6, /* pretty much flight of fancy from here on? */
00166     SI2DR_EXPR_OP_LOG10 = 7,
00167     SI2DR_EXPR_OP_EXP = 8, /* exponent? */
00168 } si2drExprTypeT;
00169
00170
00171
00172 typedef struct si2drExprT
00173 {
00174     si2drExprTypeT type;
00175     union
00176     {
00177         si2drInt32T i;
00178         si2drFloat64T d;
00179         si2drStringT s; /* most likely an identifier */
00180         si2drBooleanT b;
00181     } u;
00182     si2drValueType valuetype; /* if the type is a fixed value */
00183     struct si2drExprT *left; /* the exprs form a classic binary tree rep of an arithmetic expression
00184     */
00185     struct si2drExprT *right;
00186 } si2drExprT;
00187
00188
00189
00190 typedef enum si2drErrorT
00191 {
00192     SI2DR_NO_ERROR = 0,
00193     SI2DR_INTERNAL_SYSTEM_ERROR = 1,
00194     SI2DR_INVALID_VALUE = 4,
00195     SI2DR_INVALID_NAME = 5,
00196     SI2DR_INVALID_OBJECTTYPE = 6,
00197     SI2DR_INVALID_ATTRTYPE = 10,
00198     SI2DR_UNUSABLE_OID = 7,
00199     SI2DR_OBJECT_ALREADY_EXISTS = 8,
00200     SI2DR_OBJECT_NOT_FOUND = 9,
00201     SI2DR_SYNTAX_ERROR = 2,
00202     SI2DR_TRACE_FILES_CANNOT_BE_OPENED = 3,
00203     SI2DR_PIINIT_NOT_CALLED = 11,
00204     SI2DR_SEMANTIC_ERROR = 12,
00205     SI2DR_REFERENCE_ERROR = 13,
00206     SI2DR_MAX_ERROR = 14
00207 } si2drErrorT;
00208
00209 si2drVoidT si2drDefaultMessageHandler(si2drSeverityT sev,
00210                                       si2drErrorT errToPrint,
00211                                       si2drStringT auxText,
00212                                       si2drErrorT *err);
00213
00214
00215     typedef si2drVoidT (*si2drMessageHandlerT)(si2drSeverityT sev,
00216                                                 si2drErrorT errToPrint,
00217                                                 si2drStringT auxText,
00218                                                 si2drErrorT *err);
00219
00220
00221 si2drMessageHandlerT si2drPIMessageHandler( si2drErrorT *err);
00222
00223
00224 si2drVoidT si2drPISetMessageHandler( si2drMessageHandlerT MsgHandFuncPtr,
00225                                       si2drErrorT *err);
00226
00227
00228     si2drGroupIdT si2drPICreateGroup SI2_ARGS(( si2drStringT name,
00229                                                 si2drStringT group_type,
00230                                                 si2drErrorT *err));

```

```

00231
00232
00233     si2drAttrIdT    si2drGroupCreateAttr    SI2_ARGS(( si2drGroupIdT    group,
00234                                                     si2drStringT        name,
00235                                                     si2drAttrTypeT     type,
00236                                                     si2drErrorT        *err));
00237
00238     si2drAttrTypeT si2drAttrGetAttrType    SI2_ARGS(( si2drAttrIdT    attr,
00239                                                     si2drErrorT        *err));
00240
00241     si2drStringT   si2drAttrGetName        SI2_ARGS(( si2drAttrIdT    attr,
00242                                                     si2drErrorT        *err));
00243
00244     si2drVoidT     si2drComplexAttrAddInt32Value    SI2_ARGS(( si2drAttrIdT    attr,
00245                                                     si2drInt32T        intgr,
00246                                                     si2drErrorT        *err ));
00247
00248     si2drVoidT     si2drComplexAttrAddStringValue    SI2_ARGS(( si2drAttrIdT    attr,
00249                                                     si2drStringT        string,
00250                                                     si2drErrorT        *err ));
00251
00252     si2drVoidT     si2drComplexAttrAddBooleanValue    SI2_ARGS(( si2drAttrIdT    attr,
00253                                                     si2drBooleanT        boolval,
00254                                                     si2drErrorT        *err ));
00255
00256     si2drVoidT     si2drComplexAttrAddFloat64Value    SI2_ARGS(( si2drAttrIdT    attr,
00257                                                     si2drFloat64T        float64,
00258                                                     si2drErrorT        *err ));
00259
00260     si2drVoidT     si2drComplexAttrAddExprValue    SI2_ARGS(( si2drAttrIdT    attr,
00261                                                     si2drExprT          *expr,
00262                                                     si2drErrorT        *err ));
00263
00264     si2drValuesIdT si2drComplexAttrGetValues    SI2_ARGS(( si2drAttrIdT    attr,
00265                                                     si2drErrorT        *err ));
00266
00267     si2drVoidT     si2drIterNextComplexValue    SI2_ARGS(( si2drValuesIdT    iter,
00268                                                     si2drValueTypeT    *type,
00269                                                     si2drInt32T        *intgr,
00270                                                     si2drFloat64T        *float64,
00271                                                     si2drStringT        *string,
00272                                                     si2drBooleanT        *boolval,
00273                                                     si2drExprT          **expr,
00274                                                     si2drErrorT        *err ));
00275
00276     si2drAttrComplexValIdT si2drIterNextComplex    SI2_ARGS(( si2drValuesIdT    iter,
00277                                                     si2drErrorT        *err ));
00278
00279     si2drValueTypeT si2drComplexValGetValueType    SI2_ARGS(( si2drAttrComplexValIdT    attr,
00280                                                     si2drErrorT        *err ));
00281
00282     si2drInt32T     si2drComplexValGetInt32Value    SI2_ARGS(( si2drAttrComplexValIdT    attr,
00283                                                     si2drErrorT        *err ));
00284
00285     si2drFloat64T   si2drComplexValGetFloat64Value    SI2_ARGS(( si2drAttrComplexValIdT    attr,
00286                                                     si2drErrorT        *err ));
00287
00288     si2drStringT     si2drComplexValGetStringValue    SI2_ARGS(( si2drAttrComplexValIdT    attr,
00289                                                     si2drErrorT        *err ));
00290
00291     si2drBooleanT    si2drComplexValGetBooleanValue    SI2_ARGS(( si2drAttrComplexValIdT    attr,
00292                                                     si2drErrorT        *err ));
00293
00294     si2drExprT       *si2drComplexValGetExprValue    SI2_ARGS(( si2drAttrComplexValIdT    attr,
00295                                                     si2drErrorT        *err ));
00296
00297
00298
00299
00300
00301
00302     si2drValueTypeT si2drSimpleAttrGetValueType    SI2_ARGS(( si2drAttrIdT    attr,
00303                                                     si2drErrorT        *err ));
00304
00305     si2drInt32T     si2drSimpleAttrGetInt32Value    SI2_ARGS(( si2drAttrIdT    attr,
00306                                                     si2drErrorT        *err ));
00307
00308     si2drFloat64T   si2drSimpleAttrGetFloat64Value    SI2_ARGS(( si2drAttrIdT    attr,
00309                                                     si2drErrorT        *err ));
00310
00311     si2drStringT     si2drSimpleAttrGetStringValue    SI2_ARGS(( si2drAttrIdT    attr,
00312                                                     si2drErrorT        *err ));
00313
00314     si2drBooleanT    si2drSimpleAttrGetBooleanValue    SI2_ARGS(( si2drAttrIdT    attr,
00315                                                     si2drErrorT        *err ));
00316
00317     si2drExprT       *si2drSimpleAttrGetExprValue    SI2_ARGS(( si2drAttrIdT    attr,

```

```

00318                                     si2drErrorT      *err ));
00319
00320
00321
00322     si2drVoidT      si2drSimpleAttrSetInt32Value      SI2_ARGS(( si2drAttrIdT attr,
00323                                     si2drInt32T      intgr,
00324                                     si2drErrorT      *err ));
00325
00326     si2drVoidT      si2drSimpleAttrSetBooleanValue    SI2_ARGS(( si2drAttrIdT attr,
00327                                     si2drBooleanT    intgr,
00328                                     si2drErrorT      *err ));
00329
00330     si2drVoidT      si2drSimpleAttrSetFloat64Value    SI2_ARGS(( si2drAttrIdT attr,
00331                                     si2drFloat64T    float64,
00332                                     si2drErrorT      *err ));
00333
00334     si2drVoidT      si2drSimpleAttrSetStringValue     SI2_ARGS(( si2drAttrIdT attr,
00335                                     si2drStringT      string,
00336                                     si2drErrorT      *err ));
00337
00338     si2drVoidT      si2drSimpleAttrSetExprValue       SI2_ARGS(( si2drAttrIdT attr,
00339                                     si2drExprT        *expr,
00340                                     si2drErrorT      *err ));
00341
00342     si2drBooleanT    si2drSimpleAttrGetIsVar          SI2_ARGS(( si2drAttrIdT attr,
00343                                     si2drErrorT      *err ));
00344
00345     si2drVoidT      si2drSimpleAttrSetIsVar          SI2_ARGS(( si2drAttrIdT attr,
00346                                     si2drErrorT      *err ));
00347
00348 /* helper functions for expr creation, building, and destruction */
00349
00350     si2drVoidT      si2drExprDestroy                  SI2_ARGS(( si2drExprT *expr, /* recursively
free the structures */
00351                                     si2drErrorT *err));
00352
00353     si2drExprT      *si2drCreateExpr                  SI2_ARGS(( si2drExprTypeT type, /* malloc an
Expr and return it */
00354                                     si2drErrorT *err));
00355
00356     si2drExprT      *si2drCreateBooleanValExpr        SI2_ARGS(( si2drBooleanT b,
00357                                     si2drErrorT *err ));
00358
00359     si2drExprT      *si2drCreateDoubleValExpr         SI2_ARGS(( si2drFloat64T d,
00360                                     si2drErrorT *err ));
00361
00362     si2drExprT      *si2drCreateStringValExpr         SI2_ARGS(( si2drStringT s,
00363                                     si2drErrorT *err ));
00364
00365     si2drExprT      *si2drCreateIntValExpr            SI2_ARGS(( si2drInt32T i,
00366                                     si2drErrorT *err ));
00367
00368     si2drExprT      *si2drCreateBinaryOpExpr         SI2_ARGS(( si2drExprT *left,
00369                                     si2drExprTypeT optype,
00370                                     si2drExprT *right,
00371                                     si2drErrorT *err ));
00372
00373     si2drExprT      *si2drCreateUnaryOpExpr          SI2_ARGS(( si2drExprTypeT optype,
00374                                     si2drExprT *expr,
00375                                     si2drErrorT *err ));
00376
00377     si2drStringT     si2drExprToString                SI2_ARGS(( si2drExprT *expr,
00378                                     si2drErrorT *err ));
00379
00380     si2drExprTypeT   si2drExprGetType                 SI2_ARGS(( si2drExprT *expr,
00381                                     si2drErrorT *err ));
00382
00383     si2drValueTypeT  si2drValExprGetValueType        SI2_ARGS(( si2drExprT *expr,
00384                                     si2drErrorT *err ));
00385
00386     si2drInt32T      si2drIntValExprGetInt           SI2_ARGS(( si2drExprT *expr,
00387                                     si2drErrorT *err ));
00388
00389     si2drFloat64T    si2drDoubleValExprGetDouble     SI2_ARGS(( si2drExprT *expr,
00390                                     si2drErrorT *err ));
00391
00392     si2drBooleanT     si2drBooleanValExprGetBoolean  SI2_ARGS(( si2drExprT *expr,
00393                                     si2drErrorT *err ));
00394
00395     si2drStringT     si2drStringValExprGetString     SI2_ARGS(( si2drExprT *expr,
00396                                     si2drErrorT *err ));
00397
00398     si2drExprT      *si2drOpExprGetLeftExpr          SI2_ARGS(( si2drExprT *expr, /* will apply to Unary & binary
Ops */
00399                                     si2drErrorT *err ));
00400
00401     si2drExprT      *si2drOpExprGetRightExpr         SI2_ARGS(( si2drExprT *expr,

```

```

00402                                     si2drErrorT *err ));
00403
00404 /* Define related funcs */
00405
00406 si2drDefineIdT si2drGroupCreateDefine SI2_ARGS(( si2drGroupIdT group,
00407                                                  si2drStringT name,
00408                                                  si2drStringT allowed_group_name,
00409                                                  si2drValueTypeT valtype,
00410                                                  si2drErrorT *err));
00411
00412 si2drVoidT si2drDefineGetInfo SI2_ARGS(( si2drDefineIdT def,
00413                                           si2drStringT *name,
00414                                           si2drStringT *allowed_group_name,
00415                                           si2drValueTypeT *valtype,
00416                                           si2drErrorT *err));
00417
00418 si2drStringT si2drDefineGetName SI2_ARGS(( si2drDefineIdT def,
00419                                             si2drErrorT *err));
00420
00421 si2drStringT si2drDefineGetAllowedGroupName SI2_ARGS(( si2drDefineIdT def,
00422                                                         si2drErrorT *err));
00423
00424 si2drValueTypeT si2drDefineGetValueType SI2_ARGS(( si2drDefineIdT def,
00425                                                       si2drErrorT *err));
00426
00427 si2drGroupIdT si2drGroupCreateGroup SI2_ARGS(( si2drGroupIdT group,
00428                                                  si2drStringT name,
00429                                                  si2drStringT group_type,
00430                                                  si2drErrorT *err));
00431
00432 si2drStringT si2drGroupGetGroupType SI2_ARGS(( si2drGroupIdT group,
00433                                                  si2drErrorT *err));
00434
00435 si2drStringT si2drGroupGetComment SI2_ARGS(( si2drGroupIdT group,
00436                                                si2drErrorT *err));
00437
00438 si2drVoidT si2drGroupSetComment SI2_ARGS(( si2drGroupIdT group,
00439                                             si2drStringT comment,
00440                                             si2drErrorT *err));
00441
00442 si2drStringT si2drAttrGetComment SI2_ARGS(( si2drAttrIdT attr,
00443                                               si2drErrorT *err));
00444
00445 si2drVoidT si2drAttrSetComment SI2_ARGS(( si2drAttrIdT attr,
00446                                             si2drStringT comment,
00447                                             si2drErrorT *err));
00448
00449 si2drStringT si2drDefineGetComment SI2_ARGS(( si2drDefineIdT def,
00450                                                 si2drErrorT *err));
00451
00452 si2drVoidT si2drDefineSetComment SI2_ARGS(( si2drDefineIdT def,
00453                                               si2drStringT comment,
00454                                               si2drErrorT *err));
00455
00456 si2drVoidT si2drGroupAddName SI2_ARGS(( si2drGroupIdT group,
00457                                           si2drStringT name,
00458                                           si2drErrorT *err));
00459
00460 si2drVoidT si2drGroupDeleteName SI2_ARGS(( si2drGroupIdT group,
00461                                              si2drStringT name,
00462                                              si2drErrorT *err));
00463
00464
00465 si2drGroupIdT si2drPIFindGroupByName SI2_ARGS(( si2drStringT name,
00466                                                  si2drStringT type,
00467                                                  si2drErrorT *err));
00468
00469 si2drGroupIdT si2drGroupFindGroupByName SI2_ARGS(( si2drGroupIdT group,
00470                                                     si2drStringT name,
00471                                                     si2drStringT type,
00472                                                     si2drErrorT *err));
00473
00474 si2drAttrIdT si2drGroupFindAttrByName SI2_ARGS(( si2drGroupIdT group,
00475                                                    si2drStringT name,
00476                                                    si2drErrorT *err));
00477
00478 si2drDefineIdT si2drGroupFindDefineByName SI2_ARGS(( si2drGroupIdT group,
00479                                                       si2drStringT name,
00480                                                       si2drErrorT *err));
00481
00482
00483 si2drDefineIdT si2drPIFindDefineByName SI2_ARGS(( si2drStringT name,
00484                                                    si2drErrorT *err));
00485
00486
00487
00488 si2drGroupsIdT si2drPIGetGroups SI2_ARGS(( si2drErrorT *err));

```

```

00489
00490
00491     si2drGroupsIdT    si2drGroupGetGroups    SI2_ARGS(( si2drGroupIdT group,
00492             si2drErrorT *err));
00493
00494     si2drNamesIdT     si2drGroupGetNames     SI2_ARGS(( si2drGroupIdT group,
00495             si2drErrorT *err));
00496
00497     si2drAttrsIdT     si2drGroupGetAttrs     SI2_ARGS(( si2drGroupIdT group,
00498             si2drErrorT *err));
00499
00500     si2drDefinesIdT   si2drGroupGetDefines   SI2_ARGS(( si2drGroupIdT group,
00501             si2drErrorT *err));
00502
00503     si2drGroupIdT     si2drIterNextGroup     SI2_ARGS(( si2drGroupsIdT iter,
00504             si2drErrorT *err));
00505     si2drStringT      si2drIterNextName      SI2_ARGS(( si2drNamesIdT iter,
00506             si2drErrorT *err));
00507     si2drAttrIdT      si2drIterNextAttr      SI2_ARGS(( si2drAttrsIdT iter,
00508             si2drErrorT *err));
00509     si2drDefineIdT    si2drIterNextDefine    SI2_ARGS(( si2drDefinesIdT iter,
00510             si2drErrorT *err));
00511     si2drVoidT        si2drIterQuit          SI2_ARGS(( si2drIterIdT iter,
00512             si2drErrorT *err));
00513
00514
00515
00516     si2drVoidT        si2drObjectDelete      SI2_ARGS(( si2drObjectIdT object,
00517             si2drErrorT *err));
00518
00519
00520
00521     si2drStringT      si2drPIGetErrorText     SI2_ARGS(( si2drErrorT errorCode,
00522             si2drErrorT *err));
00523
00524     si2drObjectIdT    si2drPIGetNullId        SI2_ARGS(( si2drErrorT *err));
00525
00526     si2drVoidT        si2drPIInit            SI2_ARGS(( si2drErrorT *err));
00527
00528     si2drVoidT        si2drPIQuit            SI2_ARGS(( si2drErrorT *err));
00529
00530     si2drObjectTypeT  si2drObjectGetObjectType SI2_ARGS(( si2drObjectIdT object,
00531             si2drErrorT *err));
00532
00533     si2drObjectIdT    si2drObjectGetOwner      SI2_ARGS(( si2drObjectIdT object,
00534             si2drErrorT *err));
00535
00536     si2drBooleanT     si2drObjectIsNull        SI2_ARGS(( si2drObjectIdT object,
00537             si2drErrorT *err));
00538
00539     si2drBooleanT     si2drObjectIsSame        SI2_ARGS(( si2drObjectIdT object1,
00540             si2drObjectIdT object2,
00541             si2drErrorT *err));
00542
00543     si2drBooleanT     si2drObjectIsUsable      SI2_ARGS(( si2drObjectIdT object,
00544             si2drErrorT *err));
00545
00546     si2drVoidT        si2drObjectSetFileName  SI2_ARGS(( si2drObjectIdT object,
00547             si2drStringT filename,
00548             si2drErrorT *err));
00549
00550     si2drVoidT        si2drObjectSetLineNo     SI2_ARGS(( si2drObjectIdT object,
00551             si2drInt32T lineno,
00552             si2drErrorT *err));
00553
00554     si2drInt32T       si2drObjectGetLineNo     SI2_ARGS(( si2drObjectIdT object,
00555             si2drErrorT *err));
00556
00557     si2drStringT      si2drObjectGetFileName  SI2_ARGS(( si2drObjectIdT object,
00558             si2drErrorT *err));
00559
00560     si2drVoidT        si2drReadLibertyFile     SI2_ARGS(( char *filename,
00561             si2drErrorT *err));
00562
00563     si2drVoidT        si2drWriteLibertyFile    SI2_ARGS(( char *filename,
00564             si2drGroupIdT group,
00565             char* cellname,
00566             si2drErrorT *err ));
00567
00568     si2drVoidT        si2drCheckLibertyLibrary SI2_ARGS(( si2drGroupIdT group,
00569             si2drErrorT *err));
00570
00571
00572     si2drBooleanT     si2drPIGetTraceMode      SI2_ARGS(( si2drErrorT *err));
00573
00574     si2drVoidT        si2drPIUnSetTraceMode    SI2_ARGS(( si2drErrorT *err));
00575

```

```

00576     si2drVoidT      si2drPISetTraceMode      SI2_ARGS((si2drStringT fname,
00577                                                    si2drErrorT  *err));
00578
00579
00580     si2drVoidT      si2drPISetDebugMode      SI2_ARGS((si2drErrorT  *err));
00581
00582     si2drVoidT      si2drPIUnSetDebugMode    SI2_ARGS((si2drErrorT  *err));
00583
00584     si2drBooleanT   si2drPIGetDebugMode      SI2_ARGS((si2drErrorT  *err));
00585
00586
00587     si2drVoidT      si2drPISetNocheckMode    SI2_ARGS((si2drErrorT  *err));
00588
00589     si2drVoidT      si2drPIUnSetNocheckMode  SI2_ARGS((si2drErrorT  *err));
00590
00591     si2drBooleanT   si2drPIGetNocheckMode    SI2_ARGS((si2drErrorT  *err));
00592
00593
00594     si2drVoidT      si2drGroupMoveAfter SI2_ARGS((si2drGroupIdT groupToMove,
00595                                                    si2drGroupIdT targetGroup,
00596                                                    si2drErrorT  *err));
00597
00598     si2drVoidT      si2drGroupMoveBefore SI2_ARGS((si2drGroupIdT groupToMove,
00599                                                    si2drGroupIdT targetGroup,
00600                                                    si2drErrorT  *err));
00601
00602
00603 /* HELPER FUNCTIONS/STRUCTURES */
00604
00605 #ifdef NO_LONG_DOUBLE
00606 #define LONG_DOUBLE double
00607 #define strtold strtod
00608 #else
00609 #define LONG_DOUBLE long double
00610 #endif
00611
00612 struct liberty_value_data
00613 {
00614     int dimensions;
00615     int *dim_sizes; /* a malloc'd array of <dimensions> numbers,
00616 each number is the size of the array in that dim.*/
00617     LONG_DOUBLE **index_info; /* a Ptr to a malloc'd array of size <dimensions>, ptrs to
00618 malloc'd arrays of long double index values.
00619 Each array of long double is of length set by
00620 the corresponding dim_sizes array values */
00621     LONG_DOUBLE *values; /* a ptr to a malloc'd array of long doubles,
00622 starting with [0,0,...,0,0], and ending with
00623 [z-1,y-1,...,b-1,a-1], where a-z are the max
00624 number of elements in each dimension. */
00625 };
00626
00627 LONG_DOUBLE liberty_get_element(struct liberty_value_data *vd, ...); /* returns NaN if bounds
exceeded */
00628
00629 void liberty_destroy_value_data(struct liberty_value_data *vd);
00630
00631 struct liberty_value_data *liberty_get_values_data( si2drGroupIdT table_group);
00632
00633 // directly call the main function of liberty parser
00634 extern void print_version();
00635 extern void usage();
00636 extern int main_liberty_parser(int argc, char* argv[]);
00637 extern void get_function(char* filename);
00638
00639 #ifdef __cplusplus
00640 }
00641 #endif
00642
00643 #endif
00644

```

6.15 include/verilog_utils.hpp File Reference

```

#include <string>
#include <iostream>
#include <fstream>
#include "slang/syntax/SyntaxPrinter.h"
#include "slang/syntax/SyntaxTree.h"

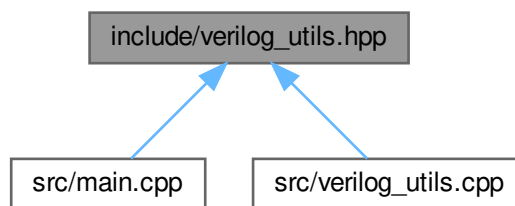
```

```
#include "slang/syntax/SyntaxVisitor.h"
```

Include dependency graph for verilog_utils.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class [VerilogVisitor](#)
- class [CellExtractor](#)
- class [CellPrinter](#)

Functions

- void [getAST](#) (const std::string &verilog_file, const std::string &cell)

6.15.1 Function Documentation

6.15.1.1 getAST()

```
void getAST (
    const std::string & verilog_file,
    const std::string & cell )
```

Definition at line [397](#) of file [verilog_utils.cpp](#).

Here is the call graph for this function:



Here is the caller graph for this function:



6.16 verilog_utils.hpp

[Go to the documentation of this file.](#)

```

00001 #ifndef VERILOG_UTILS_H
00002 #define VERILOG_UTILS_H
00003
00004 #include <string>
00005 #include <iostream> // For std::ostream
00006 #include <fstream>
00007
00008 #include "slang/syntax/SyntaxPrinter.h"
00009 #include "slang/syntax/SyntaxTree.h"
00010 #include "slang/syntax/SyntaxVisitor.h"
00011
00012 // Creating custom visitor class
00013 class VerilogVisitor : public slang::syntax::SyntaxVisitor<VerilogVisitor> {
00014 public:
00015     explicit VerilogVisitor(const std::string &targetCell);
00016     void handle(const slang::syntax::SyntaxNode &node);
00017     void handle(const slang::syntax::ModuleDeclarationSyntax &module);
00018     void handle(const slang::syntax::PortDeclarationSyntax &portDecl);
00019     void handle(const slang::syntax::HierarchyInstantiationSyntax &hierarchyInst);
00020     void handle(const slang::syntax::PrimitiveInstantiationSyntax &primitiveInst);
00021     void handle(const slang::syntax::SpecifyBlockSyntax &specifyBlock);
00022
00023 private:
00024     std::string targetCell_;
00025     int depth_;
00026     bool inTargetModule_;
00027 };
00028
00029 // Creating a Rewriter to extract a specific cell
00030 class CellExtractor : public slang::syntax::SyntaxRewriter<CellExtractor> {
00031 public:
00032     explicit CellExtractor(const std::string &targetCell);
00033     void handle(const slang::syntax::ModuleDeclarationSyntax &module);
00034     bool foundTargetCell() const;
00035
00036 private:
00037     std::string targetCell_;
00038     bool foundTarget_;
00039 };
00040
00041 // Print specific cell when visiting SyntaxTree
  
```

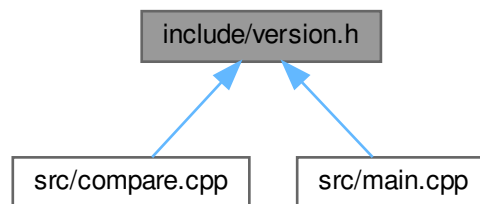
```

00042 class CellPrinter : public slang::syntax::SyntaxVisitor<CellPrinter> {
00043 public:
00044     explicit CellPrinter(const std::string &targetCell, std::ostream &out);
00045     void handle(const slang::syntax::ModuleDeclarationSyntax &module);
00046
00047 private:
00048     std::string targetCell_;
00049     std::ostream &out_;
00050     bool foundTarget_;
00051 };
00052
00053 void getAST(const std::string &verilog_file, const std::string &cell);
00054
00055 #endif // VERILOG_UTILS_H

```

6.17 include/version.h File Reference

This graph shows which files directly or indirectly include this file:



Macros

- `#define APP_NAME "ZlibValidation"`
- `#define APP_VERSION_MAJOR 0`
- `#define APP_VERSION_MINOR 1`
- `#define APP_VERSION_PATCH 0`
- `#define APP_VERSION "0.1.0"`
- `#define APP_AUTHOR "Song Zixuan"`
- `#define APP_CONTACT "cedar@zju.edu.cn"`
- `#define BUILD_TIMESTAMP "2025-03-25 22:54:12"`

6.17.1 Macro Definition Documentation

6.17.1.1 APP_AUTHOR

```
#define APP_AUTHOR "Song Zixuan"
```

Definition at line 10 of file [version.h](#).

6.17.1.2 APP_CONTACT

```
#define APP_CONTACT "cedar@zju.edu.cn"
```

Definition at line 11 of file [version.h](#).

6.17.1.3 APP_NAME

```
#define APP_NAME "ZlibValidation"
```

Definition at line 5 of file [version.h](#).

6.17.1.4 APP_VERSION

```
#define APP_VERSION "0.1.0"
```

Definition at line 9 of file [version.h](#).

6.17.1.5 APP_VERSION_MAJOR

```
#define APP_VERSION_MAJOR 0
```

Definition at line 6 of file [version.h](#).

6.17.1.6 APP_VERSION_MINOR

```
#define APP_VERSION_MINOR 1
```

Definition at line 7 of file [version.h](#).

6.17.1.7 APP_VERSION_PATCH

```
#define APP_VERSION_PATCH 0
```

Definition at line 8 of file [version.h](#).

6.17.1.8 BUILD_TIMESTAMP

```
#define BUILD_TIMESTAMP "2025-03-25 22:54:12"
```

Definition at line 12 of file [version.h](#).

6.18 version.h

[Go to the documentation of this file.](#)

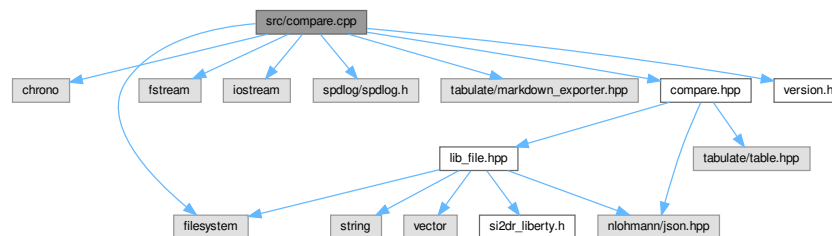
```
00001 // version.h.in
00002 #pragma once
00003
00004 // Auto-generated by CMake - DO NOT EDIT
00005 #define APP_NAME          "ZlibValidation"
00006 #define APP_VERSION_MAJOR 0
00007 #define APP_VERSION_MINOR 1
00008 #define APP_VERSION_PATCH 0
00009 #define APP_VERSION       "0.1.0"
00010 #define APP_AUTHOR        "Song Zixuan"
00011 #define APP_CONTACT       "cedar@zju.edu.cn"
00012 #define BUILD_TIMESTAMP   "2025-03-25 22:54:12"
```

6.19 README.md File Reference

6.20 src/compare.cpp File Reference

```
#include <chrono>
#include <filesystem>
#include <fstream>
#include <iostream>
#include "spdlog/spdlog.h"
#include <tabulate/markdown_exporter.hpp>
#include "compare.hpp"
#include "version.h"
```

Include dependency graph for compare.cpp:



6.21 compare.cpp

[Go to the documentation of this file.](#)

```

00001 #include <chrono>
00002 #include <filesystem>
00003 #include <fstream>
00004 #include <iostream>
00005
00006 #include "spdlog/spdlog.h"
00007 #include <tabulate/markdown_exporter.hpp>
00008
00009 #include "compare.hpp"
00010 #include "version.h"
00011
00023 LibraryComparator::LibraryComparator(LibFile &ref_libfile, LibFile &comp_libfile, double reltol,
00024                                     double abstol) {
00025     : reltol_(reltol), abstol_(abstol) {
00026     std::string ref_json_name = ref_libfile.basename_ + ".json";
00027     if (!std::filesystem::exists(ref_json_name)) {
00028         spdlog::info("Reference JSON file {} not found. Parsing first.", ref_json_name);
00029         ref_libfile.parse();
00030         ref_libfile.writeJsonToFile();
00031         ref_json_ = ref_libfile.lib_json_;
00032     } else {
00033         // Read the JSON file into json object
00034         std::ifstream ref_in(ref_json_name);
00035         if (!ref_in.is_open()) {
00036             spdlog::error("Could not open file '{}' for reading", ref_json_name);
00037             return;
00038         }
00039         try {
00040             ref_json_ = json::parse(ref_in);
00041         } catch (const json::parse_error &e) {
00042             spdlog::error("Error parsing reference JSON file '{}': {}", ref_json_name, e.what());
00043             return;
00044         }
00045     }
00046     std::string comp_json_name = comp_libfile.basename_ + ".json";
00047     if (!std::filesystem::exists(comp_json_name)) {
00048         spdlog::info("Comparison JSON file {} not found. Parsing first.", comp_json_name);
00049         comp_libfile.parse();
00050         comp_libfile.writeJsonToFile();
00051         comp_json_ = comp_libfile.lib_json_;
00052     } else {
00053         // Read the JSON file into json object
00054         std::ifstream comp_in(comp_json_name);
00055         if (!comp_in.is_open()) {
00056             spdlog::error("Could not open file '{}' for reading", comp_json_name);
00057             return;
00058         }
00059         try {
00060             comp_json_ = json::parse(comp_in);
00061         } catch (const json::parse_error &e) {
00062             spdlog::error("Error parsing comparison JSON file '{}': {}", comp_json_name, e.what());
00063             return;
00064         }
00065     }
00066
00067     ref_lib_path_ = ref_libfile.filepath_;
00068     comp_lib_path_ = comp_libfile.filepath_;
00069     spdlog::info("Successfully loaded JSON files for comparison");
00070 }
00071
00098 void LibraryComparator::compareLut(const std::string &cell_name, const std::string &pin_name,
00099                                   const std::string &timing_type, const std::string &related_pin,
00100                                   const std::string &arc_name, const json &ref_lut,
00101                                   const json &comp_lut, Table &table) {
00102     spdlog::info("Comparing LUT of '{}' ...", arc_name);
00103     std::vector<double> ref_index_1 = ref_lut["index_1"].get<std::vector<double>>();
00104     std::vector<double> ref_index_2 = ref_lut["index_2"].get<std::vector<double>>();
00105     std::vector<double> comp_index_1 = comp_lut["index_1"].get<std::vector<double>>();
00106     std::vector<double> comp_index_2 = comp_lut["index_2"].get<std::vector<double>>();
00107     if (ref_index_1 != comp_index_1 || ref_index_2 != comp_index_2) {
00108         spdlog::warn("Mismatch in LUT indices for '{}' in cell: '{}', pin: '{}', related_pin: '{}', "
00109                     "timing_type: '{}', "
00110                     "arc_name, cell_name, pin_name, related_pin, timing_type);
00111         return;
00112     } else {
00113         spdlog::debug("LUT indices match for '{}'", arc_name);
00114
00115         std::vector<std::vector<double>> comp_value_matrix;
00116         // Generate a matrix of values from the JSON array for comparison library
00117         for (const auto &row_val : comp_lut["values"]) {
00118             std::vector<double> row_data;
00119             // Check if the row value is an array

```

```

00120         if (!row_val.is_array()) {
00121             spdlog::error("Invalid LUT format: '{}' values should be an array of arrays in comp_lib '{}',
"
00122                 "cell '{}', pin '{}', "
00123                 "related_pin '{}', timing_type '{}'",
00124                 arc_name, this->comp_lib_path_.string(), cell_name, pin_name, related_pin,
00125                 timing_type);
00126             return;
00127         }
00128         // Check if non-numeric values
00129         for (const auto &val : row_val) {
00130             if (val.is_number()) {
00131                 row_data.push_back(val.get<double>());
00132             } else {
00133                 spdlog::error(
00134                     "Non-numeric value found in '{}'.values, skipping value: {} in comp_lib '{}', cell
'{}', "
00135                     "pin '{}', related_pin '{}', timing_type '{}'",
00136                     arc_name, val.dump(), this->comp_lib_path_.string(), cell_name, pin_name, related_pin,
00137                     timing_type);
00138                 return;
00139             }
00140         }
00141         comp_value_matrix.push_back(row_data);
00142     }
00143     // Generate a matrix of values from the JSON array for reference library
00144     std::vector<std::vector<double>> ref_value_matrix;
00145     for (const auto &row_val : ref_lut["values"]) {
00146         std::vector<double> row_data;
00147         // Check if the row value is an array
00148         if (!row_val.is_array()) {
00149             spdlog::error("Invalid LUT format: '{}' values should be an array of arrays in ref_lib '{}',
"
00150                 "cell '{}', pin '{}', "
00151                 "related_pin '{}', timing_type '{}'",
00152                 arc_name, this->ref_lib_path_.string(), cell_name, pin_name, related_pin,
00153                 timing_type);
00154             return;
00155         }
00156         // Check if non-numeric values
00157         for (const auto &val : row_val) {
00158             if (val.is_number()) {
00159                 row_data.push_back(val.get<double>());
00160             } else {
00161                 spdlog::error(
00162                     "Non-numeric value found in '{}'.values, skipping value: {} in ref_lib '{}', cell '{}',
"
00163                     "pin '{}', related_pin '{}', timing_type '{}'",
00164                     arc_name, val.dump(), this->ref_lib_path_.string(), cell_name, pin_name, related_pin,
00165                     timing_type);
00166                 return;
00167             }
00168         }
00169         ref_value_matrix.push_back(row_data);
00170     }
00171     // Compare the matrix for relative tolerance
00172     if (!comp_value_matrix.empty() && !comp_value_matrix[0].empty()) {
00173         for (size_t i = 0; i < comp_value_matrix.size(); ++i) {
00174             for (size_t j = 0; j < comp_value_matrix[i].size(); ++j) {
00175                 double ref_val = ref_value_matrix[i][j];
00176                 double comp_val = comp_value_matrix[i][j];
00177                 if (std::abs(ref_val - comp_val) > reltol_ * std::abs(ref_val) &&
00178                     std::abs(ref_val - comp_val) > abstol_) {
00179                     spdlog::debug("LUT value mismatch for '{}', index_1: {}, index_2: {}", arc_name,
00180                         ref_index_1[i], ref_index_2[j]);
00181                     table.add_row({related_pin + "->" + pin_name, std::to_string(ref_val),
00182                         std::to_string(comp_val), std::to_string(comp_val - ref_val),
00183                         std::to_string((comp_val - ref_val) / ref_val * 100), timing_type,
00184                         arc_name,
00185                         std::to_string(i + 1), std::to_string(ref_index_1[i]), std::to_string(j +
1),
00186                         std::to_string(ref_index_2[j]), "<"});
00187                     spdlog::debug("table size: {}", table.size());
00188                 } else {
00189                     spdlog::debug("LUT value match for '{}', index_1: {}, index_2: {}", arc_name,
00190                         ref_index_1[i],
00191                         ref_index_2[j]);
00192                 }
00193             }
00194         }
00195         else {
00196             spdlog::error(
00197                 "Empty or invalid 'values' array found for cell: '{}', pin: '{}', related_pin: '{}', "
00198                 "timing_type: '{}', arc: '{}'",
00199                 cell_name, pin_name, related_pin, timing_type, arc_name);
00200         }
00201     }

```

```

00200 }
00201
00219 void LibraryComparator::compareTimingArc(const std::string &cell_name, const std::string &pin_name,
00220                                           const std::string &timing_type, const json &ref_timing_arc,
00221                                           const json &comp_timing_arc, Table &table) {
00222     spdlog::info("Comparing timing type '{}'. ...", timing_type);
00223
00224     std::string related_pin = ref_timing_arc["related_pin"].get<std::string>();
00225     spdlog::debug("Related pin: '{}'", related_pin);
00226
00227     std::vector<std::string> arc_names = {"cell_rise", "cell_fall", "rise_transition",
00228     "fall_transition"};
00229     for (auto arc_name : arc_names) {
00230         if (comp_timing_arc.contains(arc_name)) {
00231             spdlog::debug("Comparing timing arc: '{}'", arc_name);
00232             // Look for the arc in the reference JSON
00233             if (ref_timing_arc.contains(arc_name)) {
00234                 // Found this arc
00235                 spdlog::debug("Found timing arc: '{}'", arc_name);
00236                 compareLut(cell_name, pin_name, timing_type, related_pin, arc_name, ref_timing_arc[arc_name],
00237                 comp_timing_arc[arc_name], table);
00238             } else {
00239                 // arc not found in reference JSON
00240                 spdlog::warn("Timing arc: '{}'. not found in reference JSON", arc_name);
00241             }
00242         }
00243     }
00244
00259 void LibraryComparator::comparePin(const std::string &cell_name, const std::string &pin_name,
00260                                    const json &ref_pin, const json &comp_pin, Table &table) {
00261     spdlog::info("Comparing pin '{}'. ...", pin_name);
00262
00263     if (comp_pin.contains("timing_arcs")) {
00264         for (const auto &comp_arc : comp_pin["timing_arcs"]) {
00265             std::string timing_type = comp_arc["timing_type"].get<std::string>();
00266             spdlog::debug("Comparing timing type: '{}'", timing_type);
00267             // Look for the timing type in the reference JSON
00268             auto ref_arc_it = std::find_if(
00269                 ref_pin["timing_arcs"].begin(), ref_pin["timing_arcs"].end(),
00270                 [&timing_type](const json &ref_arc) { return ref_arc["timing_type"] == timing_type; });
00271             if (ref_arc_it != ref_pin["timing_arcs"].end()) {
00272                 // Found this timing type
00273                 spdlog::debug("Found timing type: '{}'", timing_type);
00274                 compareTimingArc(cell_name, pin_name, timing_type, *ref_arc_it, comp_arc, table);
00275             } else {
00276                 // timing arc not found in reference JSON
00277                 spdlog::warn("Timing arc: '{}'. not found in reference JSON", timing_type);
00278             }
00279         }
00280     } else {
00281         spdlog::info("No timing arcs found for pin: '{}'.", comp_pin["pin_name"].get<std::string>());
00282     }
00283 }
00284
00298 void LibraryComparator::compareCell(const std::string &cell_name, const json &ref_cell,
00299                                     const json &comp_cell, Table &table) {
00300     spdlog::info("Comparing cell '{}'. ...", cell_name);
00301
00302     if (comp_cell.contains("output_pins")) {
00303         for (const auto &comp_pin : comp_cell["output_pins"]) {
00304             std::string pin_name = comp_pin["pin_name"].get<std::string>();
00305             spdlog::debug("Comparing pin: '{}'", pin_name);
00306             // Look for the pin in the reference JSON
00307             auto ref_pin_it =
00308                 std::find_if(ref_cell["output_pins"].begin(), ref_cell["output_pins"].end(),
00309                 [&pin_name](const json &ref_pin) { return ref_pin["pin_name"] == pin_name; });
00310             if (ref_pin_it != ref_cell["output_pins"].end()) {
00311                 // Found this pin
00312                 spdlog::debug("Found pin: '{}'", pin_name);
00313                 comparePin(cell_name, pin_name, *ref_pin_it, comp_pin, table);
00314             } else {
00315                 // pin not found in reference JSON
00316                 spdlog::warn("Pin: '{}'. not found in reference JSON", pin_name);
00317             }
00318         }
00319     } else {
00320         spdlog::info("No output pins found for cell: '{}'.", cell_name);
00321     }
00322 }
00323
00340 void LibraryComparator::generateReport(const std::string &output_file) {
00341     std::ofstream outfile(output_file);
00342     outfile << "# LIBRARY comparison\n" << std::endl;
00343     outfile << "***Reference library: " << ref_lib_path_ << "***\n" << std::endl;
00344     outfile << "***Comparison library: " << comp_lib_path_ << "***\n" << std::endl;
00345     outfile << "***Absolute tolerance: " << abstol_ << "***\n" << std::endl;

```

```

00346 outfile << "**Relative tolerance: " << reltol_ << "**\n" << std::endl;
00347 outfile << "**Performed by " << APP_NAME << " v" << APP_VERSION << " from " << APP_AUTHOR;
00348 auto now = std::chrono::system_clock::now();
00349 std::time_t now_time_t = std::chrono::system_clock::to_time_t(now);
00350 std::tm *now_tm = std::localtime(&now_time_t);
00351 outfile << ". on: " << std::put_time(now_tm, "%c") << "**\n" << std::endl;
00352 outfile << "> Legend: < outlier, * scaled, ! indices switched, ^ slews extrapolated, ~ loads "
00353 "extrapolated,\n";
00354 outfile << "> \n";
00355 outfile
00356 << "> Legend: + padding added, /0 divide by zero, / slews interpolated, # loads interpolated\n";
00357 outfile << "> \n";
00358 outfile << "> Legend: << value is less but unknown, >> value is more but unknown\n\n";
00359
00360 spdlog::info("Starting comparison report generation ...");
00361 for (const auto &comp_cell : comp_json_["cells"]) {
00362     std::string cell_name = comp_cell["cell_name"].get<std::string>();
00363     spdlog::debug("Comparing cell: '{}'", cell_name);
00364     // Look for the cell in the reference JSON
00365     auto ref_cell_it =
00366         std::find_if(ref_json_["cells"].begin(), ref_json_["cells"].end(),
00367             [&cell_name](const json &ref_cell) { return ref_cell["cell_name"] == cell_name;
00368 });
00369 if (ref_cell_it != ref_json_["cells"].end()) {
00370     // Found this cell
00371     spdlog::debug("Found cell: '{}'", cell_name);
00372     Table table;
00373     table.add_row({"Pin Name", "Reference", "Comparison", "Diff", "Diff %", "Type", "Arc Name",
00374         "Row #", "Index_1", "Col #", "Index_2", "Note"});
00375     // Header formatting
00376     for (size_t i = 0; i < table[0].size(); ++i) {
00377         table[0][i].format().font_color(Color::yellow).font_style({FontStyle::bold});
00378     }
00379     compareCell(cell_name, *ref_cell_it, comp_cell, table);
00380
00381     if (table.size() > 1) {
00382         // Data starts from row 1, row 0 is header
00383         size_t failed_count = table.size() - 1;
00384         double sum_diff = 0.0;
00385         double sum_diff_percent = 0.0;
00386         size_t outlier_count = 0;
00387         double max_diff = -1e9;
00388         double max_diff_percent = -1e9;
00389         std::vector<double> diff_values; // Store diff values for std deviation calculation
00390
00391         // Index of columns
00392         const int diff_index = 3;
00393         const int diff_percent_index = 4;
00394         const int note_index = 11;
00395
00396         for (size_t i = 1; i < table.size(); ++i) {
00397             try {
00398                 double diff = std::stod(table[i][diff_index].get_text());
00399                 double diff_percent = std::stod(table[i][diff_percent_index].get_text());
00400
00401                 sum_diff += diff;
00402                 sum_diff_percent += diff_percent;
00403                 diff_values.push_back(diff_percent);
00404
00405                 if (table[i][note_index].get_text() == "<") {
00406                     outlier_count++;
00407                 }
00408                 max_diff = std::max(max_diff, diff);
00409                 max_diff_percent = std::max(max_diff_percent, diff_percent);
00410             } catch (const std::invalid_argument &e) {
00411                 spdlog::error("Could not convert value to double: {}", e.what());
00412                 continue; // Skip this row if there's an error
00413             }
00414         }
00415
00416         double avg_diff = sum_diff / failed_count;
00417         double avg_diff_percent = sum_diff_percent / failed_count;
00418
00419         outfile << "## " << cell_name << "\n\n";
00420         outfile << "Delay Comparison in ns\n\n";
00421         if (output_file.substr(output_file.size() - 3) == ".md") {
00422             MarkdownExporter exporter;
00423             auto markdown = exporter.dump(table);
00424             outfile << markdown << std::endl;
00425         } else {
00426             outfile << table << std::endl;
00427         }
00428         outfile << std::endl;
00429         std::cout << table << std::endl;
00430         outfile << cell_name << " delay SUMMARY (abstol: " << abstol_ << ", reltol: " << reltol_
00431 << ")\n";

```

```

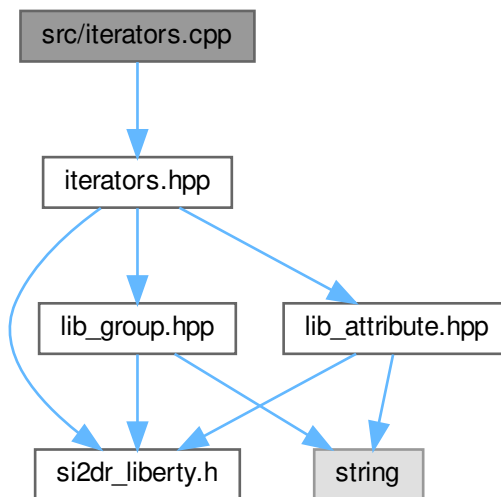
00432
00433     // Create summary table
00434     Table summary_table;
00435     summary_table.add_row({"Cell Name", "Data Type", "Failed Count", "Avg Diff", "Avg Diff%",
00436                           "Max Diff", "Max Diff%", "Outliers"});
00437     summary_table.add_row({cell_name, "delay(ns)", std::to_string(failed_count),
00438                           std::to_string(avg_diff), std::to_string(avg_diff_percent) + "%",
00439                           std::to_string(max_diff), std::to_string(max_diff_percent) + "%",
00440                           std::to_string(outlier_count)});
00441
00442     // Output summary table
00443     if (output_file.substr(output_file.size() - 3) == ".md") {
00444         MarkdownExporter exporter;
00445         auto markdown = exporter.dump(summary_table);
00446         outfile << markdown << std::endl;
00447     } else {
00448         outfile << summary_table << std::endl;
00449     }
00450     outfile << "Worst delay outlier: Max Abs: " << max_diff << ", Max Rel: " << max_diff_percent
00451              << "%, Outliers: " << outlier_count << "\n\n";
00452 }
00453 } else {
00454     // cell not found in reference JSON
00455     spdlog::warn("Cell: '{}' not found in reference JSON", cell_name);
00456 }
00457 }
00458
00459 outfile.close();
00460 }

```

6.22 src/iterators.cpp File Reference

#include "iterators.hpp"

Include dependency graph for iterators.cpp:



6.23 iterators.cpp

[Go to the documentation of this file.](#)

```
00001 #include "iterators.hpp"
```

```

00002
00012 GroupsIterator::GroupsIterator(si2drGroupsIdT groups, si2drErrorT &err) : groups_(groups), err_(err)
    {
00013     group_ = si2drIterNextGroup(groups_, &err_);
00014 }
00015 GroupsIterator::~GroupsIterator() { si2drIterQuit(groups_, &err_); }
00016
00017 void GroupsIterator::next() { group_ = si2drIterNextGroup(groups_, &err_); }
00018 bool GroupsIterator::end() { return si2drObjectIsNull(group_, &err_); }
00019
00020 LibGroup GroupsIterator::get() { return LibGroup(group_, err_); }
00021
00032 AttributesIterator::AttributesIterator(si2drAttrsIdT attrs, si2drErrorT &err) : attrs_(attrs),
    err_(err) {
00033     attr_ = si2drIterNextAttr(attrs_, &err_);
00034 }
00035 AttributesIterator::~AttributesIterator() { si2drIterQuit(attrs_, &err_); }
00036
00037 void AttributesIterator::next() { attr_ = si2drIterNextAttr(attrs_, &err_); }
00038 bool AttributesIterator::end() { return si2drObjectIsNull(attr_, &err_); }
00039
00040 LibAttribute AttributesIterator::get() { return LibAttribute(attr_, err_); }
00041
00051 ValuesIterator::ValuesIterator(si2drValuesIdT values, si2drErrorT &err) : values_(values), err_(err)
    {
00052     si2drIterNextComplexValue(values_, &vtype_, &int_, &float_, &str_, &bool_, &exprp_, &err_);
00053 }
00054 ValuesIterator::~ValuesIterator() { si2drIterQuit(values_, &err_); }
00055
00056 void ValuesIterator::next() {
00057     si2drIterNextComplexValue(values_, &vtype_, &int_, &float_, &str_, &bool_, &exprp_, &err_);
00058 }
00059 bool ValuesIterator::end() { return vtype_ == SI2DR_UNDEFINED_VALUETYPE; }

```

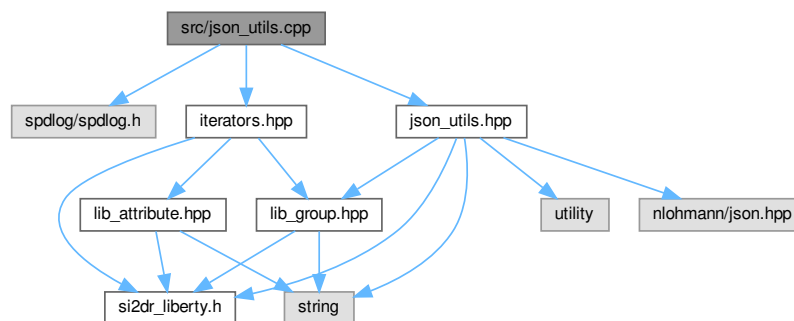
6.24 src/json_utils.cpp File Reference

```

#include "spdlog/spdlog.h"
#include "iterators.hpp"
#include "json_utils.hpp"

```

Include dependency graph for json_utils.cpp:



Functions

- `std::vector< double > parseStringToVector (const std::string &str)`
Parses a comma-separated string into a vector of doubles.
- `json generateLutJson (LibGroup &lib_lut_group, si2drErrorT &err)`
Generates a JSON object representing a lookup table (LUT) from a given library group.
- `json generateTimingJson (LibGroup &lib_timing_group, si2drErrorT &err)`

- Generates a JSON representation of timing data from a Liberty timing group.*
 - `json generatePowerJson (LibGroup &lib_power_group, si2drErrorT &err)`
Generates a JSON object representation of a power group from Liberty format.
- `std::pair< std::string, json > generatePinJson (LibGroup &lib_pin_group, si2drErrorT &err)`
Generates a JSON representation of a library pin group.
- `json generateCellJson (LibGroup &lib_cell_group, si2drErrorT &err)`
Generates a JSON representation of a library cell group.

6.24.1 Function Documentation

6.24.1.1 generateCellJson()

```
json generateCellJson (  
    LibGroup & lib_cell_group,  
    si2drErrorT & err )
```

Generates a JSON representation of a library cell group.

This function builds a JSON object containing cell properties by extracting data from the provided `LibGroup` object. It processes:

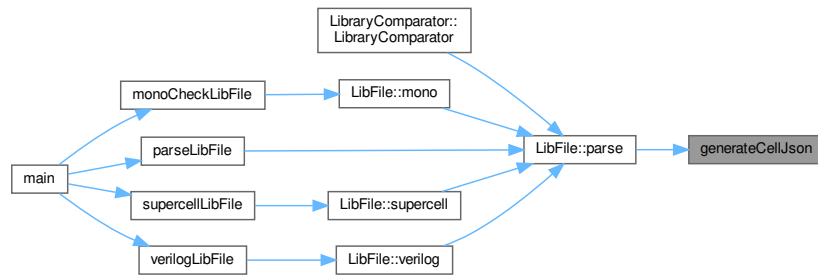
- Cell name
- Basic attributes (area, cell_leakage_power, cell_footprint)
- Pin information (categorized by direction: input, output, internal, inout)

The function iterates through both attributes and sub-groups of the cell group, converting relevant information to a structured JSON format.

Parameters

<code>lib_cell_group</code>	The <code>LibGroup</code> object representing the cell
<code>err</code>	Reference to an error tracking object used during iteration

Here is the caller graph for this function:



6.24.1.2 generateLutJson()

```

json generateLutJson (
    LibGroup & lib_lut_group,
    si2drErrorT & err )

```

Generates a JSON object representing a lookup table (LUT) from a given library group.

This function iterates over the attributes of the provided library group and processes them to generate a JSON object. It specifically handles attributes named "index_1", "index_2", and "values", converting their string representations into vectors and adding them to the JSON object. If an attribute with an unknown name is encountered, a warning is logged.

Parameters

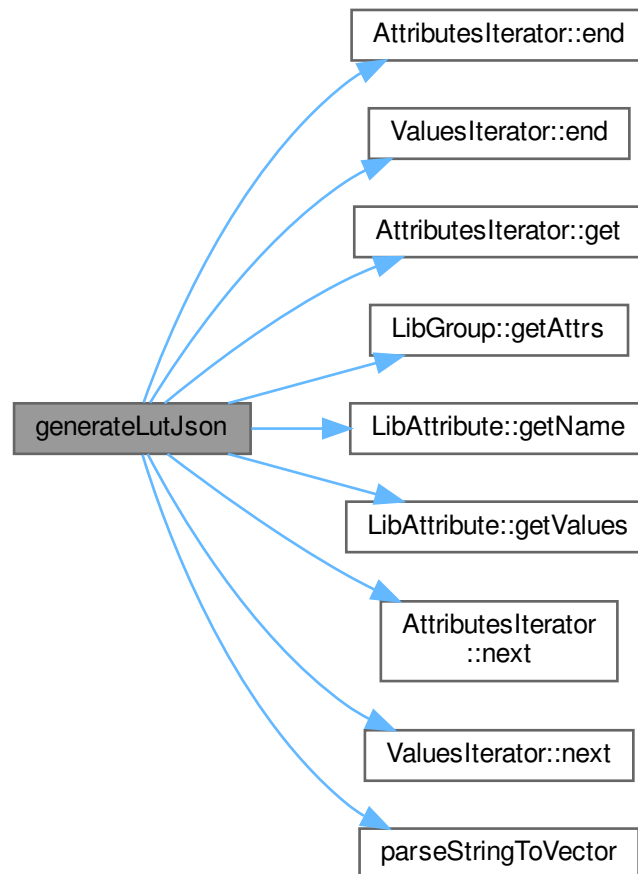
<i>lib_lut_group</i>	The library group containing the LUT attributes.
<i>err</i>	Reference to an error variable to capture any errors during attribute iteration.

Returns

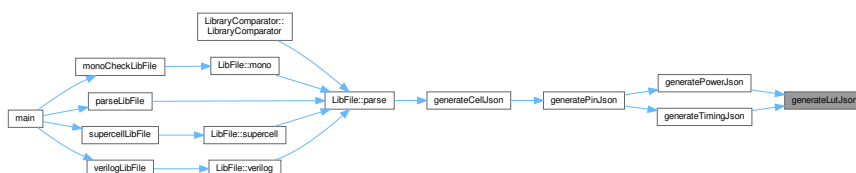
A JSON object representing the LUT.

Definition at line 57 of file [json_utils.cpp](#).

Here is the call graph for this function:



Here is the caller graph for this function:



6.24.1.3 generatePinJson()

```
std::pair< std::string, json > generatePinJson (
    LibGroup & lib_pin_group,
    si2drErrorT & err )
```

Generates a JSON representation of a library pin group.

This function processes a Liberty format pin group and converts it into a JSON object. It extracts various pin attributes such as direction, capacitance values, and functional attributes. It also recursively processes sub-groups including timing and power arcs.

Parameters

<i>lib_pin_group</i>	The Liberty pin group to be processed
<i>err</i>	Reference to a si2drErrorT object to track any errors that occur during processing

Returns

A pair containing:

- first: The pin direction as a string ("input", "output", "inout", etc.)
- second: A JSON object containing all the extracted pin information including:
 - pin_name: Name of the pin
 - direction: Pin direction (stored in the returned pair's first element)
 - Various attributes like max_transition, capacitance values, function, etc.
 - timing_arcs: Array of timing arcs if present
 - power_arcs: Array of power arcs if present

Note

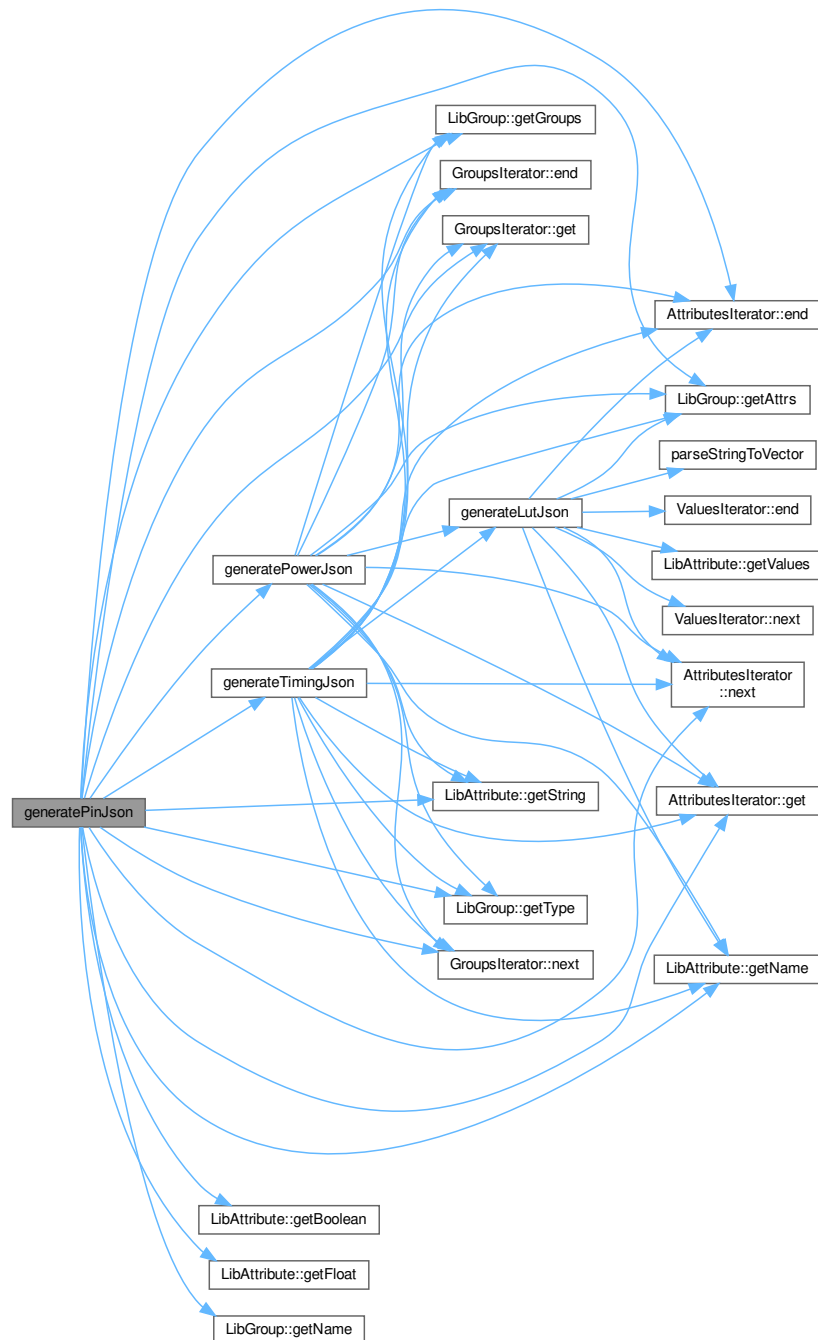
Handles the following pin attributes:

- direction
- max_transition, capacitance, rise_capacitance, fall_capacitance, max_capacitance
- function, power_down_function, related_ground_pin, related_power_pin, three_state
- clock

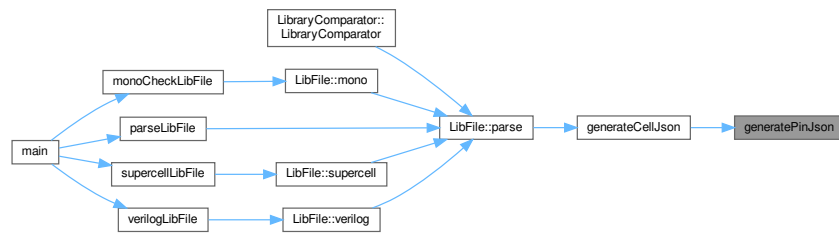
Processes "internal_power" and "timing" sub-groups recursively

Definition at line 218 of file [json_utils.cpp](#).

Here is the call graph for this function:



Here is the caller graph for this function:



6.24.1.4 generatePowerJson()

```

json generatePowerJson (
    LibGroup & lib_power_group,
    si2drErrorT & err )

```

Generates a JSON object representation of a power group from Liberty format.

This function processes a [LibGroup](#) representing a power group from a Liberty file and converts it to a JSON object. It extracts relevant attributes such as "when", "related_pin", and "related_pg_pin" and processes subgroups like "rise_power" and "fall_power", typically containing lookup tables (LUTs).

Parameters

<i>lib_power_group</i>	The Liberty power group to convert to JSON
<i>err</i>	Reference to a <code>si2drErrorT</code> object for error handling

Returns

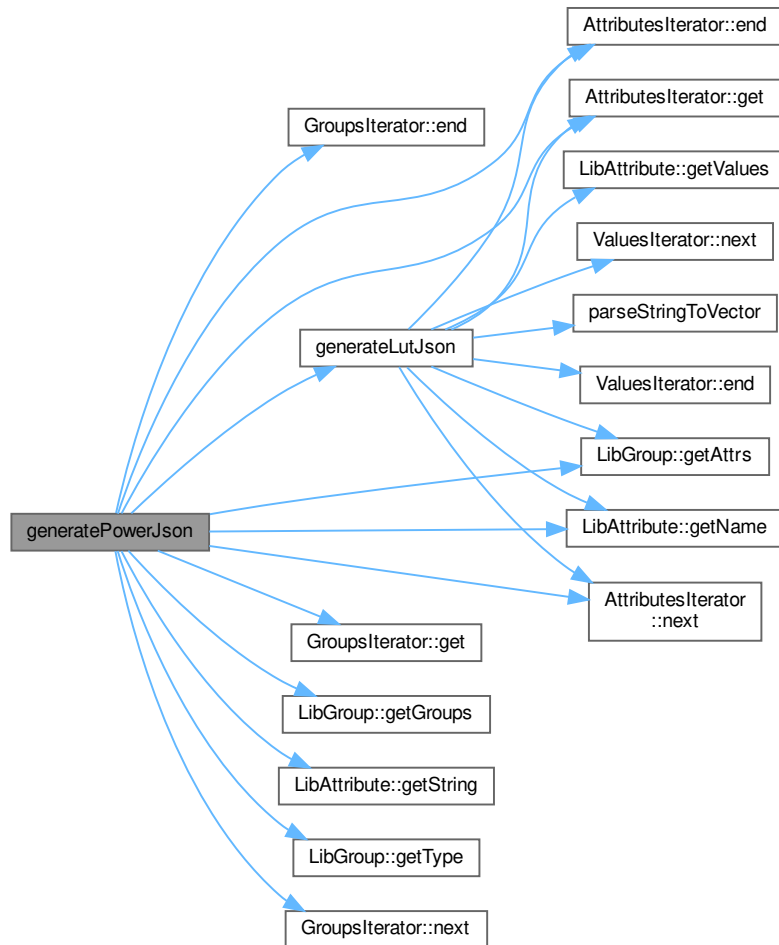
json A JSON object containing the power group information

Note

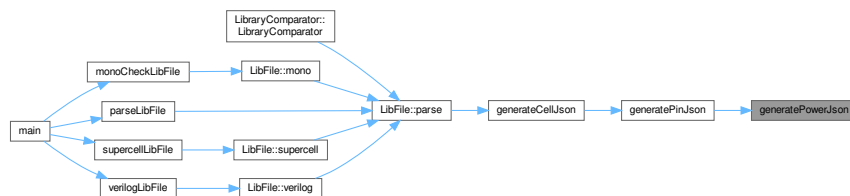
The function currently handles the attributes "when", "related_pin", "related_pg_pin" and the subgroup types "rise_power" and "fall_power". Other subgroup types will generate a warning.

Definition at line 160 of file [json_utils.cpp](#).

Here is the call graph for this function:



Here is the caller graph for this function:



6.24.1.5 generateTimingJson()

```

json generateTimingJson (
    LibGroup & lib_timing_group,
    si2drErrorT & err )

```

Generates a JSON representation of timing data from a Liberty timing group.

This function extracts timing-related information from a Liberty timing group and organizes it into a JSON object. It processes both attributes and subgroups of the timing group.

For attributes, it extracts string values for:

- `related_pin`: The related pin for this timing arc
- `timing_type`: The type of timing (setup, hold, etc.)
- `timing_sense`: The timing sense (positive_unate, negative_unate, non_unate)
- `when`: The timing condition

For subgroups, it processes lookup tables (LUTs) for:

- `cell_fall`: Cell delay for falling transitions
- `cell_rise`: Cell delay for rising transitions
- `fall_transition`: Output transition time for falling edges
- `rise_transition`: Output transition time for rising edges
- `fall_constraint`: Timing constraints for falling edges
- `rise_constraint`: Timing constraints for rising edges

Parameters

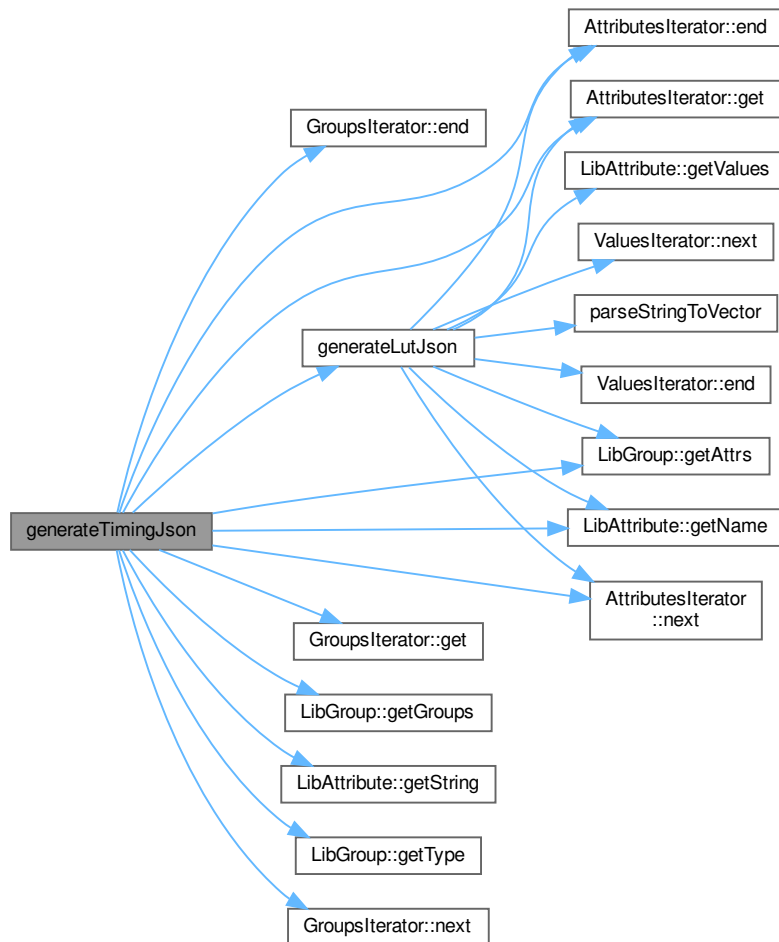
<i>lib_timing_group</i>	The Liberty timing group to process
<i>err</i>	Reference to an <code>si2drErrorT</code> object for error handling

Returns

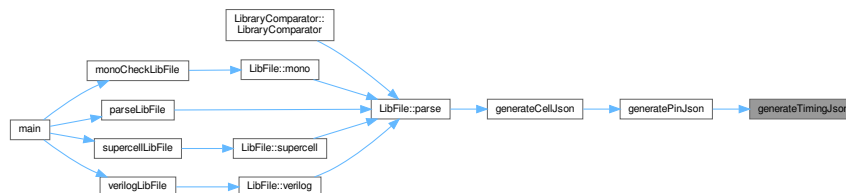
`json` A JSON object containing the extracted timing information

Definition at line 113 of file [json_utils.cpp](#).

Here is the call graph for this function:



Here is the caller graph for this function:



6.24.1.6 parseStringToVector()

```

std::vector< double > parseStringToVector (
    const std::string & str )

```

Parses a comma-separated string into a vector of doubles.

This function takes a string containing comma-separated values and converts it into a vector of doubles. Each value in the string is expected to be a valid representation of a double.

Parameters

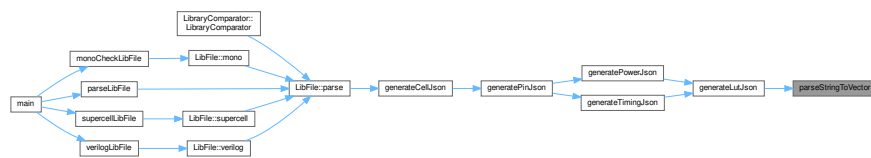
<code>str</code>	The input string containing comma-separated double values.
------------------	--

Returns

A vector of doubles parsed from the input string.

Definition at line 25 of file [json_utils.cpp](#).

Here is the caller graph for this function:



6.25 json_utils.cpp

[Go to the documentation of this file.](#)

```

00001 #include "spdlog/spdlog.h"
00002
00003 #include "iterators.hpp"
00004 #include "json_utils.hpp"
00005
00006 /*
00007 json
00008 []
00009 JSON null
00010 {}
00011 JSON
00012 null
00013 */
00014
00025 std::vector<double> parseStringToVector(const std::string &str) {
00026     std::vector<double> result;
00027     std::string cleaned_str;
00028
00029     // Remove backslashes and newline characters
00030     for (char c : str) {
00031         if (c != '\\' && c != '\n') {
00032             cleaned_str += c;
00033         }
00034     }
00035
00036     std::stringstream ss(cleaned_str);
00037     std::string item;
00038     while (std::getline(ss, item, ',')) {
00039         result.push_back(std::stod(item));
00040     }
00041     return result;
00042 }
00043
00057 json generateLutJson(LibGroup &lib_lut_group, si2drErrorT &err) {
00058     json lut_json;
00059
00060     AttributesIterator attr_iter(lib_lut_group.getAttrs(), err);
00061     for (; !attr_iter.end(); attr_iter.next()) {

```

```

00062     LibAttribute lib_attr = attr_iter.get();
00063     std::string lut_attr_name = lib_attr.getName();
00064
00065     // spdlog::debug("LUT Attribute Name: {}", lib_attr.getName());
00066     // spdlog::debug("Is Complex? {}", lib_attr.isComplex());
00067
00068     if (lut_attr_name == "index_1" || lut_attr_name == "index_2") {
00069         ValuesIterator values_iter(lib_attr.getValues(), err);
00070         for (; !values_iter.end(); values_iter.next()) {
00071             // spdlog::debug("Type: {}", int(values_iter.vtype_)); // 5 is string
00072             // spdlog::debug("Str: {}", values_iter.str_);
00073             lut_json[lut_attr_name] = parseStringToVector(std::string(values_iter.str_));
00074         }
00075     } else if (lut_attr_name == "values") {
00076         ValuesIterator values_iter(lib_attr.getValues(), err);
00077         for (; !values_iter.end(); values_iter.next()) {
00078             // spdlog::debug("{}", values_iter.str_);
00079             lut_json["values"].push_back(parseStringToVector(std::string(values_iter.str_)));
00080         }
00081     } else {
00082         spdlog::warn("Unknown LUT attribute name: {}", lut_attr_name);
00083     }
00084 }
00085 return lut_json;
00086 }
00087
00113 json generateTimingJson(LibGroup &lib_timing_group, si2drErrorT &err) {
00114     json timing_json;
00115
00116     AttributesIterator attr_iter(lib_timing_group.getAttrs(), err);
00117     for (; !attr_iter.end(); attr_iter.next()) {
00118         LibAttribute lib_attr = attr_iter.get();
00119
00120         std::string attr_name = lib_attr.getName();
00121         if (attr_name == "related_pin" || attr_name == "timing_type" || attr_name == "timing_sense" ||
00122             attr_name == "when") {
00123             timing_json[attr_name] = lib_attr.getString();
00124         }
00125         // More timing attributes can be added here
00126     }
00127
00128     GroupsIterator timing_sub_group_iter(lib_timing_group.getGroups(), err);
00129     for (; !timing_sub_group_iter.end(); timing_sub_group_iter.next()) {
00130         LibGroup lib_timing_sub_group = timing_sub_group_iter.get();
00131
00132         std::string timing_sub_group_type = lib_timing_sub_group.getType();
00133         // std::string timing_sub_group_name = lib_timing_sub_group.getName();
00134         if (timing_sub_group_type == "cell_fall" || timing_sub_group_type == "cell_rise" ||
00135             timing_sub_group_type == "fall_transition" || timing_sub_group_type == "rise_transition" ||
00136             timing_sub_group_type == "fall_constraint" || timing_sub_group_type == "rise_constraint") {
00137             timing_json[timing_sub_group_type] = generateLutJson(lib_timing_sub_group, err);
00138         } else {
00139             spdlog::warn("Unknown timing sub group type: {}", timing_sub_group_type);
00140         }
00141     }
00142     return timing_json;
00143 }
00144
00160 json generatePowerJson(LibGroup &lib_power_group, si2drErrorT &err) {
00161     json power_json;
00162
00163     AttributesIterator attr_iter(lib_power_group.getAttrs(), err);
00164     for (; !attr_iter.end(); attr_iter.next()) {
00165         LibAttribute lib_attr = attr_iter.get();
00166
00167         std::string attr_name = lib_attr.getName();
00168         if (attr_name == "when" || attr_name == "related_pin" || attr_name == "related_pg_pin") {
00169             power_json[attr_name] = lib_attr.getString();
00170         }
00171         // More power attributes can be added here
00172     }
00173
00174     GroupsIterator power_sub_group_iter(lib_power_group.getGroups(), err);
00175     for (; !power_sub_group_iter.end(); power_sub_group_iter.next()) {
00176         LibGroup lib_power_sub_group = power_sub_group_iter.get();
00177
00178         std::string power_sub_group_type = lib_power_sub_group.getType();
00179         // std::string power_sub_group_name = lib_power_sub_group.getName();
00180         if (power_sub_group_type == "rise_power") {
00181             // spdlog::debug("Rise Power: {}", power_sub_group_name);
00182             power_json["rise_power"] = generateLutJson(lib_power_sub_group, err);
00183         } else if (power_sub_group_type == "fall_power") {
00184             power_json["fall_power"] = generateLutJson(lib_power_sub_group, err);
00185         } else {
00186             spdlog::warn("Unknown power sub group type: {}", power_sub_group_type);
00187         }
00188     }

```

```

00189     return power_json;
00190 }
00191
00218 std::pair<std::string, json> generatePinJson(LibGroup &lib_pin_group, si2drErrorT &err) {
00219     json pin_json;
00220     std::string direction;
00221     pin_json["pin_name"] = lib_pin_group.getName();
00222
00223     AttributesIterator attr_iter(lib_pin_group.getAttrs(), err);
00224     for (; !attr_iter.end(); attr_iter.next()) {
00225         LibAttribute lib_attr = attr_iter.get();
00226
00227         std::string attr_name = lib_attr.getName();
00228         if (attr_name == "direction") {
00229             direction = lib_attr.getString();
00230         } else if (attr_name == "max_transition" || attr_name == "capacitance" ||
00231             attr_name == "rise_capacitance" || attr_name == "fall_capacitance" ||
00232             attr_name == "max_capacitance") {
00233             pin_json[attr_name] = lib_attr.getFloat();
00234         } else if (attr_name == "function" || attr_name == "power_down_function" ||
00235             attr_name == "related_ground_pin" || attr_name == "related_power_pin" ||
00236             attr_name == "three_state") {
00237             pin_json[attr_name] = lib_attr.getString();
00238         } else if (attr_name == "clock") {
00239             pin_json[attr_name] = lib_attr.getBoolean();
00240         }
00241         // More pin attributes can be added here
00242     }
00243
00244     GroupsIterator pin_sub_group_iter(lib_pin_group.getGroups(), err);
00245     for (; !pin_sub_group_iter.end(); pin_sub_group_iter.next()) {
00246         LibGroup lib_pin_sub_group = pin_sub_group_iter.get();
00247
00248         std::string pin_sub_group_type = lib_pin_sub_group.getType();
00249         // std::string pin_sub_group_name = lib_pin_sub_group.getName();
00250         if (pin_sub_group_type == "internal_power") {
00251             json power_json = generatePowerJson(lib_pin_sub_group, err);
00252             pin_json["power_arcs"].push_back(power_json);
00253         } else if (pin_sub_group_type == "timing") {
00254             // spdlog::debug("Has Timing: {}", lib_pin_group.getName());
00255             json timing_json = generateTimingJson(lib_pin_sub_group, err);
00256             pin_json["timing_arcs"].push_back(timing_json);
00257         } else {
00258             spdlog::warn("Unknown pin sub group type: {}", pin_sub_group_type);
00259         }
00260     }
00261     return std::make_pair(direction, pin_json);
00262 }
00263
00280 json generateCellJson(LibGroup &lib_cell_group, si2drErrorT &err) {
00281     json cell_json;
00282     cell_json["cell_name"] = lib_cell_group.getName();
00283
00284     AttributesIterator attr_iter(lib_cell_group.getAttrs(), err);
00285     for (; !attr_iter.end(); attr_iter.next()) {
00286         LibAttribute lib_attr = attr_iter.get();
00287
00288         std::string attr_name = lib_attr.getName();
00289         if (attr_name == "area" || attr_name == "cell_leakage_power") {
00290             cell_json[attr_name] = lib_attr.getFloat();
00291         } else if (attr_name == "cell_footprint") {
00292             cell_json[attr_name] = lib_attr.getString();
00293         }
00294         // More cell attributes can be added here
00295     }
00296
00297     GroupsIterator cell_sub_group_iter(lib_cell_group.getGroups(), err);
00298     for (; !cell_sub_group_iter.end(); cell_sub_group_iter.next()) {
00299         LibGroup lib_cell_sub_group = cell_sub_group_iter.get();
00300
00301         std::string cell_sub_group_type = lib_cell_sub_group.getType();
00302         // std::string cell_sub_group_name = lib_cell_sub_group.getName();
00303
00304         // spdlog::debug("Cell Sub Group Type: {}", cell_sub_group_type);
00305         // spdlog::debug("Cell Sub Group Name: {}", cell_sub_group_name);
00306
00307         if (cell_sub_group_type == "pin") {
00308             auto [direction, pin_json] = generatePinJson(lib_cell_sub_group, err);
00309             if (direction == "input") {
00310                 cell_json["input_pins"].push_back(pin_json);
00311             } else if (direction == "output") {
00312                 cell_json["output_pins"].push_back(pin_json);
00313             } else if (direction == "internal") {
00314                 cell_json["internal_pins"].push_back(pin_json);
00315             } else if (direction == "inout") {
00316                 cell_json["inout_pins"].push_back(pin_json);
00317             } else {

```

```

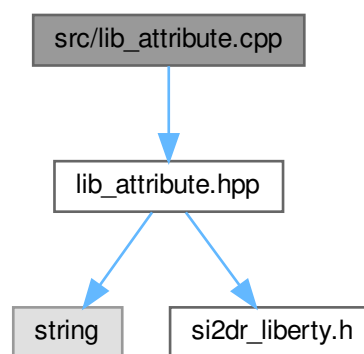
00318         spdlog::warn("Unknown direction: {}", direction);
00319     }
00320 }
00321 }
00322 return cell_json;
00323 }

```

6.26 src/lib_attribute.cpp File Reference

```
#include "lib_attribute.hpp"
```

Include dependency graph for lib_attribute.cpp:



6.27 lib_attribute.cpp

[Go to the documentation of this file.](#)

```

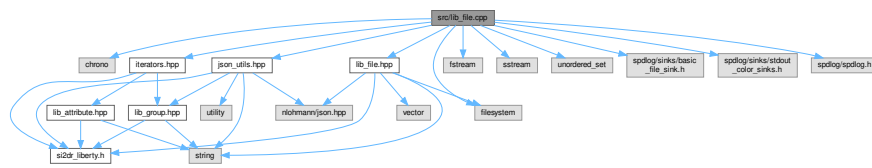
00001 #include "lib_attribute.hpp"
00002
00009 LibAttribute::LibAttribute(si2drAttrIdT attr, si2drErrorT &err) : attr_(attr), err_(err) {}
00010
00011 LibAttribute::~LibAttribute() {}
00012
00022 std::string LibAttribute::getName() {
00023     si2drStringT name = si2drAttrGetName(attr_, &err_);
00024     return name ? std::string(name) : std::string();
00025 }
00026
00032 bool LibAttribute::isComplex() { return si2drAttrGetAttrType(attr_, &err_) ? 1 : 0; }
00033
00042 si2drValuesIdT LibAttribute::getValues() { return si2drComplexAttrGetValues(attr_, &err_); }
00043
00053 long int LibAttribute::getInt() { return si2drSimpleAttrGetInt32Value(attr_, &err_); }
00054
00063 double LibAttribute::getFloat() { return si2drSimpleAttrGetFloat64Value(attr_, &err_); }
00064
00075 std::string LibAttribute::getString() {
00076     si2drStringT str = si2drSimpleAttrGetStringValue(attr_, &err_);
00077     return str ? std::string(str) : std::string();
00078 }
00079
00089 bool LibAttribute::getBoolean() { return si2drSimpleAttrGetBooleanValue(attr_, &err_) ? 1 : 0; }

```

6.28 src/lib_file.cpp File Reference

```
#include <chrono>
#include <filesystem>
#include <fstream>
#include <sstream>
#include <unordered_set>
#include "spdlog/sinks/basic_file_sink.h"
#include "spdlog/sinks/stdout_color_sinks.h"
#include "spdlog/spdlog.h"
#include "iterators.hpp"
#include "json_utils.hpp"
#include "lib_file.hpp"
#include "lib_file.cpp"
```

Include dependency graph for lib_file.cpp:



6.29 lib_file.cpp

[Go to the documentation of this file.](#)

```
00001 #include <chrono>
00002 #include <filesystem>
00003 #include <fstream>
00004 #include <sstream>
00005 #include <unordered_set>
00006
00007 #include "spdlog/sinks/basic_file_sink.h"
00008 #include "spdlog/sinks/stdout_color_sinks.h"
00009 #include "spdlog/spdlog.h"
00010
00011 #include "iterators.hpp"
00012 #include "json_utils.hpp"
00013 #include "lib_file.hpp"
00014
00015 LibFile::LibFile(const std::string &filepath, const std::string &loggername)
00016 : filepath_(filepath), loggername_(loggername) {
00017     filename_ = filepath_.string();
00018     basename_ = filepath_.stem().string();
00019     jsonname_ = basename_ + ".json";
00020
00021     auto console_sink = std::make_shared<spdlog::sinks::stdout_color_sink_mt>();
00022     console_sink->set_level(spdlog::level::info);
00023
00024     auto file_sink = std::make_shared<spdlog::sinks::basic_file_sink_mt>(loggername_, true);
00025     file_sink->set_level(spdlog::level::debug);
00026
00027     std::vector<spdlog::sink_ptr> sinks{console_sink, file_sink};
00028     logger_ = std::make_shared<spdlog::logger>(loggername_, sinks.begin(), sinks.end());
00029     logger_->set_level(spdlog::level::debug);
00030
00031     logger_->info("Created LibFile object for '{}'", filename_);
00032     logger_->info("Debug log in: '{}'", loggername_);
00033 }
00034
00035 LibFile::~LibFile() { logger_->info("Closing file: '{}'", filename_); }
00036
00047 void LibFile::writeJsonToFile() {
00048     std::ofstream out(jsonname_);
00049     if (!out.is_open()) {
00050         logger_->error("Could not open file '{}' for writing", jsonname_);
00051         return;
00052     }
```

```

00053 out « lib_json_.dump(2);
00054 out.close();
00055 logger->info("JSON data written to '{}'", jsonname_);
00056 }
00057
00078 void LibFile::read() {
00079     logger->info("Reading '{} ...', filename_");
00080
00081     auto start = std::chrono::high_resolution_clock::now();
00082     si2drReadLibertyFile(const_cast<char *>(filepath_.c_str()), &err_);
00083     auto end = std::chrono::high_resolution_clock::now();
00084     std::chrono::duration<double> duration = end - start;
00085
00086     if (err_ == SI2DR_INVALID_NAME) {
00087         logger->error("Could not open file '{}' for parsing, quitting...", filename_);
00088         exit(301);
00089     } else if (err_ == SI2DR_SYNTAX_ERROR) {
00090         logger->error("Syntax Errors were detected in the input file!");
00091         exit(401);
00092     } else {
00093         logger->info("Done. Read time: {:.2f} seconds", duration.count());
00094     }
00095 }
00096
00114 void LibFile::parse() {
00115     si2drPIInit(&err_); // Initialize private error handler
00116     this->read();        // Read the Liberty file
00117
00118     logger->info("Parsing '{} ...', filename_");
00119     // Create a scope for the top-level groups iteration
00120     {
00121         GroupsIterator group_iter(si2drPIGetGroups(&err_), err_);
00122         for (; !group_iter.end(); group_iter.next()) {
00123             LibGroup lib_group = group_iter.get();
00124
00125             if (lib_group.getName() != "") {
00126                 libname_ = lib_group.getName();
00127                 lib_json_["library_name"] = this->libname_;
00128                 logger->info("Library Name: {}", libname_);
00129             } else {
00130                 logger->warn("Library Name: <NONAME>");
00131             }
00132             // Second level groups
00133             GroupsIterator sub_group_iter(lib_group.getGroups(), err_);
00134             for (; !sub_group_iter.end(); sub_group_iter.next()) {
00135                 LibGroup lib_sub_group = sub_group_iter.get();
00136
00137                 std::string sub_group_type = lib_sub_group.getType();
00138                 std::string sub_group_name = lib_sub_group.getName();
00139
00140                 if (sub_group_type == "cell") {
00141                     logger->debug("Cell Name: {}", sub_group_name);
00142                     // if (sub_group_name != "AN2D0") {
00143                     //     continue;
00144                     // }
00145                     // Handle each cell
00146                     json cell_json = generateCellJson(lib_sub_group, err_);
00147                     lib_json_["cells"].push_back(cell_json);
00148                 } else if (sub_group_type == "operating_conditions") {
00149                     logger->info("Operating Conditions: {}", sub_group_name);
00150                     // Get PVT from Operating Conditions
00151                     AttributesIterator attr_iter(lib_sub_group.getAttrs(), err_);
00152                     for (; !attr_iter.end(); attr_iter.next()) {
00153                         LibAttribute lib_attr = attr_iter.get();
00154                         logger->debug("Attribute Name: {}", lib_attr.getName());
00155                         logger->debug("Int Value: {}", lib_attr.getInt());
00156                         logger->debug("Float Value: {}", lib_attr.getFloat());
00157                         logger->debug("String Value: {}", lib_attr.getString());
00158                         if (lib_attr.getName() == "process") {
00159                             process_ = lib_attr.getInt();
00160                             lib_json_["process"] = process_;
00161                         } else if (lib_attr.getName() == "voltage") {
00162                             voltage_ = lib_attr.getFloat();
00163                             // Round to 2 decimal places to avoid floating-point precision issues
00164                             lib_json_["voltage"] = std::round(voltage_ * 100) / 100.0;
00165                         } else if (lib_attr.getName() == "temperature") {
00166                             temperature_ = lib_attr.getInt();
00167                             lib_json_["temperature"] = temperature_;
00168                         }
00169                     }
00170                     logger->info("P: {}, V: {}, T: {}", process_, voltage_, temperature_);
00171                 }
00172             } // sub_group_iter's lifetime ends here, and the destructor is called
00173         }
00174     }
00175     // group_iter's lifetime ends here, and the destructor is called
00176 }

```

```

00177     si2drPIQuit(&err_);
00178 }
00179
00180 void LibFile::modify() { logger_>info("Modifying the file..."); }
00181
00208 bool LibFile::checkTimingArcMonotonicity(const json &cell, const json &pin, const json &arc,
00209                                           const std::string &timing_arc_name, const bool is_slew) {
00210     if (arc.contains("when") && !arc["when"].get<std::string>().empty()) {
00211         logger_>debug("Checking cell: '{}', pin: '{}', related_pin: '{}', timing_arc: '{}', when:
'{}'",
00212                      cell["cell_name"].get<std::string>(), pin["pin_name"].get<std::string>(),
00213                      arc["related_pin"].get<std::string>(), timing_arc_name,
00214                      arc["when"].get<std::string>());
00215     } else {
00216         logger_>debug("Checking cell: '{}', pin: '{}', related_pin: '{}', timing_arc: '{}'",
00217                      cell["cell_name"].get<std::string>(), pin["pin_name"].get<std::string>(),
00218                      arc["related_pin"].get<std::string>(), timing_arc_name);
00219     }
00220     bool is_monotonic = true; // Assume the values are monotonic initially
00221     if (arc.contains(timing_arc_name) && arc[timing_arc_name].contains("values")) {
00222         std::vector<std::vector<double>> value_matrix;
00223         // Generate a matrix of values from the JSON array
00224         for (const auto &row_val : arc[timing_arc_name]["values"]) {
00225             std::vector<double> row_data;
00226             // Check if the value is an array
00227             if (!row_val.is_array()) {
00228                 logger_>error("Invalid format: '{}' values should be an array of arrays in cell '{}', pin "
00229                               "'{}', related_pin '{}'",
00230                               timing_arc_name, cell["cell_name"].get<std::string>(),
00231                               pin["pin_name"].get<std::string>(), arc["related_pin"].get<std::string>());
00232                 return false;
00233             }
00234             // Check for non-numeric values
00235             for (const auto &val : row_val) {
00236                 if (val.is_number()) {
00237                     row_data.push_back(val.get<double>());
00238                 } else {
00239                     logger_>warn("Non-numeric value found in '{}' values, skipping value: {} in cell '{}', "
00240                                  "pin '{}', related_pin '{}'",
00241                                  timing_arc_name, val.dump(), cell["cell_name"].get<std::string>(),
00242                                  pin["pin_name"].get<std::string>(), arc["related_pin"].get<std::string>());
00243                 }
00244             }
00245             value_matrix.push_back(row_data);
00246         }
00247         // Check the matrix for monotonicity
00248         if (!value_matrix.empty() && !value_matrix[0].empty()) {
00249             // Check the monotonic incrementality of rows (by output load capacitance, index 2)
00250             for (size_t i = 0; i < value_matrix.size(); ++i) {
00251                 for (size_t j = 1; j < value_matrix[i].size(); ++j) {
00252                     if ((value_matrix[i][j] < value_matrix[i][j - 1] && pin["pin_name"] != arc["related_pin"])
00253 ||
00254                     (value_matrix[i][j] == 0 && value_matrix[i][j - 1] == 0)) {
00255                         // If contains "when" key, log the when value
00256                         if (arc.contains("when") && !arc["when"].get<std::string>().empty()) {
00257                             logger_>warn("Non-monotonic (by load) '{}' values: ({}, {}) {} < ({}, {}) {} "
00258                                           "for Cell: {} Pin: {}->{} when: \\\{}\\",
00259                                           timing_arc_name, i, j, value_matrix[i][j], i, j - 1, value_matrix[i][j -
1],
00260                                           cell["cell_name"].get<std::string>(),
00261                                           arc["related_pin"].get<std::string>(), pin["pin_name"].get<std::string>(), arc["when"].get<std::string>());
00262                         } else {
00263                             logger_>warn("Non-monotonic (by load) '{}' values: ({}, {}) {} < ({}, {}) {} "
00264                                           "for Cell: {} Pin: {}->{} when: \\\{}\\",
00265                                           timing_arc_name, i, j, value_matrix[i][j], i, j - 1, value_matrix[i][j -
1],
00266                                           cell["cell_name"].get<std::string>(),
00267                                           arc["related_pin"].get<std::string>(), pin["pin_name"].get<std::string>());
00268                         }
00269                         is_monotonic = false;
00270                     }
00271                 }
00272             }
00273             if (is_slew) {
00274                 // Check the monotonic incrementality of columns (by input slew, index 1)
00275                 for (size_t j = 0; j < value_matrix[0].size(); ++j) {
00276                     for (size_t i = 1; i < value_matrix.size(); ++i) {
00277                         if ((value_matrix[i][j] < value_matrix[i - 1][j] && pin["pin_name"] != arc["related_pin"])
00278 ||
00279                         (value_matrix[i][j] == 0 && value_matrix[i - 1][j] == 0)) {
00280                             // If contains "when" key, log the when value
00281                             if (arc.contains("when") && !arc["when"].get<std::string>().empty()) {
00282                                 logger_>warn("Non-monotonic (by slew) '{}' values: ({}, {}) {} < ({}, {}) {} "
00283                                               "for Cell: {} Pin: {}->{} when: \\\{}\\",
00284                                               timing_arc_name, i, j, value_matrix[i][j], i - 1, j,

```

```

00283         value_matrix[i - 1][j], cell["cell_name"].get<std::string>(),
00284         arc["related_pin"].get<std::string>(),
00285         pin["pin_name"].get<std::string>(),
00286         arc["when"].get<std::string>());
00287     } else {
00288         logger_>warn("Non-monotonic (by slew) '{}' values:  ({} , {}) {} < ({} , {}) {} "
00289         "for Cell: {} Pin: {}->{}",
00290         timing_arc_name, i, j, value_matrix[i][j], i - 1, j,
00291         value_matrix[i - 1][j], cell["cell_name"].get<std::string>(),
00292         arc["related_pin"].get<std::string>(),
00293         pin["pin_name"].get<std::string>());
00294     }
00295     is_monotonic = false;
00296 }
00297 }
00298 } else {
00299     logger_>warn(
00300     "Empty or invalid 'values' array found for cell: '{}', pin: '{}', related_pin: '{}', "
00301     "timing_arc: '{}'",
00302     cell["cell_name"].get<std::string>(), pin["pin_name"].get<std::string>(),
00303     arc["related_pin"].get<std::string>(), timing_arc_name);
00304 }
00305 }
00306 return is_monotonic;
00307 }
00308
00328 void LibFile::mono(const bool is_slew) {
00329     /*
00330     * Use this command to check the following data in the current library to ensure
00331     * the tables are monotonically increasing with respect to output load:
00332     * cell_rise retaining_rise
00333     * cell_fall retaining_fall
00334     * rise_transition retain_rise_slew
00335     * fall_transition retain_fall_slew
00336     * mpw
00337     */
00338     logger_>info("Monotonicity check of '{}' starting ...", filename_);
00339
00340     if (!std::filesystem::exists(jsonname_)) {
00341         logger_>info("JSON file not found. Parsing Liberty file first.");
00342         this->parse();
00343         this->writeJsonToFile();
00344     } else {
00345         // Read the JSON file into json object
00346         std::ifstream in(jsonname_);
00347         if (!in.is_open()) {
00348             logger_>error("Could not open file '{}' for reading", jsonname_);
00349             return;
00350         }
00351         try {
00352             lib_json_ = json::parse(in);
00353         } catch (const json::parse_error &e) {
00354             logger_>error("JSON parsing error in file '{}': {}", jsonname_, e.what());
00355             in.close();
00356             return;
00357         }
00358         in.close();
00359     }
00360
00361     std::map<std::string, bool> cell_monotonicity_status; // Track pass/fail status for each cell
00362     std::vector<std::string> failed_cells;               // List of failed cell names
00363     int total_cells = 0;
00364     int passed_cells = 0;
00365
00366     // Check the monotonicity of delay values
00367     // The index 1 (time values) must be monotonically increasing and >= 0
00368     for (const auto &cell : lib_json_["cells"]) {
00369         total_cells++;
00370         bool cell_is_monotonic = true; // Assume cell is monotonic initially
00371         std::string cell_name = cell["cell_name"].get<std::string>();
00372         cell_monotonicity_status[cell_name] =
00373             true; // Initialize to pass, will be set to false if any check fails
00374
00375         if (cell.contains("output_pins")) {
00376             for (const auto &pin : cell["output_pins"]) {
00377                 if (pin.contains("timing_arcs")) {
00378                     for (const auto &arc : pin["timing_arcs"]) {
00379                         // Check 4 timing arc names and accumulate monotonicity status
00380                         for (const auto &name : {"cell_rise", "cell_fall", "rise_transition", "fall_transition"})
00381                             if (!checkTimingArcMonotonicity(cell, pin, arc, name, is_slew)) {
00382                                 cell_is_monotonic = false;
00383                             }
00384                     }
00385                 }
00386             }
00387         }
00388     }

```

```

00386     }
00387 }
00388 }
00389 if (cell.contains("input_pins")) {
00390     for (const auto &pin : cell["input_pins"]) {
00391         if (!pin.contains("clock")) {
00392             continue; // Skip non-clock pins
00393         }
00394         // logger_>debug("Clock pin: {}", pin["pin_name"].get<std::string>());
00395         if (pin.contains("timing_arcs")) {
00396             for (const auto &arc : pin["timing_arcs"]) {
00397                 if (arc.contains("timing_type")) {
00398                     // logger_>debug("Timing type: {}", arc["timing_type"].get<std::string>());
00399                     if (arc["timing_type"].get<std::string>() == "min_pulse_width") {
00400                         for (const auto &name : {"rise_constraint", "fall_constraint"}) {
00401                             if (!checkTimingArcMonotonicity(cell, pin, arc, name, is_slew)) {
00402                                 cell_is_monotonic = false;
00403                             }
00404                         }
00405                     }
00406                 }
00407             }
00408         }
00409     }
00410 }
00411 // Update cell status based on all checks
00412 cell_monotonicity_status[cell_name] = cell_is_monotonic;
00413 if (cell_is_monotonic) {
00414     passed_cells++;
00415 } else {
00416     failed_cells.push_back(cell_name);
00417 }
00418 }
00419 logger_>info("Monotonicity check of '{}' completed.", filename_);
00420
00421 // Output summary statistics
00422 logger_>info("validate_monotonicity : {} out of {} cells passed", passed_cells, total_cells);
00423 logger_>info("validate_monotonicity : {} out of {} cells failed", total_cells - passed_cells,
00424             total_cells);
00425 if (!failed_cells.empty()) {
00426     std::stringstream failed_cells_ss;
00427     for (size_t i = 0; i < failed_cells.size(); ++i) {
00428         failed_cells_ss << failed_cells[i];
00429         if (i < failed_cells.size() - 1) {
00430             failed_cells_ss << ", "; // Add comma if not the last element
00431         }
00432     }
00433     logger_>info("Failed cell list : {}", failed_cells_ss.str());
00434 }
00435 }
00436
00455 void LibFile::supercell(const int chain_length, const std::vector<std::string> &cell_names) {
00456     /*
00457     * Supercells are named as follows:
00458     * <cellname>_X<chain_length>_<input_pin>_<output_pin>
00459     */
00460     logger_>info("Creating supercells for '{}'", filename_);
00461
00462     if (!std::filesystem::exists(jsonname_)) {
00463         logger_>info("JSON file not found. Parsing Liberty file first.");
00464         this->parse();
00465         this->writeJsonToFile();
00466     } else {
00467         // Read the JSON file into json object
00468         std::ifstream in(jsonname_);
00469         if (!in.is_open()) {
00470             logger_>error("Could not open file '{}' for reading", jsonname_);
00471             return;
00472         }
00473         try {
00474             lib_json_ = json::parse(in);
00475         } catch (const json::parse_error &e) {
00476             logger_>error("JSON parsing error in file '{}': {}", jsonname_, e.what());
00477             in.close();
00478             return;
00479         }
00480         in.close();
00481     }
00482
00483     // write to .map file
00484     std::ofstream out(basename_ + ".map");
00485     if (!out.is_open()) {
00486         logger_>error("Could not open file '{}' for writing", basename_ + ".map");
00487         return;
00488     }
00489     // if cell_names is empty, create supercells for all cells
00490     // else create supercells for only the specified cells

```

```

00491
00492 // Check if we're processing specific cells or all cells
00493 bool process_all_cells = cell_names.empty();
00494
00495 if (process_all_cells) {
00496     logger_>info("Creating supercells for ALL cells in '{}'", filename_);
00497 } else {
00498     logger_>info("Creating supercells for {} specified cells in '{}'", cell_names.size(), filename_);
00499 }
00500
00501 // Create a set for faster lookups if we have specific cell names
00502 std::unordered_set<std::string> cell_set;
00503 std::unordered_set<std::string> found_cells;
00504 if (!process_all_cells) {
00505     cell_set.insert(cell_names.begin(), cell_names.end());
00506 }
00507
00508 for (const auto &cell : lib_json_["cells"]) {
00509     bool is_sequential = false;
00510     std::string cell_name = cell["cell_name"].get<std::string>();
00511
00512     // Skip cells not in the specified list
00513     if (!process_all_cells && cell_set.find(cell_name) == cell_set.end()) {
00514         continue;
00515     }
00516
00517     // Mark this cell as found
00518     if (!process_all_cells) {
00519         found_cells.insert(cell_name);
00520     }
00521
00522     logger_>debug("Creating supercells for cell: '{}'", cell_name);
00523
00524     std::unordered_set<std::string> output_pins;
00525     std::unordered_set<std::string> input_pins;
00526     // Extract input and output pins
00527     if (cell.contains("output_pins")) {
00528         for (const auto &pin : cell["output_pins"]) {
00529             output_pins.insert(pin["pin_name"].get<std::string>());
00530         }
00531     }
00532     if (cell.contains("input_pins")) {
00533         for (const auto &pin : cell["input_pins"]) {
00534             if (pin.contains("clock")) {
00535                 is_sequential = true;
00536                 // clock pin not use for supercell
00537                 continue;
00538             }
00539             input_pins.insert(pin["pin_name"].get<std::string>());
00540         }
00541     }
00542
00543     // create supercells for all combinations of input and output pins
00544     for (const auto &output_pin : output_pins) {
00545         for (const auto &input_pin : input_pins) {
00546             if (is_sequential) {
00547                 const int chain_len = 1;
00548                 std::string supercell_name =
00549                     cell_name + "__X" + std::to_string(chain_len) + "__" + input_pin + "__" + output_pin;
00550                 out << cell_name << " " << supercell_name << std::endl;
00551             } else {
00552                 std::string supercell_name =
00553                     cell_name + "__X" + std::to_string(chain_length) + "__" + input_pin + "__" + output_pin;
00554                 out << cell_name << " " << supercell_name << std::endl;
00555             }
00556         }
00557     }
00558 }
00559
00560 // Check for cells that were specified but not found
00561 if (!process_all_cells) {
00562     std::vector<std::string> not_found_cells;
00563     for (const auto &requested_cell : cell_names) {
00564         if (found_cells.find(requested_cell) == found_cells.end()) {
00565             not_found_cells.push_back(requested_cell);
00566         }
00567     }
00568
00569     // Output warning for cells that weren't found
00570     if (!not_found_cells.empty()) {
00571         std::string missing_cells = not_found_cells[0];
00572         for (size_t i = 1; i < not_found_cells.size(); ++i) {
00573             missing_cells += ", " + not_found_cells[i];
00574         }
00575         logger_>warn("{} specified cell{} not found in the library: {}", not_found_cells.size(),
00576             not_found_cells.size() > 1 ? "s were" : " was", missing_cells);
00577     }

```

```

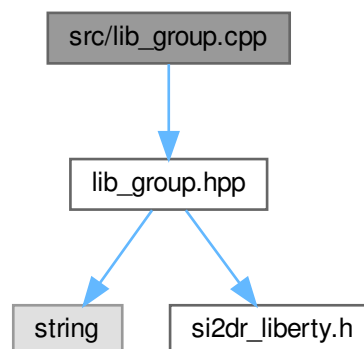
00578     }
00579
00580     out.close();
00581     logger_>info("Supercell creation complete in '{}'", basename_ + ".map");
00582 }
00583
00584 void LibFile::verilog(const int chain_length, const std::vector<std::string> &cell_names) {
00585     logger_>info("Creating Verilog for '{}'", filename_);
00586
00587     if (!std::filesystem::exists(jsonname_)) {
00588         logger_>info("JSON file not found. Parsing Liberty file first.");
00589         this->parse();
00590         this->writeJsonToFile();
00591     } else {
00592         // Read the JSON file into json object
00593         std::ifstream in(jsonname_);
00594         if (!in.is_open()) {
00595             logger_>error("Could not open file '{}' for reading", jsonname_);
00596             return;
00597         }
00598         try {
00599             lib_json_ = json::parse(in);
00600         } catch (const json::parse_error &e) {
00601             logger_>error("JSON parsing error in file '{}': {}", jsonname_, e.what());
00602             in.close();
00603             return;
00604         }
00605         in.close();
00606     }
00607
00608     // write to .v file
00609     std::ofstream out(basename_ + ".v");
00610     if (!out.is_open()) {
00611         logger_>error("Could not open file '{}' for writing", basename_ + ".v");
00612         return;
00613     }
00614     out << "timescale 1ns/10ps" << std::endl;
00615     out.close();
00616     logger_>info("Verilog creation complete in '{}'", basename_ + ".v");
00617 }

```

6.30 src/lib_group.cpp File Reference

#include "lib_group.hpp"

Include dependency graph for lib_group.cpp:



6.31 lib_group.cpp

[Go to the documentation of this file.](#)

```

00001 #include "lib_group.hpp"
00002
00011 LibGroup::LibGroup(si2drGroupIdT group, si2drErrorT &err) : group_(group), err_(err) {}
00012
00013 LibGroup::~LibGroup() {}
00014
00023 std::string LibGroup::getName() {
00024     si2drNamesIdT names = si2drGroupGetNames(group_, &err_);
00025     si2drStringT name = si2drIterNextName(names, &err_);
00026     si2drIterQuit(names, &err_);
00027     return name ? std::string(name) : std::string();
00028 }
00029
00038 std::string LibGroup::getType() {
00039     si2drStringT type = si2drGroupGetGroupType(group_, &err_);
00040     return type ? std::string(type) : std::string();
00041 }
00042
00051 si2drAttrsIdT LibGroup::getAttrs() { return si2drGroupGetAttrs(group_, &err_); }
00052
00061 si2drGroupsIdT LibGroup::getGroups() { return si2drGroupGetGroups(group_, &err_); }

```

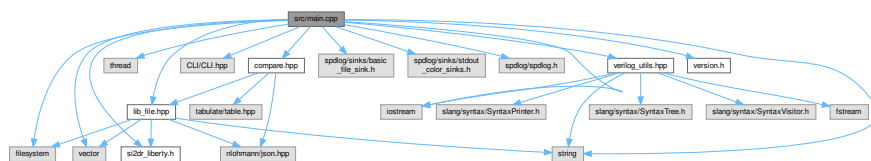
6.32 src/main.cpp File Reference

```

#include <filesystem>
#include <iostream>
#include <string>
#include <thread>
#include <vector>
#include "CLI/CLI.hpp"
#include "si2dr_liberty.h"
#include "spdlog/sinks/basic_file_sink.h"
#include "spdlog/sinks/stdout_color_sinks.h"
#include "spdlog/spdlog.h"
#include "compare.hpp"
#include "lib_file.hpp"
#include "version.h"
#include "verilog_utils.hpp"

```

Include dependency graph for main.cpp:



Functions

- void [printInfo](#) ()
Initializes and sets up the global logger with both console and file sinks.
- void [parseLibFile](#) (const std::string &library_path, const std::string log_file_name)
Parses a library file and writes the parsed data to a JSON file.
- void [monoCheckLibFile](#) (const std::string &library_path, const std::string log_file_name, bool is_slew)
Performs a monotonicity check on a given library file.
- void [supercellLibFile](#) (const std::string &library_path, const std::string &log_file_name, int chain_length, const std::vector< std::string > &cell_names)
Creates supercell variants of specified cells in a library file.

- void `verilogLibFile` (const std::string &library_path, const std::string &log_file_name, int chain_length, const std::vector< std::string > &cell_names)
- void `compareLibFiles` (const std::string &ref_lib, const std::string &comp_lib, const double reltol, const double abstol, std::string &report_file_name)
Compare two library files and generate a comparison report.
- void `funcLibFile` (const std::string &ref_file, const std::string &comp_file, const std::vector< std::string > &cell_names)
- int `main` (int argc, char *argv[])

6.32.1 Function Documentation

6.32.1.1 `compareLibFiles()`

```
void compareLibFiles (
    const std::string & ref_lib,
    const std::string & comp_lib,
    const double reltol,
    const double abstol,
    std::string & report_file_name )
```

Compare two library files and generate a comparison report.

This function compares two library files (reference and comparison) based on provided tolerance values and generates a detailed comparison report.

Parameters

<i>ref_lib</i>	Path to the reference library file
<i>comp_lib</i>	Path to the comparison library file
<i>reltol</i>	Relative tolerance for comparison (must be ≥ 0.0)
<i>abstol</i>	Absolute tolerance for comparison
<i>report_file_name</i>	Name of the output report file (will be modified if empty or has an invalid extension) If empty, defaults to [comp_lib_basename].cmp.md If doesn't end with .txt or .md, .md will be appended

Returns

void The function returns no value but generates a report file with the comparison results

Exceptions

No	exceptions are thrown, but errors are logged if relative tolerance is invalid
----	---

Note

Log files are created for both the reference and comparison libraries with the naming pattern: [lib_↔
 basename].cmp.log

Definition at line 202 of file [main.cpp](#).

Here is the call graph for this function:



Here is the caller graph for this function:



6.32.1.2 funcLibFile()

```

void funcLibFile (
    const std::string & ref_file,
    const std::string & comp_file,
    const std::vector< std::string > & cell_names )
  
```

Definition at line 228 of file [main.cpp](#).

Here is the call graph for this function:

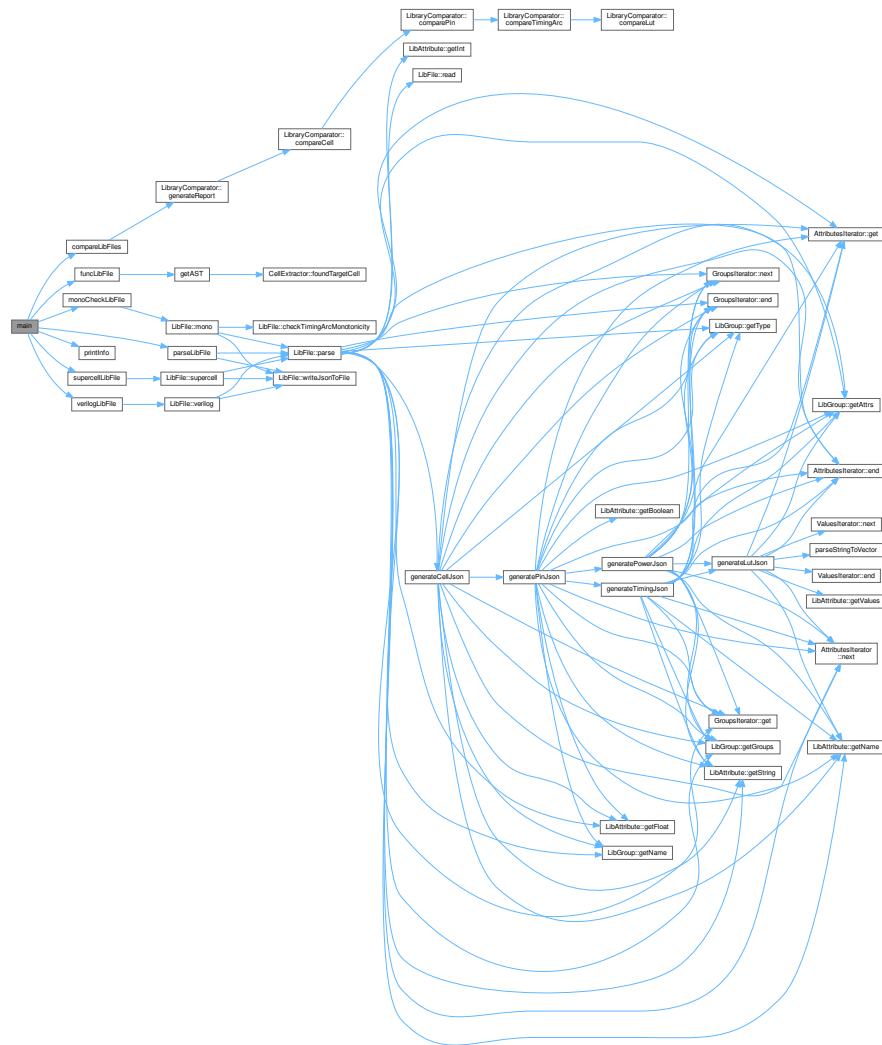


Here is the caller graph for this function:



```
int main (
    int argc,
    char * argv[ ] )
```

Here is the call graph for this function:



```
void monoCheckLibFile (
    const std::string & library_path,
    const std::string log_file_name,
    bool is_slew )
```

This function checks the monotonicity of the specified library file and logs the process. It uses the provided log file name or generates one based on the library file name if not provided.

Here is the caller graph for this function:



6.32.1.5 parseLibFile()

```
void parseLibFile (
    const std::string & library_path,
    const std::string log_file_name )
```

Parses a library file and writes the parsed data to a JSON file.

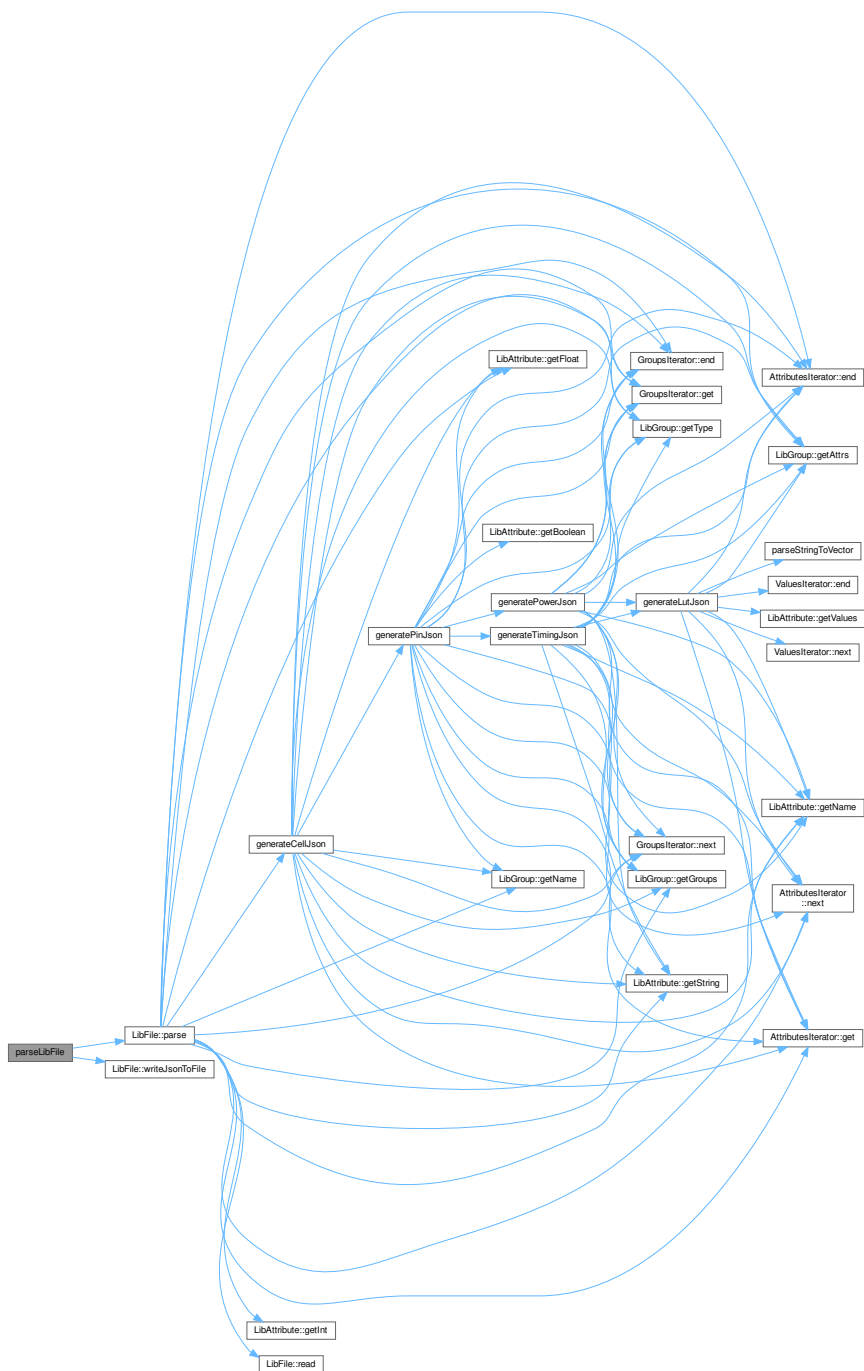
This function takes the path to a library file and an optional log file name. It creates a [LibFile](#) object, logs the start of the parsing process, and attempts to parse the file. If parsing is successful, the parsed data is written to a JSON file. If an error occurs during parsing, an error message is logged.

Parameters

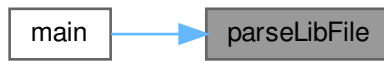
<i>library_path</i>	The path to the library file to be parsed.
<i>log_file_name</i>	The name of the log file. If empty, a default log file name is generated based on the library file name.

Definition at line 67 of file [main.cpp](#).

Here is the call graph for this function:



Here is the caller graph for this function:



6.32.1.6 printInfo()

```
void printInfo ( )
```

Initializes and sets up the global logger with both console and file sinks.

This function configures the global logger to output log messages to both the console and a log file. The log file is named after the application name with a ".log" extension. The console sink is set to display messages at the info level, while the file sink captures messages at the debug level. The logger is then set to the info level.

The function also logs the application version, build timestamp, author, and contact information.

Note

The following macros are expected to be defined:

- APP_NAME: The name of the application.
- APP_VERSION: The version of the application.
- BUILD_TIMESTAMP: The build timestamp of the application.
- APP_AUTHOR: The author of the application.
- APP_CONTACT: The contact email of the author.

Definition at line 36 of file [main.cpp](#).

Here is the caller graph for this function:



6.32.1.7 supercellLibFile()

```
void supercellLibFile (
    const std::string & library_path,
    const std::string & log_file_name,
    int chain_length,
    const std::vector< std::string > & cell_names )
```

Creates supercell variants of specified cells in a library file.

This function opens a library file and generates supercell variants for the specified cells by repeating each cell a specified number of times in a chain configuration. Operations and results are logged to a specified or auto-generated log file.

Parameters

<i>library_path</i>	Path to the library file to process
<i>log_file_name</i>	Path to the log file (if empty, a default name will be generated)
<i>chain_length</i>	Number of times to repeat each cell (must be ≥ 1)
<i>cell_names</i>	Vector of cell names to process for supercell generation

Exceptions

<i>Exceptions</i>	from the LibFile::supercell method are caught and logged
-------------------	--

Note

If chain_length is less than 1, the function will log an error and return without processing

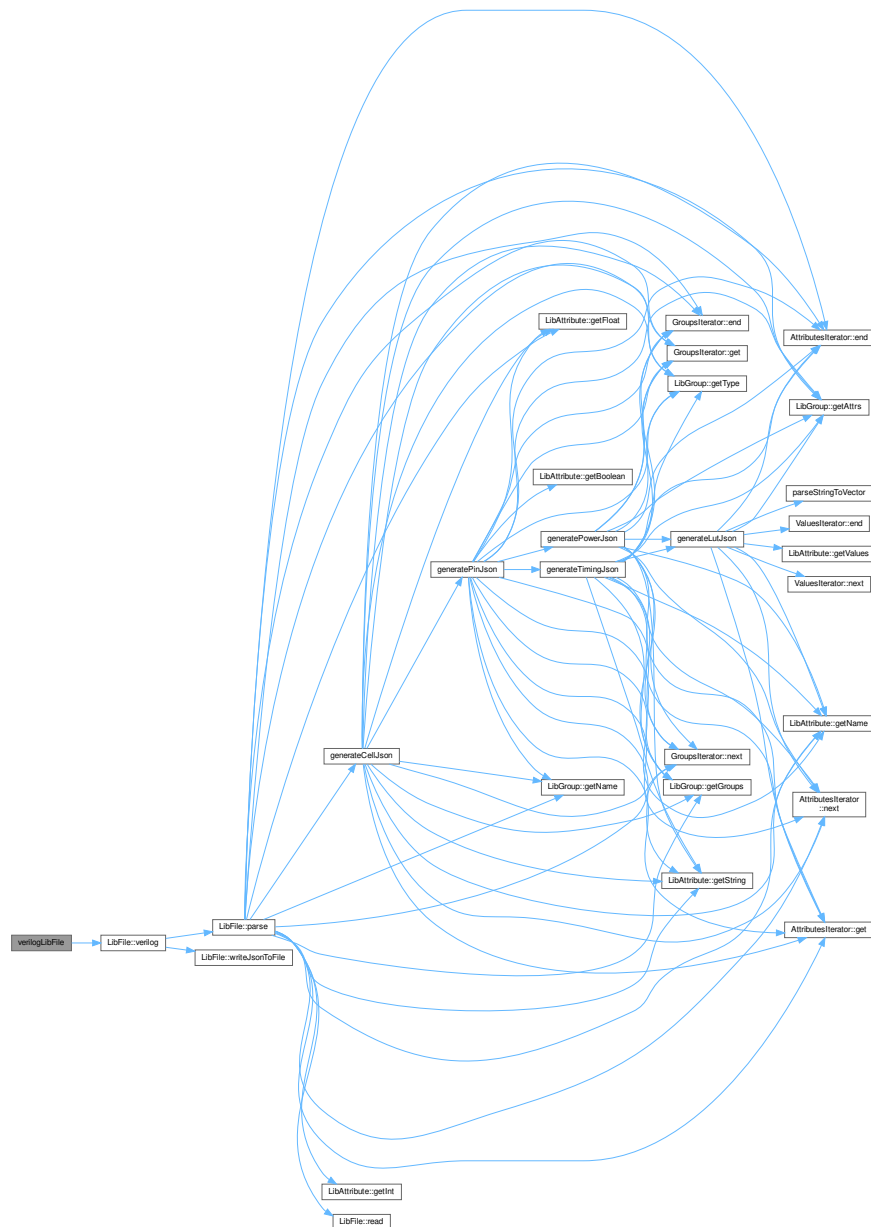
Definition at line 128 of file [main.cpp](#).

6.32.1.8 verilogLibFile()

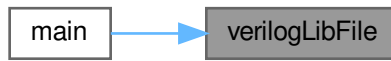
```
void verilogLibFile (
    const std::string & library_path,
    const std::string & log_file_name,
    int chain_length,
    const std::vector< std::string > & cell_names )
```

Definition at line 155 of file [main.cpp](#).

Here is the call graph for this function:



Here is the caller graph for this function:



6.33 main.cpp

[Go to the documentation of this file.](#)

```

00001 // ./src/main.cpp
00002 #include <filesystem>
00003 #include <iostream>
00004 #include <string>
00005 #include <thread>
00006 #include <vector>
00007
00008 #include "CLI/CLI.hpp"
00009 #include "si2dr_liberty.h"
00010 #include "spdlog/sinks/basic_file_sink.h"
00011 #include "spdlog/sinks/stdout_color_sinks.h"
00012 #include "spdlog/spdlog.h"
00013
00014 #include "compare.hpp"
00015 #include "lib_file.hpp"
00016 #include "version.h"
00017 #include "verilog_utils.hpp"
00018
00036 void printInfo() {
00037     // Set Global Logger
00038     auto console_sink = std::make_shared<spdlog::sinks::stdout_color_sink_mt>();
00039     console_sink->set_level(spdlog::level::info);
00040     std::string app_name = APP_NAME;
00041     auto file_sink = std::make_shared<spdlog::sinks::basic_file_sink_mt>(app_name + ".log", true);
00042     file_sink->set_level(spdlog::level::debug);
00043
00044     std::vector<spdlog::sink_ptr> sinks{console_sink, file_sink};
00045     auto logger = std::make_shared<spdlog::logger>(APP_NAME, sinks.begin(), sinks.end());
00046     logger->set_level(spdlog::level::debug);
00047     spdlog::set_default_logger(logger);
00048
00049     spdlog::info("Version {}, Built: {}", APP_VERSION, BUILD_TIMESTAMP);
00050     spdlog::info("Author: {}, Email: {}", APP_AUTHOR, APP_CONTACT);
00051     spdlog::info("Global log file in: '{}', app_name + ".log");
00052 }
00053
00067 void parseLibFile(const std::string &library_path, const std::string log_file_name) {
00068     std::string logname = log_file_name.empty()
00069         ? std::filesystem::path(library_path).stem().string() + ".parse.log"
00070         : log_file_name;
00071     LibFile libfile(library_path, logname);
00072
00073     libfile.logger_->info("Starting parse for file: '{} ...", library_path);
00074
00075     try {
00076         libfile.parse();
00077         libfile.writeJsonToFile();
00078         libfile.logger_->info("Successfully parsed file: '{}", library_path);
00079     } catch (const std::exception &e) {
00080         libfile.logger_->error("Error parsing file '{}': {}", library_path, e.what());
00081     }
00082 }
00083
00095 void monoCheckLibFile(const std::string &library_path, const std::string log_file_name, bool is_slew)
00096 {
00097     std::string logname = log_file_name.empty()
00098         ? std::filesystem::path(library_path).stem().string() + ".mono.log"
00099         : log_file_name;
00100     LibFile libfile(library_path, logname);
00101
00101     libfile.logger_->info("Starting monotonicity check for file: '{}', input slew: {}", library_path,

```

```

00102         is_slew);
00103
00104     try {
00105         libfile.mono(is_slew);
00106         libfile.logger_>info("Successfully completed monotonicity check for file: '{}'", library_path);
00107     } catch (const std::exception &e) {
00108         libfile.logger_>error("Error during monotonicity check for file '{}': {}", library_path,
00109             e.what());
00110     }
00111 }
00112
00128 void supercellLibFile(const std::string &library_path, const std::string &log_file_name,
00129     int chain_length, const std::vector<std::string> &cell_names) {
00130     std::string logname = log_file_name.empty()
00131         ? std::filesystem::path(library_path).stem().string() + ".supercell.log"
00132         : log_file_name;
00133     LibFile libfile(library_path, logname);
00134
00135     // Check chain length validity
00136     if (chain_length < 1) {
00137         libfile.logger_>error("Invalid chain length: {}. Chain length must be >= 1.", chain_length);
00138         return;
00139     }
00140     std::stringstream ss;
00141     for (const auto &cell : cell_names) {
00142         ss << cell << ", ";
00143     }
00144     libfile.logger_>info("Starting supercell generation for file: '{}', chain length: {}, cells:
00145 {}\"", library_path,
00146         chain_length, ss.str());
00147     try {
00148         libfile.supercell(chain_length, cell_names);
00149         libfile.logger_>info("Successfully generated supercells for file: '{}'", library_path);
00150     } catch (const std::exception &e) {
00151         libfile.logger_>error("Error during supercell generation for file '{}': {}", library_path,
00152             e.what());
00153     }
00154 }
00155 void verilogLibFile(const std::string &library_path, const std::string &log_file_name, int
00156     chain_length,
00157     const std::vector<std::string> &cell_names) {
00158     std::string logname = log_file_name.empty()
00159         ? std::filesystem::path(library_path).stem().string() + ".verilog.log"
00160         : log_file_name;
00161     LibFile libfile(library_path, logname);
00162
00163     // Check chain length validity
00164     if (chain_length < 1) {
00165         libfile.logger_>error("Invalid chain length: {}. Chain length must be >= 1.", chain_length);
00166         return;
00167     }
00168     std::stringstream ss;
00169     for (const auto &cell : cell_names) {
00170         ss << cell << ", ";
00171     }
00172     libfile.logger_>info("Starting Verilog generation for file: '{}', chain length: {}, cells: {}",
00173         library_path,
00174         chain_length, ss.str());
00175     try {
00176         libfile.verilog(chain_length, cell_names);
00177         libfile.logger_>info("Successfully generated Verilog for file: '{}'", library_path);
00178     } catch (const std::exception &e) {
00179         libfile.logger_>error("Error during Verilog generation for file '{}': {}", library_path,
00180             e.what());
00181     }
00182 }
00183
00202 void compareLibFiles(const std::string &ref_lib, const std::string &comp_lib, const double reltol,
00203     const double abstol, std::string &report_file_name) {
00204     std::string ref_logname = std::filesystem::path(ref_lib).stem().string() + ".cmp.log";
00205     std::string comp_logname = std::filesystem::path(comp_lib).stem().string() + ".cmp.log";
00206     LibFile ref_libfile(ref_lib, ref_logname), comp_libfile(comp_lib, comp_logname);
00207
00208     // Check relative tolerance validity
00209     if (reltol < 0.0) {
00210         spdlog::error("Invalid relative tolerance: {}. Relative tolerance must be >= 0.0.", reltol);
00211         return;
00212     }
00213     if (report_file_name.empty()) {
00214         report_file_name = comp_libfile.basename_ + ".cmp.md";
00215     } else if (std::filesystem::path(report_file_name).extension() != ".txt" &&
00216         std::filesystem::path(report_file_name).extension() != ".md") {
00217         // report file name not end in .txt or .md, warn user and append .md
00218         report_file_name = report_file_name + ".md";
00219     }
00220     spdlog::warn("Report file name does not end in .txt or .md. Report will be written to '{}'",
00221         report_file_name);

```

```

00221     }
00222
00223     LibraryComparator comparator(ref_libfile, comp_libfile, reltol, abstol);
00224     comparator.generateReport(report_file_name);
00225     spdlog::info("Comparison completed. Report written to: '{}'", report_file_name);
00226 }
00227
00228 void funcLibFile(const std::string &ref_file, const std::string &comp_file,
00229                 const std::vector<std::string> &cell_names) {
00230     // Check if the files are Liberty or Verilog
00231     std::string ref_ext = std::filesystem::path(ref_file).extension();
00232     std::string comp_ext = std::filesystem::path(comp_file).extension();
00233     for (const auto &cell : cell_names) {
00234         spdlog::info("Checking functional equivalence for cell: '{}'", cell);
00235         if (ref_ext == ".v") {
00236             spdlog::info("Reference file in Verilog format.");
00237             getAST(ref_file, cell);
00238         }
00239         if (comp_ext == ".v") {
00240             spdlog::info("Comparison file in Verilog format.");
00241             getAST(comp_file, cell);
00242         }
00243     }
00244 }
00245
00246 int main(int argc, char *argv[]) {
00247     // Parse command line arguments
00248     std::vector<std::string> library_paths; // Support multiple files
00249     std::string log_file_name = "";
00250
00251     // Command line parameter parsing using CLI11
00252     CLI::App app{APP_NAME};
00253     app.set_version_flag("-v,--version", APP_VERSION);
00254
00255     // Add subcommand for parse mode
00256     CLI::App *parse_cmd = app.add_subcommand("parse", "Parse the Liberty file and write JSON to a
file");
00257     parse_cmd->add_option("library_path", library_paths, "Specify the library file to process")
00258         ->check(CLI::ExistingFile)
00259         ->required();
00260     parse_cmd->add_option("-l,--log", log_file_name,
00261         "Specify the log file name. Default: <basename>.parse.log");
00262     parse_cmd->callback([&] {
00263         printInfo();
00264         // Check if multi files
00265         if (library_paths.size() > 1) {
00266             spdlog::info("Running sequential parsing for {} files.", library_paths.size());
00267             spdlog::info("Each library will write to its own log file.");
00268             // Sequential parsing
00269             for (const auto &library_path : library_paths) {
00270                 parseLibFile(library_path, log_file_name = "");
00271             }
00272         } else {
00273             parseLibFile(library_paths[0], log_file_name);
00274         }
00275     });
00276
00277     // Add subcommand for mono check mode
00278     bool is_slew = false;
00279     CLI::App *mono_cmd = app.add_subcommand("mono", "Check the monotonicity of timing arc values");
00280     mono_cmd->add_option("library_path", library_paths, "Specify the library file to process")
00281         ->check(CLI::ExistingFile)
00282         ->required();
00283     mono_cmd->add_option("-l,--log", log_file_name,
00284         "Specify the log file name. Default: <basename>.mono.log");
00285     mono_cmd->add_flag("-s,--slew", is_slew, "Specify that monotonicity checks also include input
slew.");
00286     mono_cmd->callback([&] {
00287         printInfo();
00288         // Check if multi files
00289         if (library_paths.size() > 1) {
00290             spdlog::info("Running multi-threaded monotonicity check for {} files.", library_paths.size());
00291             spdlog::info("Each thread will write to its own log file.");
00292
00293             // Sequential json file check
00294             for (const auto &library_path : library_paths) {
00295                 if (!std::filesystem::exists(std::filesystem::path(library_path).stem().string() + ".json")) {
00296                     spdlog::info("JSON file not found for '{}'. Parsing Liberty file first.", library_path);
00297                     parseLibFile(library_path, log_file_name = "");
00298                 }
00299             }
00300             spdlog::info("All JSON files prepared.");
00301             // Parallel monotonicity check
00302             std::vector<std::thread> threads;
00303             for (const auto &library_path : library_paths) {
00304                 threads.emplace_back(monoCheckLibFile, library_path, log_file_name = "", is_slew);
00305             }

```

```

00306         for (auto &thread : threads) {
00307             thread.join();
00308         }
00309         spdlog::info("All threads completed.");
00310     } else {
00311         monoCheckLibFile(library_paths[0], log_file_name, is_slew);
00312     }
00313 });
00314
00315 // Add subcommand for compare mode
00316 std::string ref_lib, comp_lib, report_file_name;
00317 CLI::App *compare_cmd = app.add_subcommand(
00318     "compare", "Compare the comparison library against the reference library and report
differences");
00319 compare_cmd->add_option("--ref", ref_lib, "Specify the reference library file")
00320     ->check(CLI::ExistingFile)
00321     ->required();
00322 compare_cmd->add_option("--comp", comp_lib, "Specify the comparison library file")
00323     ->check(CLI::ExistingFile)
00324     ->required();
00325 double abstol = 0.002;
00326 compare_cmd->add_option("--abstol", abstol,
00327     "Specify the absolute tolerance for comparison. Default: 0.002ns");
00328 double reltol = 0.02;
00329 compare_cmd->add_option("--reltol", reltol,
00330     "Specify the relative tolerance for comparison. Default: 0.02/2.0%");
00331 compare_cmd->add_option("--report", report_file_name,
00332     "Specify the report file name. Default: <comp_lib>.cmp.md");
00333 compare_cmd->callback([&] {
00334     printInfo();
00335     compareLibFiles(ref_lib, comp_lib, reltol, abstol, report_file_name);
00336 });
00337
00338 // Add subcommand for supercell generation
00339 int chain_length = 1;
00340 CLI::App *supercell_cmd =
00341     app.add_subcommand("supercell", "Generate supercells for the given Liberty file");
00342 supercell_cmd->add_option("library_path", library_paths, "Specify the library file to process")
00343     ->check(CLI::ExistingFile)
00344     ->required();
00345 supercell_cmd->add_option("-l,--log", log_file_name,
00346     "Specify the log file name. Default: <basename>.supercell.log");
00347 supercell_cmd->add_option("-c,--chain", chain_length,
00348     "Specify the chain length for supercell generation. Default: 1");
00349 std::vector<std::string> cell_names = {}; // "CMPE42D1" "AN2D0", "DFQD1"
00350 supercell_cmd->add_option("--cells", cell_names, "Specify the cell names to generate supercells
for");
00351 supercell_cmd->callback([&] {
00352     printInfo();
00353     // Check if multi files
00354     if (library_paths.size() > 1) {
00355         spdlog::info("Running multi-threaded supercell generation for {} files.", library_paths.size());
00356         spdlog::info("Each library will write to its own log file.");
00357     }
00358     // Sequential json file check
00359     for (const auto &library_path : library_paths) {
00360         if (!std::filesystem::exists(std::filesystem::path(library_path).stem().string() + ".json")) {
00361             spdlog::info("JSON file not found for '{}'. Parsing Liberty file first.", library_path);
00362             parseLibFile(library_path, log_file_name = "");
00363         }
00364     }
00365     spdlog::info("All JSON files prepared.");
00366     // Parallel supercell generation
00367     std::vector<std::thread> threads;
00368     for (const auto &library_path : library_paths) {
00369         threads.emplace_back(supercellLibFile, library_path, log_file_name = "", chain_length,
cell_names);
00370     }
00371     for (auto &thread : threads) {
00372         thread.join();
00373     }
00374     spdlog::info("All threads completed.");
00375 } else {
00376     supercellLibFile(library_paths[0], log_file_name, chain_length, cell_names);
00377 }
00378 });
00379
00380 // Add subcommand for zlibboost
00381 CLI::App *zlibboost_cmd =
00382     app.add_subcommand("zlibboost", "ZlibBoost - Multi-threaded Library Processing Tool");
00383 std::string config_dir = "/home/songzx/Projects/zlibboost/config.tcl";
00384 std::string python_dir = "/home/guocj/anaconda3/envs/myenv/bin/python";
00385 std::string main_py_dir = "/home/songzx/Projects/zlibboost/zlibboost.py";
00386 zlibboost_cmd->add_option(
00387     "-c, --config", config_dir,
00388     "Specify the configuration TCL file. Default: /home/songzx/Projects/zlibboost/config.tcl");
00389 zlibboost_cmd->add_option(

```

```

00390     "--python", python_dir,
00391     "Specify the python directory. Default: /home/guocj/anaconda3/envs/myenv/bin/python");
00392     zlibboost_cmd->add_option(
00393         "--main", main_py_dir,
00394         "Specify the main python script directory. Default:
/home/songzx/Projects/zlibboost/zlibboost.py");
00395     zlibboost_cmd->callback([&] {
00396         printInfo();
00397         // Run the ZlibBoost tool
00398         std::string command = python_dir + " " + main_py_dir + " -c " + config_dir;
00399         spdlog::info("Running ZlibBoost with command: '{}'", command);
00400         int ret = std::system(command.c_str());
00401         if (ret == 0) {
00402             spdlog::info("ZlibBoost completed successfully.");
00403         } else {
00404             spdlog::error("ZlibBoost failed with return code: {}", ret);
00405         }
00406     });
00407
00408     // Add subcommand for clear
00409     CLI::App *clear_cmd =
00410         app.add_subcommand("clear", "Clear the log, JSON, map, markdown files in this directory");
00411     clear_cmd->callback([&] {
00412         printInfo();
00413         std::filesystem::path current_dir = std::filesystem::current_path();
00414         for (const auto &entry : std::filesystem::directory_iterator(current_dir)) {
00415             if (entry.path().extension() == ".log" || entry.path().extension() == ".json" ||
00416                 entry.path().extension() == ".map" || entry.path().extension() == ".md") {
00417                 spdlog::info("Removing file: '{}'", entry.path().string());
00418                 std::filesystem::remove(entry.path());
00419             }
00420         }
00421         spdlog::info("All log, JSON, map, markdown files cleared.");
00422     });
00423
00424     // Add subcommand for Verilog generation
00425     CLI::App *verilog_cmd = app.add_subcommand("verilog", "Generate Verilog file for given Liberty
file");
00426     verilog_cmd->add_option("library_path", library_paths, "Specify the library file to process")
00427         ->check(CLI::ExistingFile)
00428         ->required();
00429     verilog_cmd->add_option("-l,--log", log_file_name,
00430         "Specify the log file name. Default: <basename>.verilog.log");
00431     verilog_cmd->add_option("-c,--chain", chain_length,
00432         "Specify the chain length for verilog generation. Default: 1");
00433     verilog_cmd->add_option("--cells", cell_names, "Specify the cell names to generate Verilog for");
00434     verilog_cmd->callback([&] {
00435         printInfo();
00436         // Check if multi files
00437         if (library_paths.size() > 1) {
00438             spdlog::info("Running multi-threaded Verilog generation for {} files.", library_paths.size());
00439             spdlog::info("Each library will write to its own log file.");
00440
00441             // Sequential json file check
00442             for (const auto &library_path : library_paths) {
00443                 if (!std::filesystem::exists(std::filesystem::path(library_path).stem().string() + ".json")) {
00444                     spdlog::info("JSON file not found for '{}'. Parsing Liberty file first.", library_path);
00445                     parseLibFile(library_path, log_file_name = "");
00446                 }
00447             }
00448             spdlog::info("All JSON files prepared.");
00449             // Parallel Verilog generation
00450             std::vector<std::thread> threads;
00451             for (const auto &library_path : library_paths) {
00452                 threads.emplace_back(verilogLibFile, library_path, log_file_name = "", chain_length,
cell_names);
00453             }
00454             for (auto &thread : threads) {
00455                 thread.join();
00456             }
00457             spdlog::info("All threads completed.");
00458         } else {
00459             verilogLibFile(library_paths[0], log_file_name, chain_length, cell_names);
00460         }
00461     });
00462
00463     // Add subcommand for SPICE generation
00464     CLI::App *spice_cmd = app.add_subcommand("spice", "Generate SPICE file for given Liberty file");
00465     spice_cmd->add_option("library_path", library_paths, "Specify the library file to process")
00466         ->check(CLI::ExistingFile)
00467         ->required();
00468     spice_cmd->add_option("-l,--log", log_file_name,
00469         "Specify the log file name. Default: <basename>.spice.log");
00470     spice_cmd->add_option("--cells", cell_names, "Specify the cell names to generate SPICE for");
00471     spice_cmd->callback([&] {
00472         printInfo();
00473         // Check if multi files

```

```

00474     if (library_paths.size() > 1) {
00475         spdlog::info("Running multi-threaded SPICE generation for {} files.", library_paths.size());
00476         spdlog::info("Each library will write to its own log file.");
00477
00478         // Sequential json file check
00479         for (const auto &library_path : library_paths) {
00480             if (!std::filesystem::exists(std::filesystem::path(library_path).stem().string() + ".json")) {
00481                 spdlog::info("JSON file not found for '{}'. Parsing Liberty file first.", library_path);
00482                 parseLibFile(library_path, log_file_name = "");
00483             }
00484         }
00485         spdlog::info("All JSON files prepared.");
00486         // Parallel SPICE generation
00487         std::vector<std::thread> threads;
00488         for (const auto &library_path : library_paths) {
00489             // threads.emplace_back(spiceLibFile, library_path, log_file_name = "", cell_names);
00490         }
00491         for (auto &thread : threads) {
00492             thread.join();
00493         }
00494         spdlog::info("All threads completed.");
00495     } else {
00496         // spiceLibFile(library_paths[0], log_file_name, cell_names);
00497     }
00498 });
00499
00500 // Add subcommand for funtional equivalence check
00501 CLI::App *func_cmd = app.add_subcommand("func", "Check functional equivalence of two Liberty files
or Verilog files");
00502 std::string ref_file, comp_file;
00503 func_cmd->add_option("--ref", ref_file, "Specify the reference Liberty or Verilog file")
00504     ->check(CLI::ExistingFile)
00505     ->required();
00506 func_cmd->add_option("--comp", comp_file, "Specify the comparison Liberty or Verilog file")
00507     ->check(CLI::ExistingFile)
00508     ->required();
00509 func_cmd->add_option("--cells", cell_names, "Specify the cell names to check functional equivalence
for");
00510 func_cmd->callback([&] {
00511     printInfo();
00512     funcLibFile(ref_file, comp_file, cell_names);
00513 });
00514
00515 CLI11_PARSE(app, argc, argv);
00516
00517 // End of program
00518 char hostname[256];
00519 gethostname(hostname, sizeof(hostname));
00520
00521 auto now = std::chrono::system_clock::now();
00522 auto time_now = std::chrono::system_clock::to_time_t(now);
00523 std::stringstream ss;
00524 ss << std::put_time(std::localtime(&time_now), "%c");
00525
00526 spdlog::info("ZlibValidation exited on '{}' at {}", hostname, ss.str());
00527 return 0;
00528 }

```

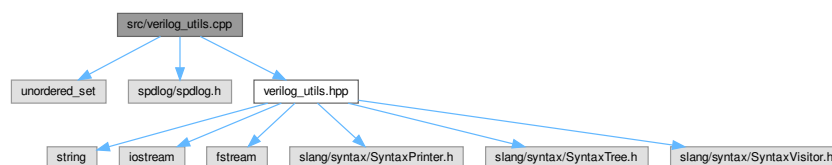
6.34 src/verilog_utils.cpp File Reference

```

#include <unordered_set>
#include "spdlog/spdlog.h"
#include "verilog_utils.hpp"

```

Include dependency graph for verilog_utils.cpp:



Functions

- void [getAST](#) (const std::string &verilog_file, const std::string &cell)

6.34.1 Function Documentation

6.34.1.1 getAST()

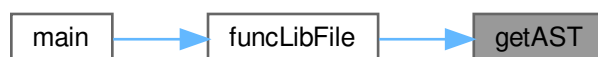
```
void getAST (
    const std::string & verilog_file,
    const std::string & cell )
```

Definition at line [397](#) of file [verilog_utils.cpp](#).

Here is the call graph for this function:



Here is the caller graph for this function:



6.35 verilog_utils.cpp

[Go to the documentation of this file.](#)

```
00001 #include <unordered_set>
00002
00003 #include "spdlog/spdlog.h"
00004
00005 #include "verilog_utils.hpp"
00006
00007 // Creating custom visitor class
00008 VerilogVisitor::VerilogVisitor(const std::string &targetCell) : targetCell_(targetCell), depth_(0) {}
00009
00010 // Adding a generic handler method to record all visited nodes and increment the depth counter
00011 void VerilogVisitor::handle(const slang::syntax::SyntaxNode &node) {
```

```

00012     std::string indent(depth_ * 2, ' ');
00013     spdlog::debug("{}Node: {}", indent, slang::syntax::toString(node.kind));
00014
00015     // Increase depth, visit child nodes, then decrease depth
00016     depth_++;
00017     visitDefault(node);
00018     depth_--;
00019 }
00020
00021 // Handle module declarations
00022 void VerilogVisitor::handle(const slang::syntax::ModuleDeclarationSyntax &module) {
00023     std::string indent(depth_ * 2, ' ');
00024
00025     if (module.header && module.header->name.valid()) {
00026         std::string_view moduleName = module.header->name.valueText();
00027         spdlog::info("{}Module Name:{}", indent, moduleName);
00028
00029         inTargetModule_ = !targetCell_.empty() && moduleName == targetCell_;
00030         if (!inTargetModule_) {
00031             return;
00032         }
00033
00034         // If a specific module name is specified, check if it matches
00035         if (inTargetModule_) {
00036             spdlog::info("{}Found target module: {}", indent, targetCell_);
00037             // Print the module ports
00038             if (module.header->ports) {
00039                 spdlog::info("{}Ports: {}", indent, module.header->ports->toString());
00040             }
00041         }
00042     }
00043
00044     // Continue processing child nodes
00045     depth_++;
00046     visitDefault(module);
00047     depth_--;
00048 }
00049
00050 // Handle port declarations
00051 void VerilogVisitor::handle(const slang::syntax::PortDeclarationSyntax &portDecl) {
00052     if (!inTargetModule_) {
00053         return;
00054     }
00055
00056     std::string indent = std::string(depth_ * 2, ' ');
00057
00058     // Get port direction
00059     std::string direction = "unknown";
00060     if (portDecl.header) {
00061         // Try to convert the header to different types of port headers
00062         if (auto *varPort = portDecl.header->as_if<slang::syntax::VariablePortHeaderSyntax>()) {
00063             if (varPort->direction) {
00064                 direction = varPort->direction.valueText();
00065             }
00066         } else if (auto *netPort = portDecl.header->as_if<slang::syntax::NetPortHeaderSyntax>()) {
00067             if (netPort->direction) {
00068                 direction = netPort->direction.valueText();
00069             }
00070         }
00071     }
00072
00073     // Get port name
00074     for (const auto &declarator : portDecl.declarators) {
00075         if (declarator->name.valid()) {
00076             std::string_view portName = declarator->name.valueText();
00077             spdlog::info("{}Port Name: {} ({})", indent, portName, direction);
00078         }
00079     }
00080 }
00081
00082 // Handle hierarchical instantiation (module instance)
00083 void VerilogVisitor::handle(const slang::syntax::HierarchyInstantiationSyntax &hierarchyInst) {
00084     if (!inTargetModule_) {
00085         return;
00086     }
00087
00088     std::string indent(depth_ * 2, ' ');
00089
00090     if (hierarchyInst.type.valid()) {
00091         std::string_view instType = hierarchyInst.type.valueText();
00092         spdlog::info("{}Hierarchy Instance Type: {}", indent, instType);
00093
00094         // Safely print parameter values (if any)
00095         try {
00096             if (hierarchyInst.parameters) {
00097                 spdlog::info("{}Parameters:", indent);
00098                 for (const auto &param : hierarchyInst.parameters->parameters) {

```

```

00099         if (!param)
00100             continue; // Null pointer check
00101
00102         if (auto *orderedParam = param->as_if<slang::syntax::OrderedParamAssignmentSyntax>()) {
00103             if (orderedParam->expr) {
00104                 spdlog::info("{} Parameter: {}", indent, orderedParam->expr->toString());
00105             }
00106         } else if (auto *namedParam = param->as_if<slang::syntax::NamedParamAssignmentSyntax>()) {
00107             if (namedParam->name.valid() && namedParam->expr) {
00108                 spdlog::info("{} Parameter {}: {}", indent, namedParam->name.valueText(),
00109                             namedParam->expr->toString());
00110             }
00111         }
00112     }
00113 }
00114 } catch (const std::exception &e) {
00115     spdlog::warn("{}Exception while processing parameters: {}", indent, e.what());
00116 }
00117
00118 // Safely handle instances
00119 try {
00120     for (const auto &instance : hierarchyInst.instances) {
00121         if (!instance) {
00122             spdlog::warn("{}Invalid instance", indent);
00123             continue;
00124         } else if (!instance->decl) {
00125             spdlog::warn("{}Invalid instance declaration", indent);
00126             // continue;
00127         } else {
00128             try {
00129                 if (instance->decl->name.valid()) {
00130                     std::string_view instName = instance->decl->name.valueText();
00131                     spdlog::info("{}Instance Name: {}", indent, instName);
00132                 }
00133             } catch (...) {
00134                 spdlog::debug("{}Error printing name", indent);
00135             }
00136
00137             // Safely print dimensions
00138             try {
00139                 if (!instance->decl->dimensions.empty()) {
00140                     spdlog::info("{} Dimensions: {}", indent, instance->decl->dimensions.toString());
00141                 }
00142             } catch (...) {
00143                 spdlog::debug("{}Error printing dimensions", indent);
00144             }
00145         }
00146     }
00147
00148     // Safely print port connections
00149     spdlog::info("{} Port connections:", indent);
00150     for (const auto &conn : instance->connections) {
00151         if (!conn) {
00152             spdlog::warn("{} Null connection", indent);
00153             continue; // Null pointer check
00154         }
00155         try {
00156             // Use as_if method for safe type conversion
00157             if (auto *ordered = conn->as_if<slang::syntax::OrderedPortConnectionSyntax>()) {
00158                 if (ordered->expr) {
00159                     spdlog::info("{} Ordered connection: {}", indent, ordered->expr->toString());
00160                 } else {
00161                     spdlog::info("{} Ordered connection: <empty>", indent);
00162                 }
00163             } else if (auto *named = conn->as_if<slang::syntax::NamedPortConnectionSyntax>()) {
00164                 if (named->name.valid()) {
00165                     std::string exprStr = named->expr ? named->expr->toString() : "<empty>";
00166                     spdlog::info("{} .{}({})", indent, named->name.valueText(), exprStr);
00167                 }
00168             } else if (conn->as_if<slang::syntax::EmptyPortConnectionSyntax>()) {
00169                 spdlog::info("{} <empty connection>", indent);
00170             } else if (conn->as_if<slang::syntax::WildcardPortConnectionSyntax>()) {
00171                 spdlog::info("{} .* (wildcard connection)", indent);
00172             } else {
00173                 spdlog::info("{} Unknown connection type: {}", indent,
00174                             slang::syntax::toString(conn->kind));
00175             }
00176         } catch (const std::exception &e) {
00177             spdlog::warn("{}Exception processing connection: {}", indent, e.what());
00178         } catch (...) {
00179             spdlog::warn("{}Unknown exception processing connection", indent);
00180         }
00181     }
00182 }
00183 } catch (const std::exception &e) {
00184     spdlog::warn("{}Exception while processing instances: {}", indent, e.what());
00185 }

```

```

00186     }
00187
00188     // Continue processing child nodes
00189     depth_++;
00190     visitDefault(hierarchyInst);
00191     depth--;
00192 }
00193 }
00194
00195 // Handle primitive gate instantiation
00196 void VerilogVisitor::handle(const slang::syntax::PrimitiveInstantiationSyntax &primitiveInst) {
00197     if (!inTargetModule_) {
00198         return;
00199     }
00200
00201     std::string indent(depth_ * 2, ' ');
00202
00203     // Get primitive gate type
00204     if (primitiveInst.type.valid()) {
00205         std::string_view gateType = primitiveInst.type.valueText();
00206         spdlog::info("{}Primitive Gate: {}", indent, gateType);
00207
00208         // Print delay information (if any)
00209         if (primitiveInst.delay) {
00210             spdlog::info("{}Delay: {}", indent, primitiveInst.delay->toString());
00211         }
00212
00213         // Print strength information (if any)
00214         if (primitiveInst.strength) {
00215             spdlog::info("{}Strength: {}", indent, primitiveInst.strength->toString());
00216         }
00217
00218         // Print each instance
00219         for (const auto &instance : primitiveInst.instances) {
00220             // Primitive gates usually don't have names, but if they do, print them
00221             if (instance->decl && instance->decl->name.valid()) {
00222                 std::string_view instName = instance->decl->name.valueText();
00223                 spdlog::info("{} Gate instance: {}", indent, instName);
00224             }
00225
00226             // Print connections
00227             spdlog::info("{} Gate connections:", indent);
00228             for (size_t i = 0; i < instance->connections.size(); ++i) {
00229                 const auto &conn = instance->connections[i];
00230                 // For gate-level instantiation, connections are usually ordered
00231                 if (auto *ordered = conn->as_if<slang::syntax::OrderedPortConnectionSyntax>()) {
00232                     if (ordered->expr) {
00233                         // The first is usually the output, the rest are inputs
00234                         std::string portType = (i == 0) ? "output" : "input";
00235                         spdlog::info("{} {} {}: {}", indent, portType, i, ordered->expr->toString());
00236                     }
00237                 }
00238             }
00239         }
00240
00241         // Continue to traverse deeper when specific standard gate type
00242         // Create a set of safe standard gate types
00243         static const std::unordered_set<std::string_view> safeGateTypes = {
00244             "and", "or", "nand", "nor", "xor", "xnor", "not", "buf",
"bufif0",
00245             "bufif1", "notif0", "notif1", "pullup", "pulldown", "cmos", "rcmos", "nmos", "pmos",
"rnmos", "rpnmos", "tran", "rtran", "tranif0", "tranif1", "rtranif0", "rtranif1"};
00246
00247         if (safeGateTypes.find(gateType) != safeGateTypes.end()) {
00248             // Continue processing child nodes
00249             depth_++;
00250             visitDefault(primitiveInst);
00251             depth--;
00252         } else {
00253             spdlog::warn("{}Skipping deeper traversal of primitive: {}", indent, gateType);
00254         }
00255     } else {
00256         spdlog::warn("{}Primitive instantiation missing gate type", indent);
00257     }
00258 }
00259 }
00260 }
00261
00262 // Handle specify block
00263 void VerilogVisitor::handle(const slang::syntax::SpecifyBlockSyntax &specifyBlock) {
00264     if (!inTargetModule_) {
00265         return;
00266     }
00267
00268     std::string indent(depth_ * 2, ' ');
00269     spdlog::info("{}Specify Block:", indent);
00270
00271     // Iterate through all path declarations

```

```

00272     for (const auto &item : specifyBlock.items) {
00273         if (auto *pathDecl = item->as_if<slang::syntax::PathDeclarationSyntax>()) {
00274             if (pathDecl->desc) {
00275                 std::string pathSrc;
00276                 std::string pathDst;
00277
00278                 // Get path source
00279                 if (!pathDecl->desc->inputs.empty()) {
00280                     // Get the first input
00281                     if (auto *identifier =
00282                         pathDecl->desc->inputs[0]->as_if<slang::syntax::IdentifierNameSyntax>()) {
00283                         if (identifier->identifier.valid()) {
00284                             pathSrc = identifier->identifier.valueText();
00285                         }
00286                     }
00287                 }
00288
00289                 // Get path destination
00290                 if (pathDecl->desc->suffix) {
00291                     if (auto *simpleSuffix =
00292                         pathDecl->desc->suffix->as_if<slang::syntax::SimplePathSuffixSyntax>()) {
00293                         if (!simpleSuffix->outputs.empty()) {
00294                             if (auto *identifier =
00295                                 simpleSuffix->outputs[0]->as_if<slang::syntax::IdentifierNameSyntax>()) {
00296                                 if (identifier->identifier.valid()) {
00297                                     pathDst = identifier->identifier.valueText();
00298                                 }
00299                             }
00300                         } else if (auto *edgeSuffix = pathDecl->desc->suffix
00301                             ->as_if<slang::syntax::EdgeSensitivePathSuffixSyntax>()) {
00302                             if (!edgeSuffix->outputs.empty()) {
00303                                 if (auto *identifier =
00304                                     edgeSuffix->outputs[0]->as_if<slang::syntax::IdentifierNameSyntax>()) {
00305                                     if (identifier->identifier.valid()) {
00306                                         pathDst = identifier->identifier.valueText();
00307                                     }
00308                                 }
00309                             }
00310                         }
00311                     }
00312
00313                     // Construct path string
00314                     std::string pathStr = pathSrc + " => " + pathDst;
00315
00316                     // Get path delay
00317                     std::string delayStr = "(";
00318                     for (size_t i = 0; i < pathDecl->delays.size(); ++i) {
00319                         if (i > 0)
00320                             delayStr += ", ";
00321                         delayStr += pathDecl->delays[i]->toString();
00322                     }
00323                     delayStr += ")";
00324
00325                     spdlog::info("{} Path: {} = {}", indent, pathStr, delayStr);
00326                 }
00327             }
00328             // Can add handling for ConditionalPathDeclarationSyntax and IfNonePathDeclarationSyntax
00329             else if (auto *condPath = item->as_if<slang::syntax::ConditionalPathDeclarationSyntax>()) {
00330                 if (condPath->path && condPath->predicate) {
00331                     spdlog::info("{} Conditional Path (if {})", indent, condPath->predicate->toString());
00332                     // Recursively handle internal path
00333                     handle(*condPath->path);
00334                 }
00335             } else if (auto *ifNonePath = item->as_if<slang::syntax::IfNonePathDeclarationSyntax>()) {
00336                 if (ifNonePath->path) {
00337                     spdlog::info("{} If-None Path", indent);
00338                     // Recursively handle internal path
00339                     handle(*ifNonePath->path);
00340                 }
00341             }
00342         }
00343
00344         // Continue processing child nodes
00345         depth_++;
00346         visitDefault(specifyBlock);
00347         depth_--;
00348     }
00349 }
00350
00351 // Creating a Rewriter to extract a specific cell
00352 CellExtractor::CellExtractor(const std::string &targetCell) : targetCell_(targetCell),
00353     foundTarget_(false) {}
00354
00355 // Handle module declarations, only keep the target module
00356 void CellExtractor::handle(const slang::syntax::ModuleDeclarationSyntax &module) {
00357     if (module.header && module.header->name.valid()) {

```

```

00357     std::string_view moduleName = module.header->name.valueText();
00358
00359     // If it is the target module, mark as found, otherwise remove it
00360     if (!targetCell_.empty() && moduleName == targetCell_) {
00361         foundTarget_ = true;
00362         // Do not modify, keep this module
00363     } else {
00364         // Remove non-target modules
00365         remove(module);
00366     }
00367 }
00368 }
00369
00370 // Get result
00371 bool CellExtractor::foundTargetCell()const { return foundTarget_; }
00372
00373 CellPrinter::CellPrinter(const std::string &targetCell, std::ostream &out)
00374 : targetCell_(targetCell), out_(out), foundTarget_(false) {}
00375
00376 // Handle module declarations
00377 void CellPrinter::handle(const slang::syntax::ModuleDeclarationSyntax &module) {
00378     if (module.header && module.header->name.valid()) {
00379         std::string_view moduleName = module.header->name.valueText();
00380
00381         // Check if it is the target module
00382         if (!targetCell_.empty() && moduleName == targetCell_) {
00383             foundTarget_ = true;
00384
00385             // Print module definition
00386             out_ << "timescale 1ns/10ps\n";
00387             out_ << module.toString();
00388
00389             return; // Do not traverse further
00390         }
00391     }
00392
00393     // Continue traversing other modules
00394     visitDefault(module);
00395 }
00396
00397 void getAST(const std::string &verilog_file, const std::string &cell) {
00398     try {
00399         spdlog::info("Starting get AST from Verilog file: '{}'", verilog_file);
00400         auto result = slang::syntax::SyntaxTree::fromFile(verilog_file);
00401         if (result) {
00402             spdlog::info("Successfully parsed Verilog file.");
00403
00404             try {
00405                 // First, use the visitor to print basic information
00406                 VerilogVisitor visitor(cell);
00407                 visitor.visit(result.value()->root());
00408
00409                 // If a target cell is specified, use the rewriter to extract the relevant code
00410                 if (!cell.empty()) {
00411                     // Create a rewriter to only keep the target cell related code
00412                     CellExtractor extractor(cell);
00413                     auto extractedTree = extractor.transform(result.value());
00414
00415                     if (extractor.foundTargetCell()) {
00416                         // Use SyntaxPrinter to print the extracted code
00417                         std::string extractedCode = slang::syntax::SyntaxPrinter::printFile(*extractedTree);
00418
00419                         // Save to a file named after the cell name
00420                         std::string outputFile = cell + ".v";
00421                         std::ofstream cellOut(outputFile);
00422                         if (cellOut) {
00423                             cellOut << extractedCode;
00424                             cellOut.close();
00425                             spdlog::info("Extracted '{}' cell code to '{}'", cell, outputFile);
00426                         } else {
00427                             spdlog::error("Failed to write extracted cell code to '{}'", outputFile);
00428                         }
00429                     } else {
00430                         spdlog::warn("Target cell '{}' not found in the Verilog file", cell);
00431                     }
00432                 }
00433
00434                 spdlog::info("Print full source code to 'full_source_code.v'");
00435                 // Optionally save the entire syntax tree
00436                 std::string fullOutput = slang::syntax::SyntaxPrinter::printFile(*result.value());
00437                 std::ofstream out("full_source_code.v");
00438                 out << fullOutput;
00439                 out.close();
00440
00441                 spdlog::info("Print target cell code to 'cell_code.v'");
00442                 // Use CellPrinter to print the target cell's code
00443                 std::ofstream cellOut("cell_code.v");

```

```
00444         CellPrinter cellPrinter(cell, cellOut);
00445         cellPrinter.visit(result.value()->root());
00446         cellOut.close();
00447
00448         } catch (const std::exception &e) {
00449             spdlog::error("Exception during AST traversal: {}", e.what());
00450         } catch (...) {
00451             spdlog::error("Unknown exception during AST traversal");
00452         }
00453     } else {
00454         spdlog::error("Error parsing Verilog file.");
00455     }
00456 } catch (const std::exception &e) {
00457     spdlog::error("Exception during Verilog parsing: {}", e.what());
00458 } catch (...) {
00459     spdlog::error("Unknown exception during Verilog parsing");
00460 }
00461 }
```


Index

- ~AttributesIterator
 - AttributesIterator, [14](#)
- ~GroupsIterator
 - GroupsIterator, [22](#)
- ~LibAttribute
 - LibAttribute, [26](#)
- ~LibFile
 - LibFile, [32](#)
- ~LibGroup
 - LibGroup, [47](#)
- ~ValuesIterator
 - ValuesIterator, [62](#)
- abstol_
 - LibraryComparator, [57](#)
- APP_AUTHOR
 - version.h, [118](#)
- APP_CONTACT
 - version.h, [118](#)
- APP_NAME
 - version.h, [119](#)
- APP_VERSION
 - version.h, [119](#)
- APP_VERSION_MAJOR
 - version.h, [119](#)
- APP_VERSION_MINOR
 - version.h, [119](#)
- APP_VERSION_PATCH
 - version.h, [119](#)
- attr_
 - AttributesIterator, [16](#)
 - LibAttribute, [29](#)
- AttributesIterator, [13](#)
 - ~AttributesIterator, [14](#)
 - attr_, [16](#)
 - AttributesIterator, [14](#)
 - attrs_, [16](#)
 - end, [14](#)
 - err_, [16](#)
 - get, [15](#)
 - next, [15](#)
- attrs_
 - AttributesIterator, [16](#)
- b
 - si2drExprT, [59](#)
- basename_
 - LibFile, [43](#)
- bool_
 - ValuesIterator, [63](#)
- BUILD_TIMESTAMP
 - version.h, [119](#)
- CellExtractor, [16](#)
 - CellExtractor, [17](#)
 - foundTarget_, [18](#)
 - foundTargetCell, [17](#)
 - handle, [18](#)
 - targetCell_, [18](#)
- CellPrinter, [19](#)
 - CellPrinter, [20](#)
 - foundTarget_, [20](#)
 - handle, [20](#)
 - out_, [20](#)
 - targetCell_, [20](#)
- checkTimingArcMonotonicity
 - LibFile, [32](#)
- comp_json_
 - LibraryComparator, [57](#)
- comp_lib_path_
 - LibraryComparator, [57](#)
- compare.hpp
 - json, [70](#)
- compareCell
 - LibraryComparator, [52](#)
- compareLibFiles
 - main.cpp, [149](#)
- compareLut
 - LibraryComparator, [53](#)
- comparePin
 - LibraryComparator, [54](#)
- compareTimingArc
 - LibraryComparator, [55](#)
- d
 - si2drExprT, [59](#)
- depth_
 - VerilogVisitor, [68](#)
- dim_sizes
 - liberty_value_data, [30](#)
- dimensions
 - liberty_value_data, [30](#)
- end
 - AttributesIterator, [14](#)
 - GroupsIterator, [22](#)
 - ValuesIterator, [63](#)
- err_
 - AttributesIterator, [16](#)
 - GroupsIterator, [23](#)

- LibAttribute, 29
- LibFile, 44
- LibGroup, 49
- ValuesIterator, 64
- exprp_
 - ValuesIterator, 64
- filename_
 - LibFile, 44
- filepath_
 - LibFile, 44
- float_
 - ValuesIterator, 64
- foundTarget_
 - CellExtractor, 18
 - CellPrinter, 20
- foundTargetCell
 - CellExtractor, 17
- funcLibFile
 - main.cpp, 150
- generateCellJson
 - json_utils.cpp, 127
 - json_utils.hpp, 74
- generateLutJson
 - json_utils.cpp, 129
 - json_utils.hpp, 76
- generatePinJson
 - json_utils.cpp, 130
 - json_utils.hpp, 77
- generatePowerJson
 - json_utils.cpp, 133
 - json_utils.hpp, 80
- generateReport
 - LibraryComparator, 56
- generateTimingJson
 - json_utils.cpp, 134
- get
 - AttributesIterator, 15
 - GroupsIterator, 22
- get_function
 - si2dr_liberty.h, 100
- getAST
 - verilog_utils.cpp, 165
 - verilog_utils.hpp, 116
- getAttrs
 - LibGroup, 47
- getBoolean
 - LibAttribute, 26
- getFloat
 - LibAttribute, 26
- getGroups
 - LibGroup, 47
- getInt
 - LibAttribute, 27
- getName
 - LibAttribute, 27
 - LibGroup, 48
- getString
 - LibAttribute, 28
- getType
 - LibGroup, 48
- getValues
 - LibAttribute, 28
- group_
 - GroupsIterator, 23
 - LibGroup, 49
- groups_
 - GroupsIterator, 24
- GroupsIterator, 21
 - ~GroupsIterator, 22
 - end, 22
 - err_, 23
 - get, 22
 - group_, 23
 - groups_, 24
 - GroupsIterator, 22
 - next, 23
- handle
 - CellExtractor, 18
 - CellPrinter, 20
 - VerilogVisitor, 66, 67
- i
 - si2drExprT, 59
- include/compare.hpp, 69, 70
- include/iterators.hpp, 71
- include/json_utils.hpp, 72, 81
- include/lib_attribute.hpp, 82, 83
- include/lib_file.hpp, 83, 84
- include/lib_group.hpp, 85, 86
- include/si2dr_liberty.h, 86, 108
- include/verilog_utils.hpp, 115, 117
- include/version.h, 118, 120
- index_info
 - liberty_value_data, 30
- int_
 - ValuesIterator, 64
- inTargetModule_
 - VerilogVisitor, 68
- isComplex
 - LibAttribute, 29
- json
 - compare.hpp, 70
 - json_utils.hpp, 74
 - lib_file.hpp, 84
- json_utils.cpp
 - generateCellJson, 127
 - generateLutJson, 129
 - generatePinJson, 130
 - generatePowerJson, 133
 - generateTimingJson, 134
 - parseStringToVector, 136
- json_utils.hpp
 - generateCellJson, 74
 - generateLutJson, 76

- generatePinJson, 77
 - generatePowerJson, 80
 - json, 74
- jsonname_
 - LibFile, 44
- left
 - si2drExprT, 59
- lib_file.hpp
 - json, 84
- lib_json_
 - LibFile, 44
- LibAttribute, 24
 - ~LibAttribute, 26
 - attr_, 29
 - err_, 29
 - getBoolean, 26
 - getFloat, 26
 - getInt, 27
 - getName, 27
 - getString, 28
 - getValues, 28
 - isComplex, 29
 - LibAttribute, 25
- liberty_destroy_value_data
 - si2dr_liberty.h, 100
- liberty_get_element
 - si2dr_liberty.h, 101
- liberty_get_values_data
 - si2dr_liberty.h, 101
- liberty_value_data, 30
 - dim_sizes, 30
 - dimensions, 30
 - index_info, 30
 - values, 31
- LibFile, 31
 - ~LibFile, 32
 - basename_, 43
 - checkTimingArcMonotonicity, 32
 - err_, 44
 - filename_, 44
 - filepath_, 44
 - jsonname_, 44
 - lib_json_, 44
 - LibFile, 32
 - libname_, 44
 - logger_, 45
 - loggername_, 45
 - modify, 33
 - mono, 33
 - parse, 36
 - process_, 45
 - read, 38
 - supercell, 39
 - temperature_, 45
 - verilog, 41
 - voltage_, 45
 - writeJsonToFile, 43
- LibGroup, 46
 - ~LibGroup, 47
 - err_, 49
 - getAttrs, 47
 - getGroups, 47
 - getName, 48
 - getType, 48
 - group_, 49
 - LibGroup, 46
- libname_
 - LibFile, 44
- LibraryComparator, 50
 - abstol_, 57
 - comp_json_, 57
 - comp_lib_path_, 57
 - compareCell, 52
 - compareLut, 53
 - comparePin, 54
 - compareTimingArc, 55
 - generateReport, 56
 - LibraryComparator, 50
 - ref_json_, 58
 - ref_lib_path_, 58
 - reitol_, 58
- logger_
 - LibFile, 45
- loggername_
 - LibFile, 45
- LONG_DOUBLE
 - si2dr_liberty.h, 90
- main
 - main.cpp, 150
- main.cpp
 - compareLibFiles, 149
 - funcLibFile, 150
 - main, 150
 - monoCheckLibFile, 151
 - parseLibFile, 153
 - printInfo, 155
 - supercellLibFile, 155
 - verilogLibFile, 157
- main_liberty_parser
 - si2dr_liberty.h, 101
- modify
 - LibFile, 33
- mono
 - LibFile, 33
- monoCheckLibFile
 - main.cpp, 151
- next
 - AttributesIterator, 15
 - GroupsIterator, 23
 - ValuesIterator, 63
- out_
 - CellPrinter, 20
- parse

- LibFile, 36
- parseLibFile
 - main.cpp, 153
- parseStringToVector
 - json_utils.cpp, 136
- print_version
 - si2dr_liberty.h, 101
- printInfo
 - main.cpp, 155
- process_
 - LibFile, 45
- read
 - LibFile, 38
- README.md, 120
- ref_json_
 - LibraryComparator, 58
- ref_lib_path_
 - LibraryComparator, 58
- reltol_
 - LibraryComparator, 58
- right
 - si2drExprT, 60
- s
 - si2drExprT, 60
- SI2_ARGS
 - si2dr_liberty.h, 90, 101–107
- SI2_BOOLEAN
 - si2dr_liberty.h, 100
- SI2_DOUBLE
 - si2dr_liberty.h, 100
- SI2_EXPR
 - si2dr_liberty.h, 100
- SI2_FALSE
 - si2dr_liberty.h, 98
- SI2_FLOAT
 - si2dr_liberty.h, 100
- SI2_ITER
 - si2dr_liberty.h, 100
- SI2_LONG
 - si2dr_liberty.h, 100
- SI2_LONG_MAX
 - si2dr_liberty.h, 90
- SI2_LONG_MIN
 - si2dr_liberty.h, 90
- SI2_MAX_VALUETYPE
 - si2dr_liberty.h, 100
- SI2_OID
 - si2dr_liberty.h, 100
- SI2_STRING
 - si2dr_liberty.h, 100
- SI2_TRUE
 - si2dr_liberty.h, 98
- SI2_ULONG_MAX
 - si2dr_liberty.h, 90
- SI2_UNDEFINED_VALUETYPE
 - si2dr_liberty.h, 100
- SI2_VOID
 - si2dr_liberty.h, 100
- si2BooleanEnum
 - si2dr_liberty.h, 97
- si2BooleanT
 - si2dr_liberty.h, 92
- si2DoubleT
 - si2dr_liberty.h, 92
- SI2DR_ATTR
 - si2dr_liberty.h, 99
- SI2DR_BOOLEAN
 - si2dr_liberty.h, 100
- SI2DR_COMPLEX
 - si2dr_liberty.h, 98
- SI2DR_COMPLEXVAL
 - si2dr_liberty.h, 99
- SI2DR_DEFINE
 - si2dr_liberty.h, 99
- SI2DR_EXPR
 - si2dr_liberty.h, 100
- SI2DR_EXPR_OP_ADD
 - si2dr_liberty.h, 99
- SI2DR_EXPR_OP_DIV
 - si2dr_liberty.h, 99
- SI2DR_EXPR_OP_EXP
 - si2dr_liberty.h, 99
- SI2DR_EXPR_OP_LOG10
 - si2dr_liberty.h, 99
- SI2DR_EXPR_OP_LOG2
 - si2dr_liberty.h, 99
- SI2DR_EXPR_OP_MUL
 - si2dr_liberty.h, 99
- SI2DR_EXPR_OP_PAREN
 - si2dr_liberty.h, 99
- SI2DR_EXPR_OP_SUB
 - si2dr_liberty.h, 99
- SI2DR_EXPR_VAL
 - si2dr_liberty.h, 99
- SI2DR_FALSE
 - si2dr_liberty.h, 91
- SI2DR_FLOAT64
 - si2dr_liberty.h, 100
- SI2DR_GROUP
 - si2dr_liberty.h, 99
- SI2DR_INT32
 - si2dr_liberty.h, 100
- SI2DR_INTERNAL_SYSTEM_ERROR
 - si2dr_liberty.h, 98
- SI2DR_INVALID_ATTRTYPE
 - si2dr_liberty.h, 98
- SI2DR_INVALID_NAME
 - si2dr_liberty.h, 98
- SI2DR_INVALID_OBJECTTYPE
 - si2dr_liberty.h, 98
- SI2DR_INVALID_VALUE
 - si2dr_liberty.h, 98
- si2dr_liberty.h
 - get_function, 100
 - liberty_destroy_value_data, 100

liberty_get_element, 101
liberty_get_values_data, 101
LONG_DOUBLE, 90
main_liberty_parser, 101
print_version, 101
SI2_ARGS, 90, 101–107
SI2_BOOLEAN, 100
SI2_DOUBLE, 100
SI2_EXPR, 100
SI2_FALSE, 98
SI2_FLOAT, 100
SI2_ITER, 100
SI2_LONG, 100
SI2_LONG_MAX, 90
SI2_LONG_MIN, 90
SI2_MAX_VALUETYPE, 100
SI2_OID, 100
SI2_STRING, 100
SI2_TRUE, 98
SI2_ULONG_MAX, 90
SI2_UNDEFINED_VALUETYPE, 100
SI2_VOID, 100
si2BooleanEnum, 97
si2BooleanT, 92
si2DoubleT, 92
SI2DR_ATTR, 99
SI2DR_BOOLEAN, 100
SI2DR_COMPLEX, 98
SI2DR_COMPLEXVAL, 99
SI2DR_DEFINE, 99
SI2DR_EXPR, 100
SI2DR_EXPR_OP_ADD, 99
SI2DR_EXPR_OP_DIV, 99
SI2DR_EXPR_OP_EXP, 99
SI2DR_EXPR_OP_LOG10, 99
SI2DR_EXPR_OP_LOG2, 99
SI2DR_EXPR_OP_MUL, 99
SI2DR_EXPR_OP_PAREN, 99
SI2DR_EXPR_OP_SUB, 99
SI2DR_EXPR_VAL, 99
SI2DR_FALSE, 91
SI2DR_FLOAT64, 100
SI2DR_GROUP, 99
SI2DR_INT32, 100
SI2DR_INTERNAL_SYSTEM_ERROR, 98
SI2DR_INVALID_ATTRTYPE, 98
SI2DR_INVALID_NAME, 98
SI2DR_INVALID_OBJECTTYPE, 98
SI2DR_INVALID_VALUE, 98
SI2DR_MAX_ERROR, 98
SI2DR_MAX_FLOAT32, 91
SI2DR_MAX_FLOAT64, 91
SI2DR_MAX_INT32, 91
SI2DR_MAX_OBJECTTYPE, 99
SI2DR_MAX_STRING_LEN, 91
SI2DR_MAX_VALUETYPE, 100
SI2DR_MIN_FLOAT32, 91
SI2DR_MIN_FLOAT64, 92
SI2DR_MIN_INT32, 92
SI2DR_NO_ERROR, 98
SI2DR_OBJECT_ALREADY_EXISTS, 98
SI2DR_OBJECT_NOT_FOUND, 98
SI2DR_PIINIT_NOT_CALLED, 98
SI2DR_REFERENCE_ERROR, 98
SI2DR_SEMANTIC_ERROR, 98
SI2DR_SEVERITY_ERR, 99
SI2DR_SEVERITY_NOTE, 99
SI2DR_SEVERITY_WARN, 99
SI2DR_SIMPLE, 98
SI2DR_STRING, 100
SI2DR_SYNTAX_ERROR, 98
SI2DR_TRACE_FILES_CANNOT_BE_OPENED, 98
SI2DR_TRUE, 92
SI2DR_UNDEFINED_OBJECTTYPE, 99
SI2DR_UNDEFINED_VALUETYPE, 100
SI2DR_UNUSABLE_OID, 98
si2drAttrComplexValldT, 92
si2drAttrldT, 93
si2drAttrslldT, 93
si2drAttrTypeT, 93, 98
si2drBooleanT, 93
si2drDefaultMessageHandler, 107
si2drDefineldT, 93
si2drDefinesldT, 93
si2drErrorT, 94, 98
si2drExprT, 94
si2drExprTypeT, 94, 98
si2drFloat32T, 94
si2drFloat64T, 94
si2drGroupldT, 94
si2drGroupsldT, 94
si2drInt32T, 95
si2drIrlldT, 95
si2drMessageHandlerT, 95
si2drNamesldT, 95
si2drObjectldT, 95
si2drObjectsldT, 95
si2drObjectTypeT, 96, 99
si2drPIGetMessageHandler, 107
si2drPISetMessageHandler, 107
si2drSeverityT, 96, 99
si2drStringT, 96
si2drValuesldT, 96
si2drValueTypeT, 96, 100
si2drVoidT, 96
si2FloatT, 96
si2IteratorldT, 97
si2LongT, 97
si2ObjectldT, 97
si2StringT, 97
si2TypeEnum, 100
si2TypeT, 97
si2VoidT, 97
usage, 108
SI2DR_MAX_ERROR

- si2dr_liberty.h, 98
- SI2DR_MAX_FLOAT32
 - si2dr_liberty.h, 91
- SI2DR_MAX_FLOAT64
 - si2dr_liberty.h, 91
- SI2DR_MAX_INT32
 - si2dr_liberty.h, 91
- SI2DR_MAX_OBJECTTYPE
 - si2dr_liberty.h, 99
- SI2DR_MAX_STRING_LEN
 - si2dr_liberty.h, 91
- SI2DR_MAX_VALUETYPE
 - si2dr_liberty.h, 100
- SI2DR_MIN_FLOAT32
 - si2dr_liberty.h, 91
- SI2DR_MIN_FLOAT64
 - si2dr_liberty.h, 92
- SI2DR_MIN_INT32
 - si2dr_liberty.h, 92
- SI2DR_NO_ERROR
 - si2dr_liberty.h, 98
- SI2DR_OBJECT_ALREADY_EXISTS
 - si2dr_liberty.h, 98
- SI2DR_OBJECT_NOT_FOUND
 - si2dr_liberty.h, 98
- SI2DR_PIINIT_NOT_CALLED
 - si2dr_liberty.h, 98
- SI2DR_REFERENCE_ERROR
 - si2dr_liberty.h, 98
- SI2DR_SEMANTIC_ERROR
 - si2dr_liberty.h, 98
- SI2DR_SEVERITY_ERR
 - si2dr_liberty.h, 99
- SI2DR_SEVERITY_NOTE
 - si2dr_liberty.h, 99
- SI2DR_SEVERITY_WARN
 - si2dr_liberty.h, 99
- SI2DR_SIMPLE
 - si2dr_liberty.h, 98
- SI2DR_STRING
 - si2dr_liberty.h, 100
- SI2DR_SYNTAX_ERROR
 - si2dr_liberty.h, 98
- SI2DR_TRACE_FILES_CANNOT_BE_OPENED
 - si2dr_liberty.h, 98
- SI2DR_TRUE
 - si2dr_liberty.h, 92
- SI2DR_UNDEFINED_OBJECTTYPE
 - si2dr_liberty.h, 99
- SI2DR_UNDEFINED_VALUETYPE
 - si2dr_liberty.h, 100
- SI2DR_UNUSABLE_OID
 - si2dr_liberty.h, 98
- si2drAttrComplexValIdT
 - si2dr_liberty.h, 92
- si2drAttrIdT
 - si2dr_liberty.h, 93
- si2drAttrslIdT
 - si2dr_liberty.h, 93
- si2drAttrTypeT
 - si2dr_liberty.h, 93, 98
- si2drBooleanT
 - si2dr_liberty.h, 93
- si2drDefaultMessageHandler
 - si2dr_liberty.h, 107
- si2drDefineIdT
 - si2dr_liberty.h, 93
- si2drDefinesIdT
 - si2dr_liberty.h, 93
- si2drErrorT
 - si2dr_liberty.h, 94, 98
- si2drExprT, 58
 - b, 59
 - d, 59
 - i, 59
 - left, 59
 - right, 60
 - s, 60
 - si2dr_liberty.h, 94
 - type, 60
 - u, 60
 - valuetype, 60
- si2drExprTypeT
 - si2dr_liberty.h, 94, 98
- si2drFloat32T
 - si2dr_liberty.h, 94
- si2drFloat64T
 - si2dr_liberty.h, 94
- si2drGroupIdT
 - si2dr_liberty.h, 94
- si2drGroupsIdT
 - si2dr_liberty.h, 94
- si2drInt32T
 - si2dr_liberty.h, 95
- si2drIterIdT
 - si2dr_liberty.h, 95
- si2drMessageHandlerT
 - si2dr_liberty.h, 95
- si2drNamesIdT
 - si2dr_liberty.h, 95
- si2drObjectIdT
 - si2dr_liberty.h, 95
- si2drObjectsIdT
 - si2dr_liberty.h, 95
- si2drObjectTypeT
 - si2dr_liberty.h, 96, 99
- si2drPIGetMessageHandler
 - si2dr_liberty.h, 107
- si2drPISetMessageHandler
 - si2dr_liberty.h, 107
- si2drSeverityT
 - si2dr_liberty.h, 96, 99
- si2drStringT
 - si2dr_liberty.h, 96
- si2drValuesIdT
 - si2dr_liberty.h, 96

- si2drValueTypeT
 - si2dr_liberty.h, 96, 100
- si2drVoidT
 - si2dr_liberty.h, 96
- si2FloatT
 - si2dr_liberty.h, 96
- si2IteratorIdT
 - si2dr_liberty.h, 97
- si2LongT
 - si2dr_liberty.h, 97
- si2ObjectIdT
 - si2dr_liberty.h, 97
- si2OID, 61
 - v1, 61
 - v2, 61
- si2StringT
 - si2dr_liberty.h, 97
- si2TypeEnum
 - si2dr_liberty.h, 100
- si2TypeT
 - si2dr_liberty.h, 97
- si2VoidT
 - si2dr_liberty.h, 97
- src/compare.cpp, 120, 121
- src/iterators.cpp, 125
- src/json_utils.cpp, 126, 137
- src/lib_attribute.cpp, 140
- src/lib_file.cpp, 141
- src/lib_group.cpp, 147
- src/main.cpp, 148, 159
- src/verilog_utils.cpp, 164, 165
- str_
 - ValuesIterator, 64
- supercell
 - LibFile, 39
- supercellLibFile
 - main.cpp, 155
- targetCell_
 - CellExtractor, 18
 - CellPrinter, 20
 - VerilogVisitor, 68
- temperature_
 - LibFile, 45
- type
 - si2drExprT, 60
- u
 - si2drExprT, 60
- usage
 - si2dr_liberty.h, 108
- v1
 - si2OID, 61
- v2
 - si2OID, 61
- values
 - liberty_value_data, 31
- values_
 - ValuesIterator, 64
- ValuesIterator, 61
 - ~ValuesIterator, 62
- bool_, 63
- end, 63
- err_, 64
- exprp_, 64
- float_, 64
- int_, 64
- next, 63
- str_, 64
- values_, 64
- ValuesIterator, 62
- vtype_, 65
- valuetype
 - si2drExprT, 60
- verilog
 - LibFile, 41
- verilog_utils.cpp
 - getAST, 165
- verilog_utils.hpp
 - getAST, 116
- verilogLibFile
 - main.cpp, 157
- VerilogVisitor, 65
 - depth_, 68
 - handle, 66, 67
 - inTargetModule_, 68
 - targetCell_, 68
 - VerilogVisitor, 66
- version.h
 - APP_AUTHOR, 118
 - APP_CONTACT, 118
 - APP_NAME, 119
 - APP_VERSION, 119
 - APP_VERSION_MAJOR, 119
 - APP_VERSION_MINOR, 119
 - APP_VERSION_PATCH, 119
 - BUILD_TIMESTAMP, 119
- voltage_
 - LibFile, 45
- vtype_
 - ValuesIterator, 65
- writeJsonToFile
 - LibFile, 43